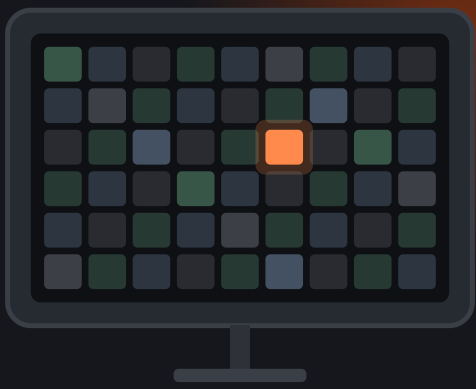
MANUAL DE CAMPO · LOTES 1 A 8 · PRÉVIA



Estado & Tela

Aplicações web reativas com React, do DOM manipulado à mão até uma aplicação completa, autenticada e testada, construída com agentes de IA sem perder o controle do código.

ORGANIZADO POR

Prof. Rogério Silva e Claude Opus 5.5

2026 · TEC.1052 · ADS · IFPI

React 19.3 · Vite 8 · TypeScript
React Router 8 · TanStack Query 5 · Vitest 5 · Playwright

Sumário

0 Introdução P. 3

PARTE I · A MUDANÇA DE PARADIGMA

1 Do DOM à mão às interfaces reativas P. 7

2 O primeiro projeto em vinte minutos P. 18

3 Como o React funciona por dentro P. 28

PARTE II · OS FUNDAMENTOS

4 Componentes, JSX e composição P. 41

5 Props, eventos e fluxo de dados P. 52

6 Estado, formulários e interações P. 62

7 Efeitos: sincronizar com o mundo lá fora P. 75

PARTE III · ARQUITETURA

8 Estrutura de projeto por funcionalidade P. 91

9 Estilo, tokens e acessibilidade P. 103

10 Componentes reutilizáveis e design system P. 115

11 Prop drilling e Context P. 127

12 Reducer e transições de estado P. 137

PARTE IV · A APLICAÇÃO COMPLETA

13 Rotas e navegação P. 148

14 Backend, HTTP e estado remoto P. 160

15 Autenticação, sessão e autorização na interface P. 173

16 Testes, depuração e desempenho P. 186

PARTE V · RENDERIZAÇÃO MODERNA

17 Servidor, cliente e a fronteira entre eles P. 198

18 Next.js: visão geral e próximo livro P. 210

PARTE VI · ENTREGA E REFERÊNCIA

19 A entrega com agentes, do plano ao PR LOTE 9

20 Guia de consulta, régua e glossário LOTE 9

ANTES DE COMEÇAR

00 Introdução

Para quem é este livro, a ideia que ele defende e como cada capítulo funciona.

Você já sabe fazer uma página reagir a um clique. Este livro trata do passo seguinte: fazer uma aplicação inteira continuar certa quando ela tiver trinta telas, várias pessoas no código e um agente de IA escrevendo boa parte dele.

É para quem já constrói páginas com HTML, CSS e JavaScript, manipula o DOM, consome uma API com `fetch` e já escreveu um backend REST com Express e TypeScript. Nada disso é ensinado de novo. Se você cursa TEC.1052, guarde por perto o front-end do Mini-Prontuário que escreveu em JavaScript puro: ele é o ponto de partida do capítulo 1.

> **A tela é uma função do estado. Quase todo bug de front-end é um estado guardado no lugar errado, duplicado ou sincronizado à mão.**

Por isso o livro não é uma lista de APIs do React. Cada capítulo responde, de um ângulo diferente, a uma pergunta: **onde este estado mora?**



Fig. 0.1 — Os quatro lugares onde um estado pode morar. Hooks, contexto, reducer, rotas e bibliotecas aparecem no livro como respostas a esta pergunta.

O laboratório

> H+ **O livro inteiro acontece num hospital**
>
> Ao lado de cada leito há um monitor. Ninguém redesenha a tela dele à mão: os sensores mudam um número e o monitor atualiza só a faixa que mudou. É o modelo mental do React, e o mundo de todas as analogias do livro.

O projeto é o cliente React do **Mini-Prontuário**, que conversa com a API do ebook *Crachá & Porta*: sessão em cookie, permissões por papel e uma suíte de ataques. A interface é nova; o servidor é real desde o primeiro dia. São três repositórios, `estado-e-tela-api`, `estado-e-tela-web` (uma tag git por capítulo) e `estado-e-tela-ebook`, com este livro, os roteiros de medição e as saídas. Todos os pacientes são fictícios.

Como cada capítulo funciona

A ordem é sempre a mesma: o **problema** num sintoma do Mini-Prontuário; o **modelo mental**, com a caixa *Onde a analogia para*; o mecanismo **por dentro** e **na prática**; o **erro clássico** e a **prova**, um teste que falha sem a correção; o trabalho **com agentes de IA**; e o **fechamento**, com exercícios em três níveis.

Este livro faz parte de uma série. O *Desenvolvimento Dirigido por IA*, o Dev de IA, ensina o método de trabalhar com agentes; aqui ele é aplicado ao React,

sem repetição. Quando um procedimento está detalhado lá, uma caixa **Ponte · Dev de IA** resume e aponta o capítulo. Os prompts seguem os formatos da disciplina:

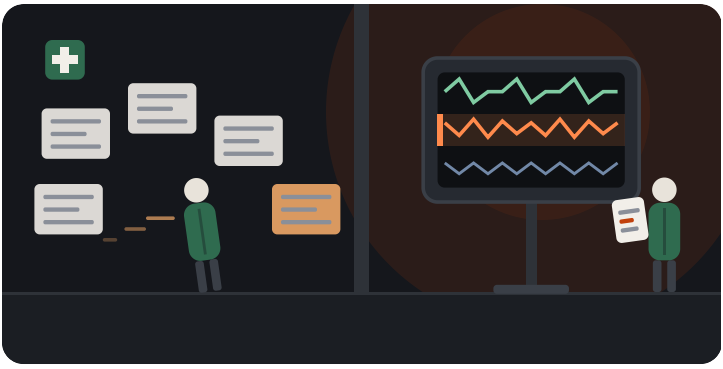
FORMATO	QUANDO	PARTES
Agente	a IA edita o projeto	Goal · Context · Constraints · Done when
C-P-T-R-F-A	conversa no chat	Contexto · Papel · Tarefa · Restrições · Formato · Aceitação
V.E.R.M.	antes de aceitar a resposta	Verifica · Explica · Reduz · Marca

O livro traz o prompt de agente; as variantes de cada capítulo ficam em `prompts/capNN.md`, no repositório da tela.

A régua e as convenções

O capítulo 20 lista **36 competências** de um desenvolvedor sênior de front-end, cada uma com a evidência exigida. Marcar 27 com evidência é chegar a **75%**. Cada capítulo diz quais itens entrega.

- **Dado** é fato externo, com fonte. **Medido** é saída real de código que rodamos, guardada em `saidas/`; nenhuma foi escrita à mão.
- Todo código impresso sai do git, na tag indicada na barra do bloco, em linhas de até 46 caracteres.



PARTE I · A MUDANÇA DE PARADIGMA

01

Do DOM à mão às interfaces reativas

O trabalho de manter a tela em dia com os dados não some quando você adota React. Ele muda de dono.

Terça, 14h. A enfermeira dá alta para Maria Souza, paciente de alto risco da Ala B. A linha some, o total cai de 6 para 5, e o painel continua dizendo: *alto risco: 2*.

Ninguém escreveu um erro; escreveram uma atualização a menos. A tela tinha três lugares que dependiam da mesma lista, e o código da alta lembrou de dois.

1.1

Como chegamos aqui

Em 2005, o AJAX deixou a página pedir dados sem recarregar. Em 2006, o jQuery tornou a manipulação do DOM o jeito padrão de programar a tela. Em 2010, Backbone e AngularJS popularizaram as aplicações de página única. Em 2013 veio o React, com uma ideia diferente: a pergunta deixou de ser *como alterar a tela* quando um dado muda e passou a ser *como a tela deve ficar* para um estado qualquer. Vue e Svelte seguiram a mesma ideia com outra mecânica, e em 2026 o React passou a ser mantido pela React Foundation, na Linux Foundation.

1.2

A Ala B em três versões

O laboratório está em `laboratorio/cap01`, na tag `cap01`: três páginas com a mesma tela, sem build. Seis pacientes fictícios, uma busca, um campo de anotação e um botão de alta por paciente, e um resumo com o total e quantos são de alto risco. Sirva a pasta com `npx serve laboratorio/cap01` e abra cada página.

Cada uma traz no topo um **medidor**, um `MutationObserver` que conta os elementos que entram e saem do DOM e os textos ou atributos trocados. É ele que transforma "o React é eficiente" em número.

1.3

Versão 1: você diz como mudar

A primeira versão é **imperativa**: o código dá ordens ao DOM, passo a passo. Na montagem ele preenche o total e o alto risco; na alta, remove a linha e corrige o total:

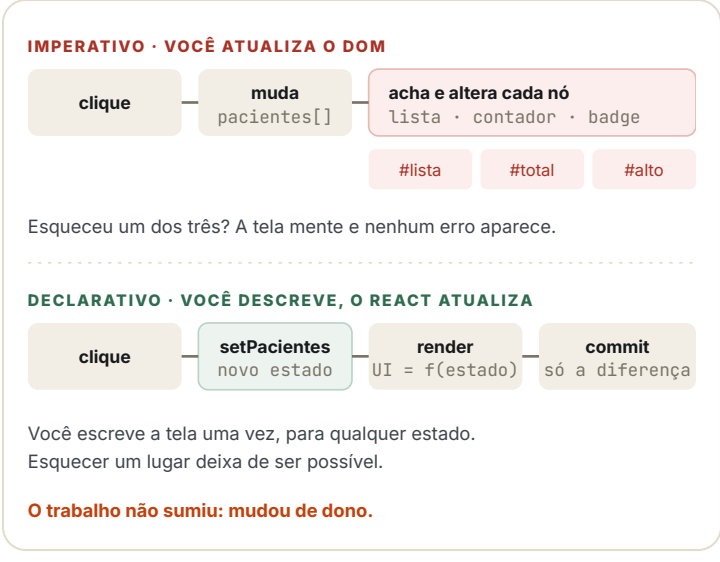


Fig. 1.1 — Quem atualiza a tela. No imperativo, cada evento precisa encontrar e alterar cada nó afetado. No declarativo, você descreve a tela para qualquer estado, e o React calcula o que mudar.

1-imperativo.html · alta cap01

```
// alta: tira a linha e corrige o total
lista.addEventListener("click", (e) => {
  if (e.target.tagName !== "BUTTON") return;
  const li = e.target.closest("li");
  pacientes = pacientes
    .filter((p) => p.id !== li.dataset.id);
  li.remove();
  total.textContent = pacientes.length;
});
```

O código precisa saber quais nós existem, quais dependem de `pacientes` e em que ordem atualizá-los. Com três dependências já houve esquecimento; uma tela real tem dezenas.

1.4

Versão 2: estado e `render()`

A segunda versão é a que a disciplina ensinou: um objeto `estado` e uma função que redesenha a tela inteira a partir dele.

2-render.html · render() cap01

```
// a tela inteira sai do estado, sempre
function render() {
  const t = estado.busca.toLowerCase();
  const vis = estado.pacientes.filter((p) =>
    p.nome.toLowerCase().includes(t));
  const alto = estado.pacientes
    .filter((p) => p.risco === "alto").length;
  app.innerHTML = `
    <p>${estado.pacientes.length} pacientes
```

```
      · alto risco: ${alto}</p>
    <ul>${vis.map((p) => `
      <li data-id="${p.id}">
        <b>${p.nome}</b> ${p.leito}
        <input placeholder="anotação">
        <button>Alta</button>
      </li>`).join("")}
    </ul>`;
}
```

Ela já pensa do jeito certo: o alto risco é **calculado** a cada render e nunca fica errado. O problema é o `innerHTML`, que joga fora todos os nós e cria outros. Cada tecla na busca reconstrói a lista, e o que estava dentro dos nós antigos vai junto: o texto das anotações, o foco.

1.5

Versão 3: React

A terceira mantém a ideia da segunda e troca o mecanismo. O React vem do `esm.sh` por um `importmap`:

3-react.html · Ala — cap01

```
// a tela é uma função do estado
function Ala() {
  const [busca, setBusca] = useState("");
  const [pacientes, setPacientes] =
    useState(PACIENTES);

  const t = busca.toLowerCase();
  const vis = pacientes.filter((p) =>
    p.nome.toLowerCase().includes(t));
```

```
  const alto = pacientes
    .filter((p) => p.risco === "alto").length;
  const alta = (id) => setPacientes(
    pacientes.filter((p) => p.id !== id));

  return h("div", null,
    h("input", {
      placeholder: "buscar", value: busca,
      onChange: (e) =>
        setBusca(e.target.value),
    }),
    h("p", null,
      `${pacientes.length} pacientes`
      + ` · alto risco: ${alto}`),
    h("ul", null, vis.map((p) =>
      h("li", { key: p.id },
        h("b", null, p.nome), ` ${p.leito} `,
        h("input",
          { placeholder: "anotação" }),
        h("button",
          { onClick: () => alta(p.id) },
          "Alta")))));
}
```

- `useState` guarda um valor entre renders e devolve a função que o troca. `setPacientes` não mexe no DOM: avisa o React de que o estado mudou.
- `h` é o `createElement` do React. Ele devolve uma *descrição* do que deveria existir, não um nó. O JSX vira exatamente estas chamadas (capítulo 3).
- `key: p.id` diz qual paciente cada linha representa.

A função `Ala` não sabe se roda pela primeira ou pela centésima vez: recebe o estado e devolve a tela. Quem

compara a descrição nova com a anterior e aplica a diferença é o React.

1.6

O que o medidor mostra

As três páginas passaram pelo mesmo roteiro num Chromium: anotar "ver febre" em João Lima, digitar `o` na busca, limpar a busca e dar alta a Maria Souza.

```
medicoes/cap01.py · Medido

Chromium 141.0.7390.37

versão     busca o     alta Maria  nota    alto
imperativo +0 -0 ~2    +0 -4 ~1    fica    2
render()   +18 -26 ~0  +22 -26 ~0  some    1
React      +0 -8 ~1    +0 -4 ~1    fica    1

+ elementos que entram  - que saem
~ textos ou atributos trocados
```

- **Busca.** O imperativo esconde duas linhas (dois atributos `hidden`). O `render()` destrói 26 elementos e cria 18. O React remove só as duas linhas que saíram e troca um atributo, o `value` da busca.
- **Alta.** Imperativo e React fazem o mesmo trabalho no DOM, e o imperativo deixa o alto risco em 2.
- **Nota.** Só o `render()` apagou a anotação.

> **O imperativo economiza e erra. O render() acerta e destrói. O React acerta e economiza.**

Eficiente não quer dizer mais rápido: na alta, o React ainda executou `Ala` e comparou duas descrições da tela. O ganho é deixar de escrever, e de esquecer, cada atualização à mão. O capítulo 16 mede quando o tempo passa a importar.

1.7

Declarativo, estado e reatividade

Declarativo é descrever o resultado para um estado e deixar o mecanismo decidir os passos. **Estado da interface** são os dados que, se mudarem, mudam o que aparece. **Reatividade** é a tela acompanhar o estado sem você escrever a atualização. No React ela é explícita: a tela só muda quando você chama o `set` do estado e o componente roda de novo. Vue, Svelte e Solid rastreiam sozinhos o que cada trecho da tela lê; são escolhas diferentes para o mesmo objetivo.

H+ **O monitor de beira-leito**

A enfermeira não redesenha a tela do monitor. Os sinais vitais mudam e o monitor atualiza a faixa que mudou. Os sinais vitais são o estado; a tela do monitor é a interface.

ONDE A ANALOGIA PARA

O monitor real lê os sensores o tempo todo. O React não observa nada: só calcula a tela de novo quando alguém chama o `set` de um estado. Mudar uma variável comum não muda a tela.

1.8

Biblioteca, não framework

O React transforma estado em interface. Rotas, busca de dados, build de produção e renderização no servidor vêm de outras ferramentas ou de um **framework** construído sobre ele.

DADO · REACT.DEV, FEV. 2025

O Create React App foi descontinuado. A documentação recomenda frameworks (Next.js, React Router, Expo) para produção e ferramentas como o Vite para montar a aplicação do zero e aprender o básico.

Este livro escolhe o caminho do zero com Vite de propósito: cada peça entra quando o problema dela aparece, e você vê o que um framework faz antes de delegar. O Next.js aparece nos capítulos 17 e 18 e terá um ebook próprio.

1.9

Com agentes de IA

ARMADILHA DE IA

Dois vícios imperativos aparecem em código gerado: `document.querySelector` dentro de componente para "atualizar um contador", que briga com o React pelo mesmo DOM; e HTML montado em string com `dangerouslySetInnerHTML`, que reabre a porta para XSS que o React fecha ao escapar todo texto.

PROMPT · AGENTE · CAP 1

```
Goal
Reescrever a tela de pacientes do meu
front-end vanilla como função do estado,
com React por importmap, sem build.

Context
@laboratorio/cap01/3-react.html (modelo)
@<sua tela vanilla>.html

Constraints
- sem document.querySelector em componente
- sem innerHTML e sem
  dangerouslySetInnerHTML
- contagens calculadas no render
- key de lista é o id do registro
- fora de escopo: estilo, API, build

Done when
- anotar e buscar: a anotação fica
- após a alta, todas as contagens batem
```

```
- no medidor, a alta só remove a linha
  do paciente e troca textos
```

Exercícios

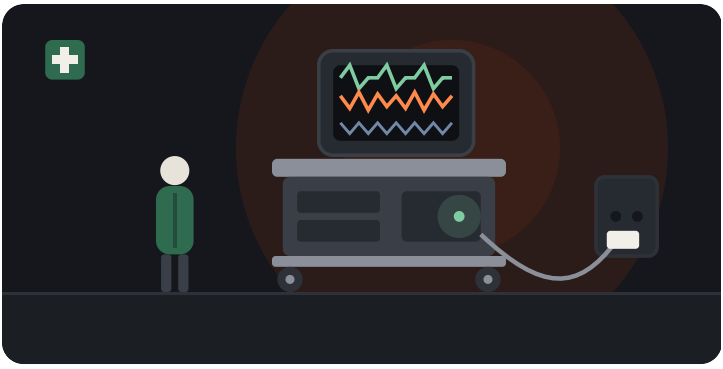
ESSENCIAL Corrija a alta na versão imperativa. Depois liste o que precisaria mudar se a tela ganhasse um terceiro número, como pacientes em isolamento.

INTERMEDIÁRIO Na versão `render()`, mova a busca para dentro do `#app` e explique, com o medidor, por que o campo perde o foco a cada tecla.

DESAFIO Na versão React, troque `key: p.id` por `key: i`. Anote em João Lima, dê alta a Maria Souza e descreva o que aconteceu. O capítulo 3 explica.

Régua: 01. **Tag:** `cap01`. **Prompts:** revisão e C-P-T-R-F-A em `prompts/cap01.md`.

Fontes. react.dev: *Sunsetting Create React App* (fev. 2025), *Creating a React App*, *Introducing the React Foundation* (out. 2025; fundação lançada em fev. 2026). MDN: *MutationObserver*. Medição em `saidas/c1-ala.txt`, Chromium 141, com React 19.3.1 servido localmente nas URLs do importmap.



PARTE I · A MUDANÇA DE PARADIGMA

02

O primeiro projeto em vinte minutos

Do terminal vazio a um projeto versionado, ligado à API e pronto para um agente trabalhar nele.

A página do capítulo 1 funcionou sem build, mas não escala: sem JSX, sem TypeScript, sem pacotes, sem build de produção. Este capítulo monta o projeto de verdade.

Você precisa do **Node 24** (ou 22.22+), Git, um editor e a extensão **React Developer Tools**. O Vite 8 aceita Node 20.19+, mas o React Router 8, que entra no capítulo 13, exige 22.22+; começar no 24 evita trocar de versão no meio do livro.

2.1

Gerando o projeto

terminal

```
npm create vite@latest mini-prontuario-web \
  -- --template react-ts
cd mini-prontuario-web && npm install
npm run dev     # http://localhost:5173
```

O `index.html` é a única página: um `<div id="root">` e o script de `src/main.tsx`, que cria a raiz do React. O `vite.config.ts` configura o Vite, os `tsconfig*.json` os tipos, e o `.oxlintrc.json` o linter, já com as regras dos Hooks.

DADO · O TEMPLATE DE OUTUBRO DE 2026

Com `create-vite` 9.3.1, o template usa **oxlint** em vez de ESLint e fixa o **TypeScript 6.0**: o 7.0, com compilador nativo, ainda não tem a API que parte das ferramentas usa. Não há `"strict": true` porque o modo estrito virou o padrão:

terminal · Medido

```
$ npx tsc -p tsconfig.app.json --noEmit
src/_t.ts(1,19): error TS7006: Parameter 'x'
  implicitly has an 'any' type.
```

2.2

Scripts e o portão

`dev` serve a tela e a atualiza quando você salva; `build` confere os tipos e gera `dist/`; `preview` serve o build; `lint` roda o oxlint. O projeto acrescenta `gate`, que roda tudo o que precisa passar antes de qualquer entrega:

```
terminal · Medido

$ npm run gate
> tsc -b && oxlint && vite build

vite v8.3.5 building client environment for
production...
✓ 16 modules transformed.
…
index-DeCkmCJv.js   219.85 kB | gzip: 68.76 kB
✓ built in 108ms
```

O `tsc -b` não gera arquivos (o `tsconfig` tem `noEmit`): quem transforma TypeScript em JavaScript é o Vite, e ele não confere tipos. Por isso o `dev` mostra a tela mesmo com erro de tipo, e o `gate` não deixa esse erro passar.

2.3

O ponto de entrada

src/main.tsx cap02

```tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.tsx";

createRoot(document.getElementById("root")!)
  .render(
    <StrictMode>
      <App />
    </StrictMode>,
  );
```

`createRoot` liga o React a um nó do DOM, e `render` faz o primeiro desenho. `StrictMode` só age em desenvolvimento: chama cada componente duas vezes por render e monta e desmonta cada efeito uma vez a mais, para revelar código que depende de rodar uma vez só. Os capítulos 3 e 7 mostram o que ele pega.

2.4

A tela e a API na mesma origem

A tela roda em `:5173` e a API em `:3000`. Portas diferentes são **origens** diferentes para o navegador, o que traria CORS e regras de cookie entre sites. O **proxy** do Vite evita isso:

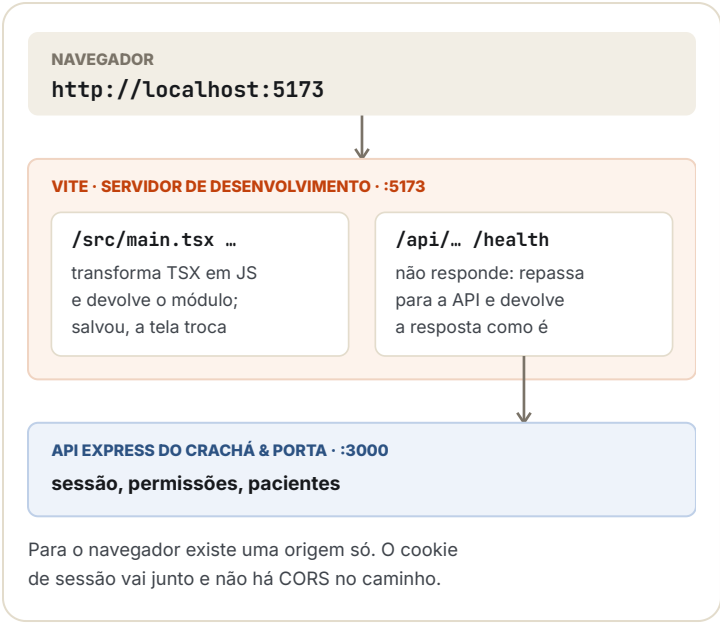


Fig. 2.1 — O navegador só conhece `localhost:5173`. O Vite serve o código da tela e repassa para a API o que começa com `/api` ou `/health`.

vite.config.ts cap02

```ts
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";

// O Vite serve a tela em :5173 e repassa para
// a API em :3000 tudo que começa com /api.
// Para o navegador, é tudo uma origem só.
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      "/api": "http://localhost:3000",
      "/health": "http://localhost:3000",
    },
```

```
  },
});
```

Em produção não há Vite: o build vira arquivos estáticos e o servidor que os publica faz o papel do proxy. O código não muda, porque nunca usa endereço de servidor:

REGRA · UI-00

O front chama a API por caminho relativo (`/api/...`). Verificação: `grep -rn "localhost:3000" src` não encontra nada.

O `App` do template virou um verificador: um botão cujo **handler de evento** chama `fetch("/health")` e guarda o resultado em estado. Buscar dados ao abrir a tela pede `useEffect`, e há bons motivos para esperar até o capítulo 7. A sessão também passa pelo proxy: este roteiro entra como Ana pela tela e consulta quem está logado.

medicoes/cap02_cookie.py · Medido

```
chromium 141.0.7390.37 · http://localhost:5173/
tela: API ok (via proxy /health)
POST /api/auth/login  -> 200
cookie __Host-sessao: httpOnly=True secure=True
sameSite=Lax
document.cookie       -> ''
GET  /api/auth/me     -> 200 ana@hospital.dev
```

O cookie chegou `HttpOnly`, `Secure` e `SameSite=Lax`, o JavaScript da página não o lê, e mesmo assim o pedido seguinte saiu autenticado: o navegador anexa o cookie sozinho. O capítulo 15 constrói a sessão da tela sobre isso; o porquê de cada atributo está no *Crachá & Porta*, capítulo 5.

Para inspecionar, a extensão React Developer Tools acrescenta as abas **Components** (árvore, props e estado) e **Profiler** (quem renderizou e quanto levou). O Vite entrega *source maps*: os *breakpoints* param no `.tsx` original.

2.5

Git e o contexto do agente

O primeiro commit vem antes da primeira feature, com quatro arquivos que o agente lê antes de qualquer tarefa:

ARQUIVO	O QUE GUARDA
`AGENTS.md`	Comandos, como o projeto funciona e regras; formato que Codex e agy leem
`CLAUDE.md`	Uma linha, `@AGENTS.md`: o Claude Code importa o mesmo conteúdo
`INVARIANTES.md`	Regras com verificação; começa com a UI-00
`IA.md`	Cada tarefa feita com agente: pedido, verificação, o que foi recusado

PONTE · DEV DE IA, CAPS. 3 A 5

Por que o contexto deve ser curto e como um invariante ganha verificação executável: o mapa dos cinco arquivos (cap. 3), `CLAUDE.md` e `AGENTS.md` (cap. 4) e `INVARIANTES.md` (cap. 5).

2.6

Com agentes de IA

A primeira tarefa num projeto novo é conferir se o agente entendeu o projeto, em modo de plano, sem editar nada. O conteúdo do prompt é o mesmo nas três ferramentas; muda como você planeja, revisa e roda sem interação:

PASSO	CLAUDE CODE	CODEX · AGY
Planejar	Shift+Tab (Plan Mode)	`/plan` · modo de plano
Revisar	`/code-review`	`/review` · subagente
Sem interação	`claude -p`	`codex exec` · `agy -p`

PROMPT · AGENTE (MODO DE PLANO) · CAP 2

```
Goal
Explicar o caminho de um clique em
"Verificar API" até a resposta da API.

Context
```

```
@AGENTS.md @vite.config.ts @src/
A API roda em :3000 (outro repositório).

Constraints
- só leitura: não edite nada
- cite arquivo e linha de cada passo
- diga o que muda em produção

Done when
- cita main.tsx, App.tsx, vite.config.ts
- explica por que não há CORS no caminho
- git status continua limpo
```

ARMADILHA DE IA

Diante de um erro de CORS, o agente costuma propor `fetch("http://localhost:3000/...")` e liberar `origin: "*"` na API. As duas mudanças funcionam e estão erradas: a primeira quebra a UI-00 e a produção, a segunda abre a API para qualquer site. O `AGENTS.md` já proíbe mexer na API e no Vite. Comandos de agente mudam rápido; confira na versão instalada.

Exercícios

ESSENCIAL Gere o projeto, configure o proxy e faça o botão mostrar **ok**. Pare a API e clique de novo: o que aparece, e por quê?

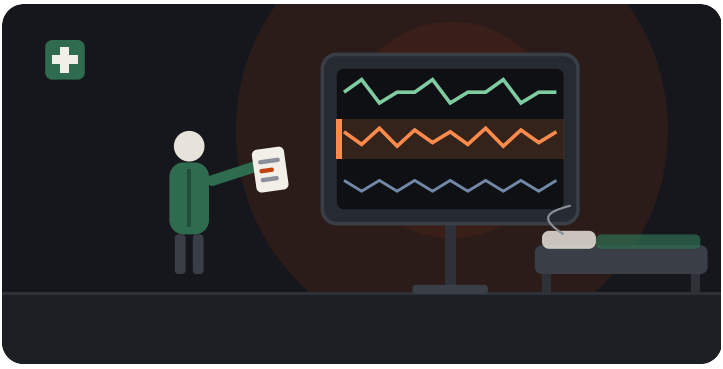
INTERMEDIÁRIO Introduza um erro de tipo no `App.tsx`. Rode `npm run dev` e depois `npm run gate` e explique a diferença.

DESAFIO

Rode `npm run build` e `npm run preview`: o botão funciona. Descubra na documentação do Vite por quê, e escreva a configuração que um servidor de produção (nginx, por exemplo) precisaria para a UI-00 continuar valendo.

Régua: prepara 34 e 35. **Invariantes:** UI-00. **Tag:** `cap02`. **Prompts:** `prompts/cap02.md`.

Fontes. Vite 8 (mar. 2026), *server.proxy*, *preview.proxy*; react.dev: *createRoot*, *StrictMode*; React Router v8; TypeScript 7 (InfoQ). Template de 21 out. 2026. Saídas em `saidas/c2-*.txt`.



PARTE I · A MUDANÇA DE PARADIGMA

03

Como o React funciona por dentro

Gatilho, render e commit, identidade e keys, provados com o motor ligado.

Quarta, 10h. A enfermeira anota "ver febre" no campo de João Lima e dá alta a Maria Souza. A anotação aparece na linha de Ana Clara Reis; João fica com o campo vazio.

O código da alta está certo. O problema está numa linha que parece irrelevante, e para achá-lo é preciso saber o que o React faz entre a sua função devolver a tela e o navegador mostrá-la.

3.1

O projeto do capítulo

Na tag `cap03`, a Ala B virou componentes com JSX e TypeScript, em `src/features/pacientes`. O cartão de cada paciente:

src/features/pacientes/CartaoPaciente.tsx `cap03`

```tsx
import type { Paciente } from "./tipos.ts";

interface Props {
  paciente: Paciente;
  onAlta: (id: string) => void;
}

export function CartaoPaciente(
  { paciente, onAlta }: Props,
) {
  const { id, nome, leito } = paciente;
  return (
    <li>
      <b>{nome}</b> {leito}
      <input
        aria-label={`Anotação de ${nome}`}
        placeholder="anotação"
      />
      <button onClick={() => onAlta(id)}>
        Alta
      </button>
    </li>
  );
}
```

A `Ala` guarda `busca` e `pacientes` em estado e calcula o resto, exatamente como a versão 3 do capítulo

1, agora em JSX.

3.2

JSX é açúcar

O navegador não entende JSX. Antes de chegar a ele, cada tag vira uma chamada de função, como o `h` do capítulo 1. É isto que o Vite entrega quando o navegador pede o cartão:

servidor do Vite · Medido

```
GET /src/features/pacientes/CartaoPaciente.tsx
(trecho; metadados de depuração viraram …)

export function CartaoPaciente({ paciente, onAlta
}) {
  const { id, nome, leito } = paciente;
  return _jsxDEV("li", { children: [
    _jsxDEV("b", { children: nome }, …),
    " ",
    leito,
    _jsxDEV("input", {
      "aria-label": `Anotação de ${nome}`,
      placeholder: "anotação"
    }, …),
    _jsxDEV("button", {
      onClick: () => onAlta(id),
      children: "Alta"
    }, …)
  ] }, …);
}
```

Em desenvolvimento a função se chama `jsxDEV` e carrega arquivo e linha de cada tag para as mensagens

de erro; em produção, `jsx`. As chaves do JSX viram expressões comuns, os filhos viram a prop `children`, e o resultado é um **elemento**: um objeto que *descreve* um `<li>`. Nenhum nó do DOM foi criado.

3.3

Gatilho, render, commit



Fig. 3.1 — As etapas de toda atualização. O **gatilho** é a montagem inicial ou um `set`. No **render**, o React chama os componentes e recebe a árvore nova. No **commit**, compara com a anterior e aplica no DOM só o que mudou. A pintura já é do navegador; os efeitos são o capítulo 7.

H+ **A passagem de plantão**

Na troca de turno, a enfermeira compara o quadro de ontem com a situação de hoje e anota só o que mudou. Olhar a situação é o render; anotar a diferença é o commit.

ONDE A ANALOGIA PARA

O plantão tem hora marcada; o render só acontece com gatilho e pode terminar sem diferença nenhuma. O React pode até descartar um render pela metade para atender algo mais urgente. O commit, nunca: é aplicado inteiro, de uma vez.

3.4

Renderizar não é mexer no DOM

Para separar as duas coisas, o projeto tem o `Renderometro`, construído sobre o `<Profiler>` do React: a cada commit em que algo dentro dele renderizou, o React chama `onRender` com o nome, a fase e o tempo.

src/lab/Renderometro.tsx cap03

```
export function Renderometro(
  { id, children }: Props,
) {
  return (
    <Profiler
      id={id}
      onRender={(quem, fase, ms) => {
```

```
        const linha = `${quem} · ${fase}`
          + ` · ${ms.toFixed(2)} ms`;
        registros.push(linha);
        console.log(`[render] ${linha}`);
      }}
    >
      {children}
    </Profiler>
  );
}
```

A lista e cada cartão estão dentro de um `Renderometro`. Um roteiro no Chromium cruza o que ele anotou com o que um `MutationObserver` viu no DOM:

medicoes/cap03.py · Medido

```
digitar "o" na busca
  Maria Souza     update
  João Lima       update
  Pedro Alves     update
  Raimundo Nonato update
  Lista           update
  DOM: +0 -8 ~1
…
anotar em João Lima
  DOM: +0 -0 ~0
```

Uma tecla na busca fez a lista e os quatro cartões visíveis renderizarem de novo. No DOM, porém, só saíram as duas linhas que deixaram de bater, oito elementos, e mudou o `value` da busca. Os cartões foram *calculados* de novo, deram o mesmo resultado, e o commit não os tocou.

Digitar na anotação de João não provocou render nenhum: aquele campo não tem estado no React, o texto vive no `<input>` do DOM. Guarde isso para a seção 3.6.

3.5

O "Virtual DOM", com precisão

"Virtual DOM" é o apelido da árvore de elementos que cada render devolve. Ele **não é uma cópia do DOM mantida em sincronia**: é a descrição produzida por um render, comparada com a do render anterior. E **não é sempre mais rápido** que mexer no DOM à mão: no capítulo 1, imperativo e React fizeram o mesmo trabalho no DOM, e o React ainda executou componentes. O que ele entrega é correção sem esforço manual.

A comparação, a **reconciliação**, segue regras simples:

NA MESMA POSIÇÃO	O QUE O REACT FAZ
Mesmo tipo	Mantém o nó e o estado; atualiza o que mudou nas props
Tipo diferente	Desmonta o que havia, com o estado, e monta do zero
Em listas, com `key`	Casa os itens pela key, não pela posição

3.6

Identidade: o estado mora na posição

A regra da documentação é curta: **o React mantém o estado de um componente enquanto ele for renderizado na mesma posição da árvore**. Ser "o mesmo", para o React, é estar no mesmo lugar, com o mesmo tipo e, em listas, a mesma key. Na tag `cap03-antes`, a lista usava o índice como key:

```
src/features/pacientes/ListaPacientes.tsx                cap03-antes → cap03

…
      <ul>
-       {pacientes.map((p, i) => (
-         <Renderometro key={i} id={p.nome}>
+       {pacientes.map((p) => (
+         <Renderometro key={p.id} id={p.nome}>
            <CartaoPaciente
```



Fig. 3.2 — Com key por índice, o React vê que a última posição sumiu e que as outras só trocaram de props. O `<input>` da posição 1 fica no DOM, com o texto, mostrando agora Ana Clara.

O texto nunca esteve no React; estava no nó. Com key por índice, os nós são reaproveitados pela posição e o texto fica preso a eles. Com o `id`, o React sabe que `p1` saiu e que `p2` continua `p2`. O teste descreve o que a enfermeira faz, sem olhar para dentro do componente:

src/features/pacientes/Ala.test.tsx — cap03

```
test("anotação acompanha o paciente",
  async () => {
    render(<Ala />);
    await userEvent.type(
      nota("João Lima"), "ver febre");
    await darAlta("Maria Souza");
    expect(nota("João Lima").value)
      .toBe("ver febre");
  });
```

CAP03-ANTES · KEY={I}

```
× anotação acompanha o paciente
→ expected '' to be 'ver febre'
```

CAP03 · KEY={P.ID}

```
✓ anotação acompanha o paciente
✓ alto risco cai na alta
```

É a **prova de revert** da série: o teste falha sem a correção e passa com ela. As saídas completas estão em `saidas/c3-key-*.txt`. Trocar a key de propósito também serve para zerar um componente; o capítulo 6 usa isso.

REGRA · UI-01

Lista usa a key do domínio (o `id`), nunca o índice. Verificação: o teste "anotação acompanha o paciente".

3.7

Montar, atualizar, desmontar

Montar é aparecer numa posição: nós criados, estado no valor inicial. **Atualizar** é renderizar de novo no mesmo lugar. **Desmontar** é sair da árvore, levando nós e estado. Na medição completa, Ana Clara e Luzia voltam como `mount` quando a busca é limpa: tinham sido desmontadas.

Para isso funcionar, o render precisa ser **puro**: as mesmas props e o mesmo estado produzem a mesma tela, e renderizar não altera nada fora do componente. O `StrictMode` testa a promessa chamando cada componente duas vezes em desenvolvimento, e é por isso que o `Renderometro` conta commits pelo `Profiler` em vez de somar uma variável no corpo do componente. Internamente, o render é dividido em unidades de trabalho, as *fibers*, o que permite pausá-lo; recursos como `startTransition` usam isso e aparecem quando forem úteis.

3.8

Com agentes de IA

ARMADILHA DE IA

Agentes geram `key={index}` com frequência, porque é o exemplo mais curto, e às vezes `key={Math.random()}`, que é pior: cada render remonta tudo e apaga todo estado da lista. Nenhum dos dois quebra o build. A UI-01 e o teste pegam ambos.

Para um bug como o da abertura, o primeiro prompt pede diagnóstico com evidência, não correção:

PROMPT · AGENTE · CAP 3 · DIAGNÓSTICO

```
Goal
Descobrir por que a anotação de um
paciente aparece em outro depois de
uma alta, e provar a causa.

Context
@src/features/pacientes/ @INVARIANTES.md
Passos: anotar em João Lima, dar alta a
Maria Souza; a nota vai para Ana Clara.

Constraints
- antes, um teste que reproduz o defeito
  pelo que o usuário vê, falhando
- depois, a correção mínima
- sem dependência nova

Done when
- npm run gate verde com a correção
- o teste novo FALHA se a correção
```

```
  for revertida
- a causa aponta arquivo e linha
```

> **PONTE · DEV DE IA, CAPS. 9 E 13**
>
> O "falha se revertido" vem do capítulo 9: agentes escrevem com facilidade testes que passam antes e depois da mudança. A revisão em contexto novo, de preferência com outra ferramenta, é o capítulo 13; o prompt dela está em `prompts/cap03.md`.

Exercícios

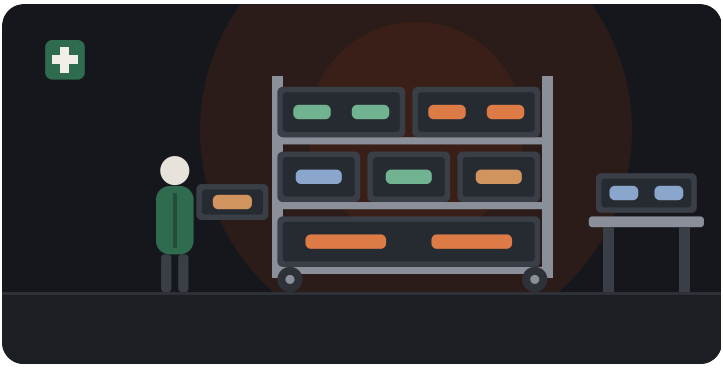
ESSENCIAL Rode `npm test` em `cap03-antes` e em `cap03`. Explique a diferença em três frases, usando "posição" e "key".

INTERMEDIÁRIO Preveja quais componentes renderizam ao dar alta ao último paciente. Rode com o `Renderometro` e compare. Por que a previsão para key por índice seria diferente?

DESAFIO Troque a key por `Math.random()`, meça uma tecla na busca com `medicoes/cap03.py` e explique o número.

Régua: 02, 03, 04. **Invariantes:** UI-01. **Tags:** `cap03-antes`, `cap03`. **Prompts:** `prompts/cap03.md`.

Fontes. react.dev: *Render and Commit*, *Preserving and Resetting State*, *Rendering Lists*, *Keeping Components Pure*, *<Profiler>*, *<StrictMode>*. Saídas em `saidas/c3-*.txt`.



PARTE II · OS FUNDAMENTOS

04

Componentes, JSX e composição

Peças pequenas que se encaixam. Nem toda repetição pede um componente, e nenhum componente nasce dentro de outro.

Quinta, 15h. A tela da Ala B ganhou resumo por risco, um selo colorido em cada paciente e um aviso para busca sem resultado. Tudo foi parar dentro de `Ala`. Desde então, a cada letra digitada na busca, as anotações somem.

O código novo não tem erro de lógica. Tem um erro de *forma*: um componente foi definido no lugar errado. Este capítulo trata de como dividir uma tela em componentes, e dos dois jeitos de errar nisso.

4.1

Um componente é uma função

Um componente é uma função que recebe um objeto, as **props**, e devolve JSX. Poucas regras sustentam tudo:

REGRA	EXEMPLO
Nome com maiúscula; minúscula é tag HTML	`<CartaoPaciente />` × `<li>`
Uma raiz, ou um fragmento	`<>…</>` agrupa sem criar nó
Atributos em camelCase	`className`, `onClick`, `aria-label` fica igual
Chaves para expressões	`{pacientes.length}`; `if` não cabe, ternário cabe
Toda tag fecha	`<input />`, ` `

4.2

O componente que cresceu demais

Na tag `cap04-antes`, a `Ala` tem 74 linhas e faz tudo. No meio dela, para "organizar", alguém extraiu a linha da lista para um componente `Linha`, escrito *dentro* de `Ala`:

Ala.tsx · Linha, dentro de Ala — cap04-antes

```
function Linha({ p }: { p: Paciente }) {
  return (
```

```
      <li>
        <b>{p.nome}</b> {p.leito}
        <span data-risco={p.risco}>
          {p.risco}
        </span>
        <input
          aria-label={`Anotação de ${p.nome}`}
          placeholder="anotação"
        />
        <button onClick={() => darAlta(p.id)}>
          Alta
        </button>
      </li>
    );
  }
```

Parece inofensivo, e o código usa `key={p.id}` corretamente. O problema é a regra do capítulo 3: **tipo diferente na mesma posição, o React desmonta e monta de novo**. A cada render de `Ala`, a função `Linha` é criada outra vez. É uma função nova, logo um tipo novo. O React não tem como saber que é "a mesma `Linha`": desmonta todas as linhas, com o que havia nelas, e monta outras.

medicoes/cap04.py · Medido

```
== cap04-antes

digitar "o" na busca
  Maria Souza     update
  João Lima       update
  Pedro Alves     update
  Raimundo Nonato update
  DOM: +20 -30 ~1
  anotação de João: ''
```

```
== cap04

digitar "o" na busca
  Maria Souza     update
  João Lima       update
  Pedro Alves     update
  Raimundo Nonato update
  DOM: +0 -10 ~1
  anotação de João: 'ver febre'
```

Na mesma tecla, `cap04-antes` recriou 20 elementos e removeu 30: as quatro linhas que ficaram foram reconstruídas e a anotação de João foi junto. Em `cap04` saíram só os 10 elementos das duas linhas filtradas. Repare que o `Renderometro` diz `update` nos dois casos: o `Profiler` está do lado de fora de `Linha`, na mesma posição, e não vê a troca de tipo lá dentro. Quem mostra a verdade é o DOM.

CAP04-ANTES · LINHA DENTRO DE ALA

`× anotação acompanha o paciente`
`× anotação sobrevive à busca`

CAP04 · COMPONENTES NO MÓDULO

`✓ os quatro testes`, inclusive os dois acima, sem mudar uma linha do teste

O lint do template não percebe o problema. O oxlint tem a regra, mas ela não vem ligada:

```
terminal · Medido

$ npx oxlint            # regras do template
(nenhuma saída) · exit 0

$ npx oxlint -D react/no-unstable-nested-
components
src/features/pacientes/Ala.tsx:25:3
  error react(no-unstable-nested-components)
  Do not define components during render.
  help: Move this component definition out of the
parent component `Ala` and pass data as props.
exit 1
```

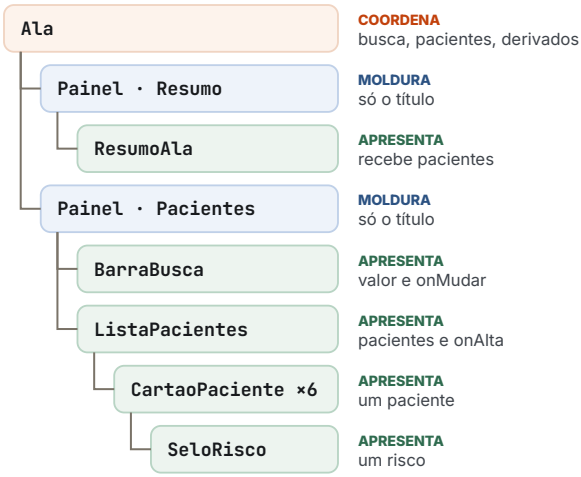
REGRA · UI-02

Componente nunca é definido dentro de outro. Verificação: `react/no-unstable-nested-components` ligada no `.oxlintrc.json`, que o `npm run gate` roda.

4.3

Composição

A correção, na tag `cap04`, não é só tirar `Linha` de dentro de `Ala`. É dividir a tela pelo que cada pedaço *é* para quem usa: o resumo, a busca, a lista, o cartão, o selo.



Quem coordena guarda estado. Quem apresenta recebe props. A moldura não sabe o que carrega.

Fig. 4.1 — A árvore da Ala na tag `cap04`. Três papéis: quem **coordena** guarda estado e compõe; quem **apresenta** recebe props e mostra; a **moldura** recebe conteúdo pronto.

Agora a `Ala` só coordena. O que ela devolve se lê como a própria tela:

Ala.tsx · composição cap04

```
return (
  <>
    <Painel titulo="Resumo">
      <ResumoAla pacientes={pacientes} />
    </Painel>
    <Painel titulo="Pacientes">
      <BarraBusca
        valor={busca}
        onMudar={setBusca}
      />
```

```
        {visiveis.length === 0 ? (
          <p>Nenhum paciente encontrado.</p>
        ) : (
          <ListaPacientes
            pacientes={visiveis}
            onAlta={darAlta}
          />
        )}
      </Painel>
    </>
  );
```

O `Painel` é a moldura. Ele recebe o conteúdo pela prop especial `children`, tudo o que vem entre a tag de abertura e a de fechamento, e não sabe o que está mostrando:

src/compartilhado/Painel.tsx cap04

```
// Moldura com título. Não sabe o que mostra:
// quem a usa decide, pelos children.
export function Painel(
  { titulo, children }: Props,
) {
  return (
    <section className="painel">
      <h2>{titulo}</h2>
      {children}
    </section>
  );
}
```

H+ **As bandejas do carrinho**

O carrinho de procedimentos não tem um compartimento para cada combinação de material. Tem bandejas padronizadas que se encaixam de jeitos diferentes. O `Painel` é o encaixe; o que vai dentro muda a cada uso.

ONDE A ANALOGIA PARA

Bandeja física tem tamanho fixo. A composição do React aninha sem limite: um `Painel` dentro de outro, uma lista dentro de um painel. O custo de aninhar é só de leitura, não de desempenho.

4.4

Composição, duplicação e abstração prematura

Dividir demais também é erro. O sintoma é o componente genérico que tenta servir a todos os casos:

ABSTRAÇÃO PREMATURA

`<Cartao tipo="paciente" mostrarSelo mostrarBotaoAlta titulo={…} subtitulo={…} … />`: dez props booleanas, um `if` para cada combinação, e ninguém sabe qual combinação a tela usa.

COMPOSIÇÃO

Componentes de domínio pequenos (`CartaoPaciente`, `SeloRisco`) e uma moldura que recebe `children`. Cada um tem um motivo para mudar.

O projeto adota três critérios, escritos em `docs/UI.md` para que o agente os siga também:

DOCS/UI.MD · QUANDO EXTRAIR UM COMPONENTE

- Um trecho com nome próprio na conversa do usuário ("o resumo").
- Um trecho repetido pela terceira vez. Na segunda, copie e espere.
- Nunca para "deixar genérico" antes de existir o segundo uso real.

PONTE · DEV DE IA, CAP. 6

O `docs/UI.md` também fixa um componente por arquivo, nomes em PascalCase e props numa `interface Props`. O Dev de IA explica por que esse arquivo reduz retrabalho e como mantê-lo curto.

4.5

Com agentes de IA

ARMADILHA DE IA

Os agentes erram para os dois lados. Pedidos de "mais uma funcionalidade" sem contexto tendem a crescer o componente que já existe até virar um componente-deus, muitas vezes com um componente auxiliar definido lá dentro, exatamente o defeito deste capítulo. Pedidos de "deixe reutilizável" tendem a produzir o cartão genérico com dez props. O `docs/UI.md` e a UI-02 cortam os dois.

PROMPT · AGENTE · CAP 4

```
Goal
Dividir a Ala em componentes de domínio,
sem mudar o comportamento.

Context
@src/features/pacientes/ @docs/UI.md
@INVARIANTES.md

Constraints
- um componente por arquivo, nomes de
  domínio, Props tipadas
- Ala só guarda estado, deriva e compõe
- nenhum componente dentro de outro
- moldura genérica só em compartilhado/
- não altere Ala.test.tsx

Done when
- npm run gate verde, com a regra
  no-unstable-nested-components ligada
```

```
- Ala.test.tsx passa sem nenhuma mudança
- git diff não toca Ala.test.tsx
```

O último critério é o que transforma "refatorar" em algo verificável: se o teste precisou mudar, o comportamento mudou. A revisão e as variantes estão em `prompts/cap04.md`.

Exercícios

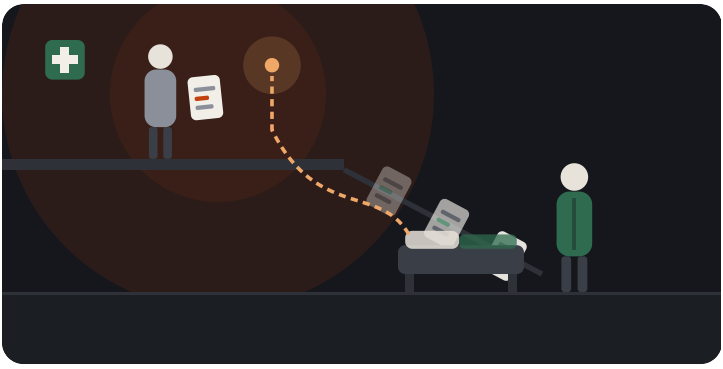
ESSENCIAL Em `cap04-antes`, mova `Linha` para fora de `Ala` sem mais nenhuma mudança. Quais props ela passa a precisar? Rode os testes.

INTERMEDIÁRIO Use o `Painel` para criar um terceiro quadro, "Avisos", com uma lista fixa de dois avisos. Não mude o `Painel`.

DESAFIO Peça a um agente um "cartão reutilizável para pacientes e profissionais". Avalie a resposta pelos critérios do `docs/UI.md` e registre no `IA.md` o que você recusou.

Régua: 05. **Invariantes:** UI-02. **Tags:** `cap04-antes`, `cap04`. **Prompts:** `prompts/cap04.md`.

Fontes. react.dev: *Your First Component*, *Writing Markup with JSX*, *Passing Props to a Component* (children), *Preserving and Resetting State* (não aninhar definições). oxlint: *react/no-unstable-nested-components*. Saídas em `saidas/c4-*.txt`, Chromium 141.



PARTE II · OS FUNDAMENTOS

05

Props, eventos e fluxo de dados

Dados descem como props, eventos sobem como chamadas. Nenhum componente altera o que recebeu.

Sexta, 8h. A enfermeira seleciona João Lima e clica em "Prioridade". Nada acontece. Ela digita uma letra na busca, e a estrela de prioridade aparece em João, sem que ninguém tenha clicado de novo.

Uma mudança que só aparece quando outra coisa acontece é a assinatura de um erro específico. Antes de chegar nele, o capítulo monta a seleção de pacientes e

o painel de detalhe, e com eles o fluxo de dados do React.

5.1

Props: o contrato do componente

As props de um componente são o seu contrato com quem o usa. Em TypeScript, o contrato fica explícito numa `interface`:

CartaoPaciente.tsx · contrato cap05

```tsx
interface Props {
  paciente: Paciente;
  selecionado: boolean;
  onSelecionar: (id: string) => void;
  onPrioridade: (id: string) => void;
  onAlta: (id: string) => void;
}

export function CartaoPaciente({
  paciente, selecionado,
  onSelecionar, onPrioridade, onAlta,
}: Props) {
  const { id, nome, leito, risco } = paciente;
  const atual = selecionado || undefined;
```

- **Obrigatória** é a prop sem `?`: esquecê-la é erro de tipo, não bug em produção.
- **Opcional** leva `?` e pode chegar `undefined`, como `selecionadoId?: string` na lista. Valor padrão vai na desestruturação: `{ risco = "baixo" }`.
- Funções também são props. Por convenção, a prop se chama `onAlgo` e a função de quem a passa tem

um nome de ação, como `darAlta`.

Props são **somente leitura**. O componente que recebe `paciente` não altera `paciente.prioridade`: ele avisa quem é dono do dado, chamando `onPrioridade(id)`.

5.2

Levantar o estado

A lista precisa saber qual paciente está selecionado, para destacá-lo, e o painel de detalhe precisa mostrá-lo. Quando dois componentes dependem do mesmo dado, o estado sobe para o ancestral comum mais próximo, aqui a `Ala`:

Ala.tsx · estado e derivados cap05

```
export function Ala() {
  const [busca, setBusca] = useState("");
  const [pacientes, setPacientes] =
    useState(ALA_B);
  const [selecionadoId, setSelecionadoId] =
    useState<string>();

  const t = busca.toLowerCase();
  const visiveis = pacientes.filter((p) =>
    p.nome.toLowerCase().includes(t));
  const selecionado = pacientes
    .find((p) => p.id === selecionadoId);
```

O estado guarda o **id** do selecionado, não uma cópia do objeto. O paciente selecionado é calculado a cada render a partir da lista. Se a `Ala` guardasse uma cópia,

a prioridade mudaria na lista e a cópia no painel continuaria velha: dois estados para a mesma verdade.

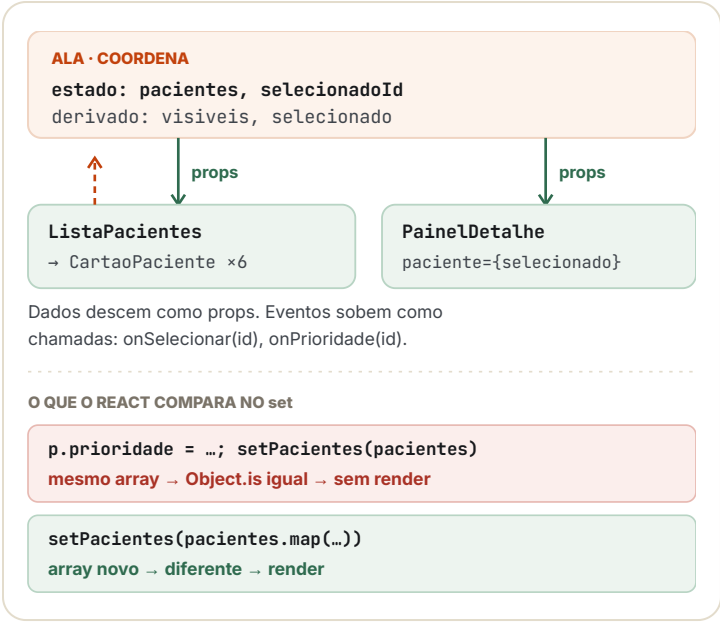


Fig. 5.1 — O fluxo de mão única. A `Ala` é dona do estado; lista e detalhe recebem props. O cartão não altera nada: chama `onSelecionar` ou `onPrioridade`, e a `Ala` decide. Embaixo, o que o React compara quando você chama `set`.

H+

A prescrição e a campainha

A prescrição desce do médico para a enfermagem, e quem a recebe não a reescreve. Quando o paciente precisa de algo, ele não vai ao posto mudar a prescrição: toca a campainha, e quem é responsável decide. Props descem; eventos sobem.

ONDE A ANALOGIA PARA

A campainha só avisa que algo aconteceu. O callback do React leva dados (`onPrioridade(id)`) e roda na hora, dentro do componente dono do estado.

A lista não usa os eventos; ela só os repassa ao cartão. O espalhamento de props (`...acoes`) evita repetir as três linhas:

src/features/pacientes/ListaPacientes.tsx cap05

```
export function ListaPacientes({
  pacientes, selecionadoId, ...acoes
}: Props) {
  return (
    <ul>
      {pacientes.map((p) => (
        <Renderometro key={p.id} id={p.nome}>
          <CartaoPaciente
            paciente={p}
            selecionado={
              p.id === selecionadoId}
            {...acoes}
          />
        </Renderometro>
      ))}
    </ul>
  );
}
```

Repassar uma prop por um componente que não a usa é normal em um ou dois níveis. Quando são cinco, o nome disso é *prop drilling*, e o capítulo 11 trata das saídas.

5.3

Renderização condicional

O painel de detalhe tem dois estados visuais, e o jeito mais legível de expressá-los é um retorno antecipado:

src/features/pacientes/PainelDetalhe.tsx cap05

```
…
export function PainelDetalhe(
  { paciente }: Props,
) {
  if (!paciente) {
    return <p>Selecione um paciente.</p>;
  }
…
```

ARMADILHA DO && COM NÚMERO

`{visiveis.length && <ListaPacientes … />}` mostra **0** na tela quando a lista está vazia: o React renderiza números, e `0 && x` vale `0`.

CONDIÇÃO BOOLEANA

`{visiveis.length > 0 && …}`, ou o ternário com o estado vazio, como a `Ala` faz.

5.4

O erro clássico: mudar o objeto no lugar

Na tag `cap05-antes`, a prioridade era alterada assim, e a correção de `cap05` cria um array novo:

src/features/pacientes/Ala.tsx — `cap05-antes → cap05`

```
…
   function alternarPrioridade(id: string) {
-    const p = pacientes
-      .find((x) => x.id === id)!;
-    p.prioridade = !p.prioridade;
-    setPacientes(pacientes);
+    setPacientes(pacientes.map((p) =>
+      p.id === id
+        ? { ...p, prioridade: !p.prioridade }
+        : p));
   }
```

A versão antiga altera o objeto que já está no estado e chama `setPacientes` com **o mesmo array**. O React compara o valor novo com o anterior por referência, com `Object.is`. É o mesmo array, então nada mudou, e nenhum render acontece. O objeto, porém, foi alterado de verdade, e aparece no próximo render que qualquer outra coisa provocar:

medicoes/cap05.py · Medido

```
== cap05-antes
clique em Prioridade: 0 render(s), ★ na tela: não
tecla na busca:       4 render(s), ★ na tela: sim

== cap05
clique em Prioridade: 6 render(s), ★ na tela: sim
tecla na busca:       4 render(s), ★ na tela: sim
```

Zero renders no clique; a estrela só aparece quando a busca provoca um render por outro motivo. A versão corrigida cria um array novo e um objeto novo só para o paciente alterado (`{ ...p, prioridade }`). Os outros cinco objetos continuam os mesmos, o que importa no capítulo 16, quando a memoização entrar.

CAP05-ANTES

`× prioridade no cartão e no detalhe`

CAP05

`✓ os seis testes`

REGRA · UI-03

Estado nunca é alterado no lugar: toda atualização cria objeto ou array novo. Verificação: o teste "prioridade no cartão e no detalhe".

5.5

Com agentes de IA

ARMADILHA DE IA

Mutação escondida em métodos que parecem leitura é o caso mais comum em código gerado: `pacientes.sort(...)` e `reverse()` alteram o array original, e `push` e `splice` também. Para ordenar sem mutar, use `toSorted()`, do ES2023, ou ordene uma cópia.

PROMPT · AGENTE · CAP 5

```
Goal
Ordenar a lista da Ala por risco (alto
primeiro), com um botão que liga e
desliga a ordenação.

Context
@src/features/pacientes/ @INVARIANTES.md
@docs/UI.md

Constraints
- a ordem é derivada no render, nunca
  guardada como outra lista em estado
- nada de sort, reverse, push ou splice
  sobre estado (UI-03)
- o botão usa aria-pressed

Done when
- npm run gate verde
- teste novo: ligar, desligar e conferir
  a ordem original de volta
- os seis testes atuais passam sem mudar
```

Exercícios

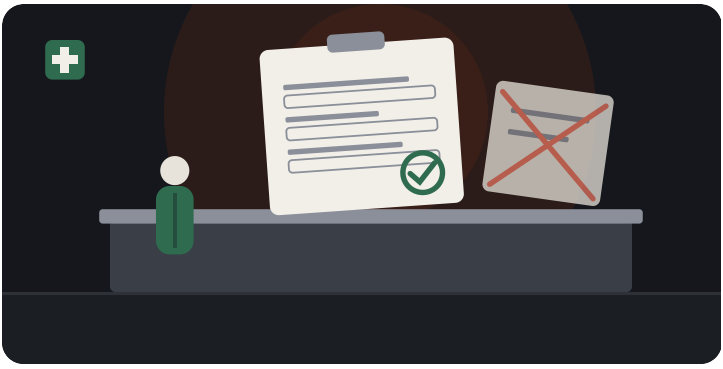
ESSENCIAL Em `cap05-antes`, troque `setPacientes(pacientes)` por `setPacientes([...pacientes])` e rode os testes. Passa? Explique por que essa correção resolve o sintoma e mesmo assim viola a UI-03.

INTERMEDIÁRIO Faça o painel de detalhe guardar uma cópia do paciente em `useState`. Marque a prioridade e descreva o que fica dessincronizado.

DESAFIO Implemente a ordenação por risco do prompt sem usar agente. Depois rode o prompt num agente e compare as duas versões com o V.E.R.M.

Régua: 06; prepara 08. **Invariantes:** UI-03. **Tags:** `cap05-antes`, `cap05`. **Prompts:** `prompts/cap05.md`.

Fontes. react.dev: *Passing Props to a Component*, *Responding to Events*, *Sharing State Between Components*, *Conditional Rendering*, *Updating Arrays in State*, *useState* (comparação com `Object.is`). MDN: *Array.prototype.toSorted*. Saídas em `saidas/c5-*.txt`, Chromium 141.



PARTE II · OS FUNDAMENTOS

06

Estado, formulários e interações

Guarde o mínimo, calcule o resto, e trate cada envio como algo que pode chegar duas vezes.

Segunda, 7h40. A rede do pronto-socorro está lenta. A recepcionista admite Rosa Amaral, nada parece acontecer, e ela clica de novo. Rosa ganha dois prontuários com o mesmo CNS, e o exame da tarde vai para um deles.

O formulário estava certo: validava, enviava e mostrava o resultado. Faltava tratar o tempo entre o clique e a resposta. Este capítulo monta os dois formulários do Mini-Prontuário, o de entrada e o de admissão, e

termina com a defesa contra esse erro, medida no navegador.

6.1

O estado é uma foto

`useState` devolve o valor *deste render* e uma função para pedir o próximo. Chamar `setX` não muda `x` na linha seguinte; agenda um render novo, em que `x` terá o valor novo.

Por isso `setTotal(total + 1)` três vezes seguidas soma 1: as três chamadas leem o mesmo `total` da foto. Com a **atualização funcional**, `setTotal((t) => t + 1)`, cada chamada recebe o resultado da anterior e a soma dá 3. Use essa forma sempre que o próximo valor depender do anterior.

6.2

Guardar ou calcular

Antes de criar um estado, faça a pergunta do capítulo 5: **dá para calcular?** Total de pacientes, lista filtrada, paciente selecionado e "o formulário é válido?" são **derivados**: calculados a cada render, a partir de outro estado ou de props. Estado guardado em dobro precisa ser sincronizado à mão, e a sincronização é onde nascem os bugs. O capítulo 7 mostra o efeito que tenta fazer essa sincronização e por que ele é o erro mais comum de React.

REGRA PRÁTICA

Estado é o que o usuário fez ou o que o servidor disse. Todo o resto é derivado.

6.3

Controlado e não controlado

Um campo pode guardar o valor em dois lugares. No **controlado**, o valor mora no estado do React e cada tecla passa por ele. É o formulário de entrada:

FormLogin.tsx · controlado cap06

```
…
// Formulário controlado: cada tecla passa
// pelo estado do React.
export function FormLogin() {
  const [email, setEmail] = useState("");
  const [senha, setSenha] = useState("");
  const [estado, setEstado] =
    useState<Estado>({ tipo: "ocioso" });

  async function entrar(
    e: FormEvent<HTMLFormElement>,
  ) {
    e.preventDefault();
    setEstado({ tipo: "enviando" });
…
```

No **não controlado**, o valor mora no próprio `<input>`, e o código só o lê no envio, com `FormData`. É o formulário de admissão, que usa `name` nos campos e `defaultValue` para devolver o que foi digitado quando

há erro. Use controlado quando a tela reage a cada tecla (busca, máscara, botão que só habilita com o campo cheio); não controlado quando o valor só importa no envio, o caso da maioria dos cadastros.

6.4

Os estados de um envio

Todo envio passa pelos mesmos momentos, e a tela precisa de uma resposta para cada um. Em vez de quatro booleanos (`enviando`, `erro`, `ok`, `invalido`) que podem se contradizer, o resultado é um tipo só, uma **união discriminada** do TypeScript:

src/features/admissao/admitir.ts cap06

```
export type Valores = Record<string, string>;
export type Resultado =
  | { tipo: "ocioso" }
  | { tipo: "invalido"; erros: Valores;
      valores: Valores }
  | { tipo: "erro"; msg: string;
      valores: Valores }
  | { tipo: "ok"; id: string };
```

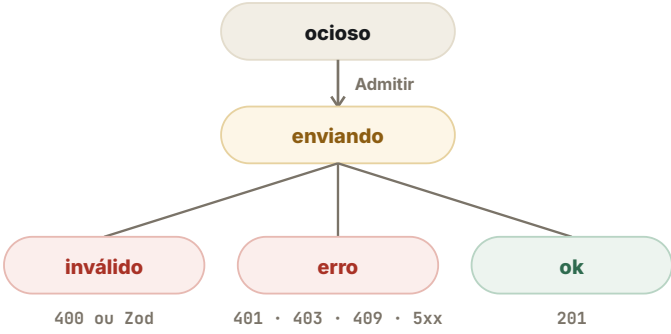


Fig. 6.1 — Os estados de um envio. Com uma união, "ok com mensagem de erro" é impossível de escrever: o TypeScript não deixa.

A função `admitir` valida, envia e traduz a resposta HTTP num desses estados. A validação, em `esquema.ts`, repete as regras do esquema Zod da API (nome com duas letras, CNS com 15 dígitos, data ISO), agora com mensagens para quem preenche.

REGRA · UI-05

A tela valida para ajudar; o servidor valida para decidir. Erro do servidor aparece no campo. Verificação: o teste "mostra o que o servidor recusou", que simula um 400 com `campos: ["cns"]`.

6.5

Actions do React 19

O React 19 trouxe um jeito próprio de tratar envio: passar uma função para `<form action>` e guardar o resultado com `useActionState`. O hook devolve o último resultado, a ação para o formulário e um booleano `pendente`:

FormAdmissao.tsx · action — cap06

```
export function FormAdmissao() {
  const emCurso = useRef(false);
  async function enviar(
    ant: Resultado, dados: FormData,
  ) {
    try {
      return await admitir(ant, dados);
    } finally {
      emCurso.current = false;
    }
  }
  const [res, acao, pendente] =
    useActionState(enviar, INICIAL);
```

A action recebe o resultado anterior e o `FormData`, e o que ela devolve vira o próximo `res`. Não há `preventDefault` nem `setRes` à mão. Um detalhe que surpreende: sempre que a action termina sem lançar erro, o React **reinicia os campos não controlados** para o `defaultValue`. Depois de admitir, os campos ficam vazios; depois de uma recusa, voltam com o que foi digitado, porque o `defaultValue` de cada `Campo` vem de `res.valores`.

6.6

O erro clássico: o envio que chega duas vezes

Na tag `cap06-antes`, o envio era um `onSubmit` comum, e nada impedia o segundo clique. O teste com servidor lento falha:

CAP06-ANTES

```
× não envia duas vezes
→ expected "vi.fn()" to be called 1 times, but got 2 times
```

CAP06

```
✓ não envia duas vezes
✓ dois cliques no mesmo instante
```

A correção óbvia é `disabled={pendente}`, e é a que a documentação mostra. Medimos se basta: um roteiro abre a tela no Chromium contra a API real, deixa a rede 800 ms mais lenta e dá dois cliques no mesmo instante. A linha "sem ref" é o `cap06` com só o `disabled`, aplicado por `medicoes/cap06-so-disabled.patch`.

`medicoes/cap06.py · Medido`

```
tela cap06-antes · api base
  prontuários de Rosa: 2 ['p101', 'p102']
  a tela diz: Paciente p102 admitido.
```

```
tela cap06 sem ref · api base
  prontuários de Rosa: 2 ['p101', 'p102']
  a tela diz: Paciente p102 admitido.

tela cap06 · api base
  prontuários de Rosa: 1 ['p101']
  a tela diz: Paciente p101 admitido.

tela cap06 · api cap06
  prontuários de Rosa: 1 ['p101']
  a tela diz: Paciente p101 admitido.
```

`disabled` **não bastou.** Ele só chega ao DOM no render seguinte, e o segundo clique veio antes. Desabilitar o botão é um aviso visual de que algo está em curso, não uma trava. A trava precisa valer no instante do clique, e para isso serve um `useRef`: uma caixa que guarda um valor entre renders sem provocar render, e que muda na hora.

FormAdmissao.tsx · a trava cap06

```
  // Roda antes da action, no mesmo instante
  // do clique: o ref não espera um render.
  function travar(e: FormEvent) {
    if (emCurso.current) e.preventDefault();
    else emCurso.current = true;
  }
```

O `onSubmit` roda antes da action. O primeiro clique liga a trava; o segundo encontra a trava ligada e cancela o envio. O `finally` da action solta a trava, com sucesso ou com erro.

ATENÇÃO · O TESTE QUE NÃO PERCEBEU

O teste "dois cliques no mesmo instante" falha em `cap06-antes` e passa em `cap06`, mas também passa com só o `disabled`. O jsdom, o navegador simulado do Vitest, não reproduz essa corrida. Por isso a prova deste capítulo é a medição no Chromium. O capítulo 16 volta a esse limite: cada ambiente de teste prova uma coisa.

6.7

A segunda barreira mora no servidor

A trava do ref vale para um formulário, numa aba. Duas abas abertas, um clique em cada, e a tela não tem como saber da outra. Só o servidor vê os dois pedidos. Por isso a API ganhou, na tag `cap06`, uma regra: o CNS identifica a pessoa, e um segundo cadastro com o mesmo CNS responde **409 Conflict**.

`medicoes/cap06.py · Medido`

```
duas abas, mesmo paciente
  api base: 2 prontuário(s)
    aba 2 diz: Paciente p102 admitido.
  api cap06: 1 prontuário(s)
    aba 2 diz: Já existe paciente com este CNS.
```

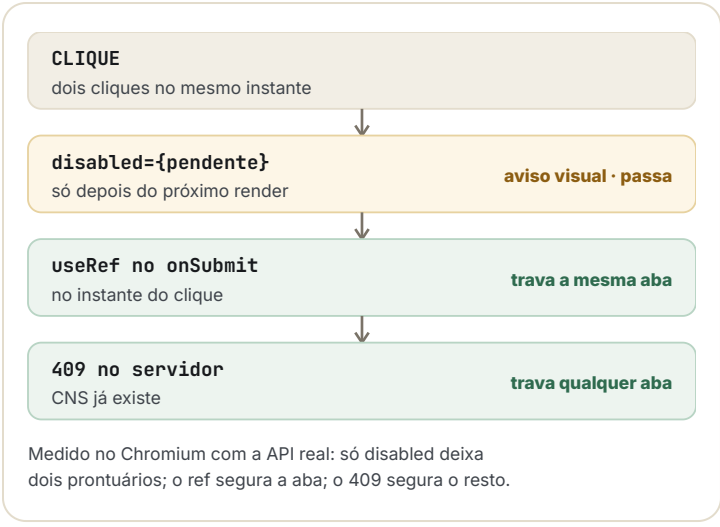


Fig. 6.2 — As barreiras contra o envio duplo, na ordem em que o clique as encontra. A terceira é a única que vale para qualquer cliente.

A função `admitir` trata o 409 à parte e mostra a mensagem do servidor. Antes dessa correção, ele caía em "Falha no servidor. Tente de novo.", uma mensagem que convida exatamente ao clique que gerou o problema.

REGRA · UI-04

Ação que cria registro trava no clique, com `useRef`, e o servidor recusa duplicata com 409. `disabled` é aviso visual, não trava. Verificação: os testes do formulário e `medicoes/cap06.py`.

H+ **O carimbo da admissão**

Na recepção, a ficha carimbada não é carimbada de novo: quem pegou a ficha segura a vez. E o arquivo central confere o CNS antes de abrir uma pasta, porque outra recepção pode estar admitindo a mesma pessoa.

ONDE A ANALOGIA PARA

O carimbo físico é visível na hora. O `disabled` do React não: ele aparece um render depois, e o ref, que trava de verdade, nem aparece na tela.

6.8

Com agentes de IA

ARMADILHA DE IA

Pedido um formulário "que não envie duas vezes", agentes costumam entregar só o `disabled`, porque é o exemplo da documentação. Também é comum a validação ficar só no cliente, e os erros do servidor caírem numa mensagem genérica. A UI-04 e a UI-05 dizem ao agente o que vale.

PROMPT · AGENTE · CAP 6

```
Goal
Formulário de novo atendimento (POST
/api/encounters) que nunca cria dois.
```

```
Context
@src/features/admissao/ @INVARIANTES.md

Constraints
- useActionState + trava com useRef
  (UI-04); disabled não é trava
- estados num tipo só, como Resultado
- 400 no campo; 403 e 409 em role=alert
- não altere a API

Done when
- npm run gate verde
- teste: servidor lento, dois cliques,
  um fetch; falha sem o ref
- teste: 403 mostra a mensagem do papel
```

Exercícios

ESSENCIAL No `FormLogin`, troque o estado `enviando` por um `useActionState`. O que acontece com os campos controlados depois de um login com erro?

INTERMEDIÁRIO Faça o botão "Admitir" só habilitar quando os três campos estiverem preenchidos, sem criar estado novo. Que tipo de campo isso exige?

DESAFIO Rode `medicoes/cap06.py` com a rede 3 segundos mais lenta e observe o que a pessoa vê enquanto espera. Proponha o texto e o estado visual que faltam.

Régua: 07, 09; prepara 08. **Invariantes:** UI-04, UI-05. **Tags:** `cap06-antes`, `cap06` (tela) e `cap06` (API). **Prompts:** `prompts/cap06.md`.

Fontes. react.dev: *State as a Snapshot*, *<form>*, *useActionState*, *useRef*. RFC 9110 (409). Saídas em `saidas/c6-*.txt`.



PARTE II · OS FUNDAMENTOS

07

Efeitos: sincronizar com o mundo lá fora

Efeito não é "rodar código depois do render". É manter algo fora do React em dia com o estado, e desfazer isso quando o estado muda.

Terça, 16h. A médica procura "mar" na busca do prontuário. A tela mostra Maria Souza e, um instante depois, João Lima também. João não tem "mar" no nome. Ela recarrega, digita de novo, e o erro não volta.

Bug que aparece e some é quase sempre uma questão de tempo. Aqui, duas respostas do servidor chegaram fora de ordem, e a mais velha venceu. Este capítulo trata do único lugar do React feito para falar com o

mundo lá fora, o efeito, e de como ele erra quando ninguém pensa em limpeza.

7.1

As regras dos Hooks

Funções que começam com `use` são **Hooks**. O React guarda o estado deles pela *ordem* em que são chamados em cada render, por isso eles só podem ser chamados no nível de cima de um componente ou de outro Hook: nunca dentro de `if`, laço ou função comum. A regra `react/rules-of-hooks` do oxlint já vem como erro no template.

7.2

Três tipos de lógica

Todo código de um componente cabe em um de três lugares, e só um deles é um efeito:



Fig. 7.1 — Em cima, a pergunta que decide se você precisa de um efeito. Embaixo, o ciclo de um efeito com limpeza, com a busca deste capítulo como exemplo.

Total e lista filtrada são cálculo. Enviar a admissão é resposta a evento. Buscar o termo atual no servidor é sincronização. A diferença entre os dois últimos é a causa. O envio acontece porque alguém clicou; a busca precisa acontecer porque o termo *é* "mar", não importa como ele chegou a ser. Efeito é para o que precisa ficar em dia com o estado, seja qual for o caminho até ele.

H+ **O pedido ao laboratório**

A enfermaria não faz o exame: pede ao laboratório, que fica fora dela. Se o médico troca o pedido antes do resultado sair, alguém precisa cancelar o pedido antigo, ou o resultado errado chega ao prontuário.

ONDE A ANALOGIA PARA

O laboratório devolve um resultado só. Um efeito pode rodar e se desfazer muitas vezes por minuto, a cada mudança de dependência, e a limpeza roda sempre antes do próximo setup.

7.3

A busca sem limpeza

Na tag `cap07-antes`, a busca do prontuário pede ao servidor, a cada tecla, os pacientes cujo nome contém o termo. A API ganhou o filtro `?q=` na tag `cap07`:

BuscaProntuario.tsx — cap07-antes

```
export function BuscaProntuario() {
  const [termo, setTermo] = useState("");
  const [achados, setAchados] =
    useState<Achado[]>([]);
  const [total, setTotal] = useState(0);

  useEffect(() => {
    if (!termo) return;
    const url = "/api/patients?q="
```

```
      + encodeURIComponent(termo)
      + location.search.replace("?", "&");
    fetch(url)
      .then((r) => r.json())
      .then((dados) => setAchados(dados));
  }, [termo]);

  // "manter o total em dia"
  useEffect(() => {
    setTotal(achados.length);
  }, [achados]);
```

O efeito declara que depende de `termo` e roda a cada mudança. Cada execução dispara um `fetch` e esquece dele. Se a resposta de "ma" chegar depois da de "mar", ela sobrescreve a tela. O `location.search` só existe para o laboratório: repassa à API o parâmetro `?atraso=`, que fora de produção segura a resposta de um termo escolhido.

Este roteiro abre a tela com `?atraso=900:lento:ma`, digita "ma" e, logo depois, "r", e registra cada pedido e cada resposta:

`medicoes/cap07.py · Medido`

```
== cap07-antes
   30 ms  pede  q=m
   32 ms  pede  q=ma
   35 ms  chega q=m
  196 ms  pede  q=mar
  204 ms  chega q=mar
  945 ms  chega q=ma
  termo na tela: 'mar'
  resultados:    ['Maria Souza', 'João Lima']
```

A resposta de "mar" chegou em 204 ms e mostrou Maria. A de "ma", pedida antes, chegou em 945 ms e trocou a tela pela lista de "ma", com João. O termo na tela continua "mar". Em produção, quem decide a ordem é a rede, não um parâmetro.

O segundo efeito do componente tem outro defeito: copia `achados.length` para um estado `total`. É um valor derivado guardado em dobro, e cada busca custa um render a mais. O oxlint avisa, mas aviso não derruba o portão:

```
terminal · Medido

$ npx oxlint
src/features/prontuario/BuscaProntuario.tsx:25:5
  warning react(set-state-in-effect)
  Calling setState synchronously within an effect
can trigger cascading renders
```

7.4

A correção: limpar e cancelar

Na tag `cap07`, a busca virou um **Hook personalizado**, uma função comum cujo nome começa com `use` e que chama outros Hooks:

```
src/features/prontuario/useBuscaPacientes.ts  cap07

export function useBuscaPacientes(
  termo: string,
) {
  const [achados, setAchados] =
```

```
    useState<Achado[]>([]);

  useEffect(() => {
    if (!termo) return;
    const ctrl = new AbortController();
    const url = "/api/patients?q="
      + encodeURIComponent(termo)
      + location.search.replace("?", "&");
    fetch(url, { signal: ctrl.signal })
      .then((r) => r.json())
      .then((dados) => setAchados(dados))
      .catch((e: Error) => {
        // cancelado de propósito: não é erro
        if (e.name !== "AbortError") throw e;
      });
    return () => ctrl.abort();
  }, [termo]);

  return termo ? achados : [];
}
```

A função devolvida pelo efeito é a **limpeza**. O React a chama antes de rodar o efeito de novo e quando o componente sai da tela. Aqui ela cancela o pedido com `AbortController`: o `fetch` antigo é interrompido e a resposta dele nunca chega ao `setAchados`. O `catch` ignora só o cancelamento, que não é erro.

medicoes/cap07.py · Medido

```
== cap07
   17 ms  pede  q=m
   23 ms  pede  q=ma
   29 ms  cancelado q=m
  188 ms  pede  q=mar
  191 ms  cancelado q=ma
  198 ms  chega q=mar
  termo na tela: 'mar'
  resultados:    ['Maria Souza']
```

Quando o termo virou "mar", a limpeza cancelou "ma" em 191 ms. A documentação oficial mostra outra forma de limpeza, uma variável `ignore` que descarta a resposta velha; ela evita o bug, mas deixa o pedido seguir na rede. O `AbortController` evita o bug e economiza o servidor.

O componente ficou com uma linha para os achados e uma para o total, agora calculado:

src/features/prontuario/BuscaProntuario.tsx cap07

```
…
export function BuscaProntuario() {
  const [termo, setTermo] = useState("");
  const achados = useBuscaPacientes(termo);
  const total = achados.length; // derivado
…
```

REGRAS · UI-06 E UI-07

UI-06: efeito só sincroniza com sistema externo; nunca copia um estado para outro. Verificação: `react/set-state-in-effect` agora como **erro** no `.oxlintrc.json`, junto com `react/exhaustive-deps`.

UI-07: efeito que busca dados cancela o pedido na limpeza. Verificação: o teste da corrida.

7.5

A prova sem servidor

A medição usa a API real. O teste não pode depender da rede, então ele cria uma rede falsa em que nenhum pedido responde sozinho: o teste decide a ordem.

BuscaProntuario.test.tsx — cap07

```
test("resposta velha não sobrescreve a nova",
  async () => {
    const fila = redeControlada({
      ma: ["Maria Souza", "João Lima"],
      mar: ["Maria Souza"],
    });
    render(<BuscaProntuario />);
    const campo = screen.getByLabelText(
      "Buscar no prontuário");
    await userEvent.type(campo, "ma");
    await userEvent.type(campo, "r");
    // "mar" responde primeiro; "ma", depois
    const porTermo = (t: string) => fila.find(
      (p) => p.url.includes(`q=${t}&`)
        || p.url.endsWith(`q=${t}`))!;
    await act(async () =>
```

```
      porTermo("mar").responder());
    await act(async () =>
      porTermo("ma").responder());

    expect(nomes())
      .toEqual(["Maria Souza"]);
    expect(screen.getByText(/1 encontrado/))
      .toBeTruthy();
  });
```

CAP07-ANTES

```
× resposta velha não sobrescreve a nova
→ expected [ 'Maria Souza', 'João Lima' ]
  to deeply equal [ 'Maria Souza' ]
```

CAP07

```
✓ resposta velha não sobrescreve a nova
```

A rede falsa respeita o `signal`: um pedido cancelado rejeita com `AbortError`, como o `fetch` de verdade. Se ela ignorasse o cancelamento, a versão corrigida falharia, e o teste estaria testando a rede falsa, não o componente.

7.6
StrictMode e a limpeza

Em desenvolvimento, o `StrictMode` monta cada componente, desmonta e monta de novo, rodando setup, limpeza e setup. É um teste automático da

limpeza: se o efeito estiver certo, ninguém percebe; se faltar limpeza, aparecem dois pedidos, duas conexões, dois ouvintes de evento. A regra da documentação é que o usuário não deve conseguir distinguir uma montagem de setup, limpeza e setup. Não desligue o `StrictMode` para "resolver" o efeito duplicado: corrija a limpeza.

7.7

useRef, useMemo e useCallback

O `useRef` apareceu no capítulo 6 como trava. Ele guarda um valor que sobrevive aos renders, muda na hora e não provoca render. Serve também para guardar um nó do DOM (`ref={campo}`), por exemplo para dar foco. Não leia nem escreva `ref.current` durante o render: o render precisa ser puro.

`useMemo` guarda o resultado de um cálculo e `useCallback` guarda uma função, ambos até uma dependência mudar. Eles existem por dois motivos: evitar repetir um cálculo caro e manter a mesma referência entre renders, para que um filho memoizado não renderize à toa ou um efeito não rode de novo.

DADO · REACT.DEV, REACT COMPILER 1.0 (OUT. 2025)

O compilador memoiza automaticamente. Para código novo, a documentação recomenda deixar a memoização com ele e usar `useMemo` e `useCallback` como válvula de escape, quando for preciso controle exato. O projeto liga o compilador no capítulo 16, com medição antes e depois.

Por isso o Mini-Prontuário ainda não usa nenhum dos dois; meça antes de usar. Por fim, o `useEffectEvent`, estável desde o React 19.2, cria uma função que o efeito chama e que sempre vê os valores atuais sem entrar nas dependências: por exemplo, registrar a busca em log com o nome de quem está logado sem refazer a busca quando a sessão muda.

7.8

Você talvez não precise de um efeito

A documentação oficial tem uma página inteira sobre efeitos desnecessários. No Mini-Prontuário, eles teriam esta cara:

TENTAÇÃO	NO LUGAR
Efeito que filtra a lista quando a busca muda	Filtre no render (cap. 3)
Efeito que copia `achados.length` para `total`	Calcule no render (este capítulo)
Efeito que envia a admissão quando `enviar` vira `true`	Envie no handler ou na action (cap. 6)
Efeito que zera o formulário quando muda o paciente	Troque a `key` do formulário (cap. 3)
Efeito que avisa o pai que algo mudou	Chame o callback no handler (cap. 5)
Efeito que busca dados	Pode ficar, com limpeza; no cap. 14, uma biblioteca de dados

7.9

Com agentes de IA

ARMADILHA DE IA

É o tema em que código gerado mais erra. Três vícios aparecem juntos: `fetch` em efeito sem limpeza; `setState` no efeito para derivar um valor; e dependência removida da lista "para parar o loop", que esconde o bug em vez de corrigi-lo. Com `set-state-in-effect` e `exhaustive-deps` como erro, os dois últimos não passam do portão; o primeiro, o teste da corrida pega.

PROMPT · AGENTE · CAP 7

```
Goal
Busca do prontuário com "buscando…" e
mensagem quando a rede falhar.

Context
@src/features/prontuario/ @INVARIANTES.md

Constraints
- o hook devolve dados, carregando e
  erro num tipo só
- AbortError não é erro de rede
- não mexa em .oxlintrc.json
- fora de escopo: debounce, cache

Done when
- npm run gate verde
- o teste da corrida passa sem mudar
- teste novo: rede que rejeita mostra
  a mensagem; um cancelamento, não
```

PONTE · DEV DE IA, CAPS. 9 E 13

"Não mexa em `.oxlintrc.json`" existe porque um agente diante de um erro de lint às vezes desliga a regra. Restrições que protegem o portão estão no capítulo 9; a revisão em contexto novo, que pega esse tipo de atalho, no 13.

Exercícios

ESSENCIAL Troque o `AbortController` pela variável `ignore` da documentação e rode o teste da corrida. Passa? Rode `medicoes/cap07.py` e compare as linhas "cancelado".

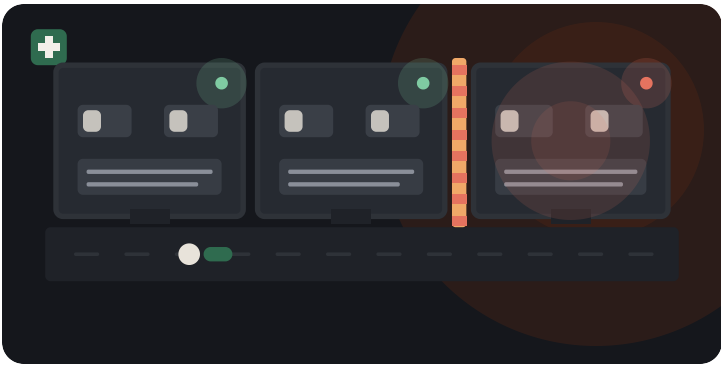
INTERMEDIÁRIO Coloque o foco no campo de busca quando a tela abre, com `useRef`. Isso precisa de efeito? Por quê?

DESAFIO Acrescente um atraso de 300 ms depois da última tecla antes de buscar (*debounce*), sem biblioteca, como um Hook `useAtrasado(valor, ms)`. Prove com um teste que três teclas rápidas geram um pedido só.

Régua: 10, 11, 12, 13, 36. **Invariantes:** UI-06, UI-07. **Tags:** `cap07-antes`, `cap07` (tela) e `cap07` (API). **Prompts:** `prompts/cap07.md`.

Fontes. react.dev: *Synchronizing with Effects*, *You Might Not Need an Effect*, *useEffectEvent*, *React Compiler*. MDN:

AbortController. Saídas em `saidas/c7-*.txt`.



PARTE III · ARQUITETURA

08

Estrutura de projeto por funcionalidade

Comece simples; reorganize quando a dor aparecer; deixe uma ferramenta vigiar as fronteiras e um limite conter cada falha.

Quarta, 7h. A recepcionista abre o Mini-Prontuário, ainda sem entrar, e digita "ma" na busca. A tela inteira fica em branco: login, admissão, a Ala B, tudo.

Um erro numa busca derrubou seis painéis. No mesmo dia, a revisão acha o prontuário importando um componente de dentro da Ala. Dois sintomas de um projeto sem planta: ninguém escreveu quem fala com quem, nem até onde uma falha vai.

8.1

Três estágios, e quando mudar

Estrutura não se escolhe no primeiro dia. Um arquivo só serve até você rolar mais do que lê; pastas por tipo (`components/`, `hooks/`) servem até uma mudança de tela tocar quatro pastas. A partir daí, organize por funcionalidade.

Na organização por funcionalidade, cada pasta de `features/` guarda tudo de uma funcionalidade: componentes, hooks, tipos e testes. É a fatia vertical do *Arquitetura em Tempos de IA*, capítulo 4, aplicada à tela. Em volta delas ficam três pastas sem domínio: `app/` monta a tela, `compartilhado/` guarda peças que qualquer uma usa e `api/` fala com o servidor.

H+ **A planta do hospital**

Cada setor tem seus leitos, seu posto e sua equipe. Todos dão para o mesmo corredor, e ninguém atravessa a UTI para chegar à farmácia. Entre os setores há portas corta-fogo: um incêndio na cozinha fica na cozinha.

ONDE A ANALOGIA PARA

Parede de hospital não se move; pasta se move com um comando, e um `import` atravessa qualquer parede em silêncio. A planta do código precisa de uma ferramenta que confira as paredes a cada portão.

8.2

A dor, medida

Na tag `cap08-antes`, um agente recebeu o pedido "a busca do prontuário deve ter a mesma barra da Ala" e reaproveitou o componente que já existia:

prontuario/BuscaProntuario.tsx — cap08-antes

```
import { useState } from "react";
import { BarraBusca }
  from "../pacientes/BarraBusca.tsx";
import { useBuscaPacientes }
  from "./useBuscaPacientes.ts";
```

O portão passa, mas agora mudar a Ala pode quebrar o prontuário sem aviso. O segundo problema é o da recepção. Este roteiro sobe a API real e a tela e digita "ma" sem login e com a API fora do ar (logado, tudo funciona):

medicoes/cap08.py · Medido

```
tela cap08-antes
 sem login
  painéis na tela: 0
  a busca diz: (a seção sumiu)
  erro na página: TypeError: achados.map is not a
function
 API fora do ar
  painéis na tela: 6
  a busca diz: 0 encontrado(s) para “ma”
  erro na página: SyntaxError: Failed to execute
'json' on 'Response': Unexpected end of JS
```

Sem login, a API responde 401 com `{"error": …}`. O código fazia `r.json()` sem olhar o status, guardava o objeto como lista e o `.map` lançou no render. Sem nada que contenha o erro, o React desmonta a árvore inteira. Com a API fora do ar é pior: a tela fica de pé e *mente*, porque "0 encontrado(s)" parece resposta.

8.3
A nova planta

Na tag `cap08`, `BarraBusca` desceu para `compartilhado/`, sem nada de domínio, e ganhou a prop `rotulo`. `App` e o verificador da API foram para `app/`. A regra de quem importa quem cabe numa figura:

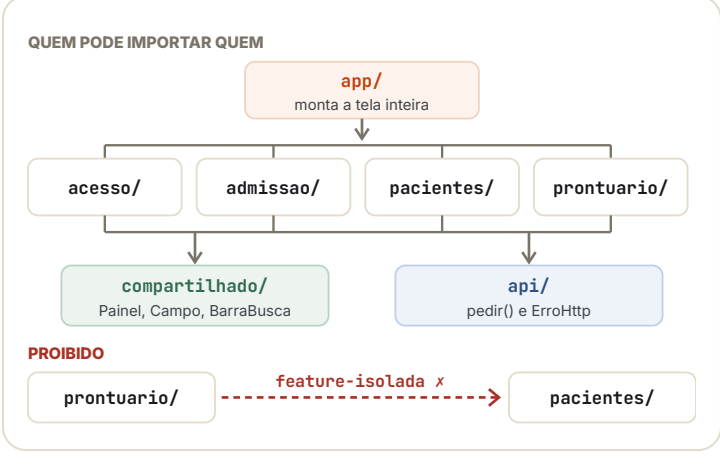


Fig. 8.1 — As dependências só descem. O que duas features usam vai para `compartilhado/`, sem domínio; a busca não importa nada da Ala.

8.4

Uma porta só para a rede

Login, admissão, busca e status tinham cada um o seu `fetch` e a sua forma de ler a resposta. Agora todos passam por `api/cliente.ts`:

api/cliente.ts cap08

```ts
export async function pedir<T>(
  caminho: string, o: Opcoes = {},
): Promise<T> {
  const temCorpo = o.corpo !== undefined;
  const r = await fetch(caminho, {
    method: o.metodo ?? "GET",
    signal: o.sinal,
    headers: temCorpo
      ? { "Content-Type": "application/json" }
      : undefined,
    body: temCorpo
      ? JSON.stringify(o.corpo) : undefined,
  });
  const tipo = r.headers.get("content-type");
  const json = tipo?.includes("json");
  const corpo: unknown = json
    ? await r.json() : await r.text();
  if (!r.ok) {
    throw new ErroHttp(r.status, corpo);
  }
  // o tipo T é promessa, não verificação
  return corpo as T;
}
```

Resposta fora de 2xx vira `ErroHttp` com status e corpo; quem chama recebe o dado ou uma exceção, nunca um `Response` para interpretar. O `T` é promessa

ao TypeScript, não verificação: validar a resposta com Zod fica para o capítulo 14.

Para que ninguém volte a chamar `fetch` por fora, a regra `no-restricted-globals` do oxlint proíbe o global, com uma mensagem que aponta para `pedir()`; um bloco `overrides` a desliga em `src/api/` e nos testes, que simulam a rede.

8.5

Erro esperado, erro inesperado

Um 401 na busca é previsível, e a feature sabe o que dizer. Um 500 ou uma rede caída não têm frase pronta: o melhor é isolar o estrago.

prontuario/useBuscaPacientes.ts cap08

```
…
      .catch((e: unknown) => {
        if (sinal.aborted) return;
        const aviso = e instanceof ErroHttp
          ? AVISO[e.status] : undefined;
        setBusca(aviso
          ? { achados: [], aviso }
          : { achados: [], falha: e });
      });
    return () => ctrl.abort();
  }, [termo]);

  if (!termo) return VAZIA;
  // Inesperado sobe para o LimiteDeErro.
  if (busca.falha) throw busca.falha;
…
```

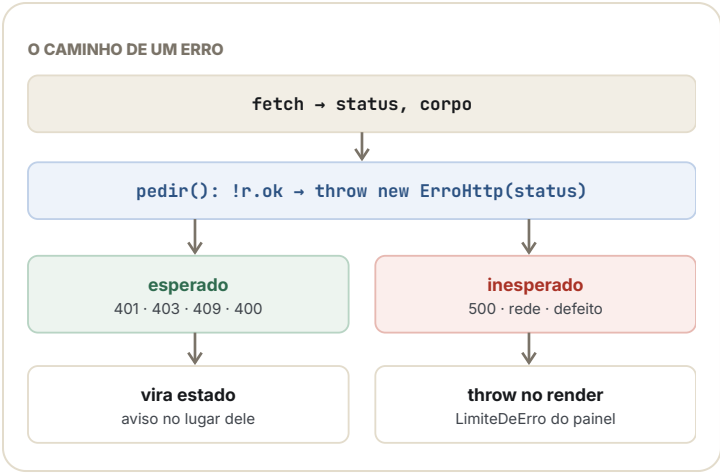


Fig. 8.2 — Dois caminhos depois do cliente. O esperado vira estado e aparece no lugar dele; o inesperado é relançado no render e o painel o contém.

O `.catch` roda numa promessa, fora do render, e um erro ali não chega a nenhum limite. Por isso o hook guarda a falha em estado e a relança no render, na penúltima linha, onde o React sabe tratá-la. Quem a recebe é o **limite de erro** (*error boundary*):

compartilhado/LimiteDeErro.tsx — cap08

```
…
export class LimiteDeErro
  extends Component<Props, Estado> {
  state: Estado = { falhou: false };

  static getDerivedStateFromError(): Estado {
    return { falhou: true };
  }
…
```

DADO · REACT.DEV, COMPONENT

Limite de erro ainda é classe: não há Hook equivalente a `componentDidCatch`. Ele captura erros do render dos filhos, não de handlers, código assíncrono, SSR ou dele mesmo. Exceção: erro dentro de `startTransition` chega ao limite, e por isso uma action do React 19 que lança cai nele.

No `render`, quando `falhou` é verdadeiro, o limite mostra "algo falhou" e um botão "Tentar de novo" no lugar dos filhos. Onde pôr os limites é decisão de produto. Aqui, cada `Painel` embrulha os filhos num `LimiteDeErro` com o próprio título: a moldura que já existia virou a porta corta-fogo. A mesma busca, na tag `cap08`:

medicoes/cap08.py · Medido

```
tela cap08
 sem login
  painéis na tela: 6
  a busca diz: Entre para buscar no prontuário.
  erro na página: nenhum
 API fora do ar
  painéis na tela: 6
  a busca diz: Busca no prontuário: algo falhou. /
Tentar de novo
  erro na página: nenhum
```

Sem login, a busca avisa. Com a API fora, só o painel da busca cai; os outros cinco continuam. "Tentar de novo" remonta os filhos do zero, com o estado limpo.

8.6

Fronteiras no portão

A planta só vale se alguém conferir. O instrumento `fronteiras` é o dependency-cruiser com as regras do projeto, rodando em `npm run gate`:

.dependency-cruiser.cjs cap08

```
…
    {
      name: "feature-isolada",
      comment: "Feature não importa outra "
        + "feature. O que duas usam vai "
        + "para compartilhado/.",
      severity: "error",
      from: {
        path: "^src/features/([^/]+)/" },
      to: {
        path: "^src/features/",
        pathNot: "^src/features/$1/",
      },
    },
…
```

O `$1` repete o nome capturado na origem: de `features/prontuario/` só se vai a `features/prontuario/`. Outras duas regras proíbem a base de importar telas e proíbem ciclos. As regras do `cap08` sobre o código do `cap08-antes`:

terminal · Medido

```
$ npx depcruise src

  error feature-isolada:
src/features/prontuario/BuscaProntuario.tsx →
src/features/pacientes/BarraBusca.tsx

x 1 dependency violations (1 errors, 0 warnings).
33 modules, 65 dependencies cruised.
```

E a prova de revert do erro de tela, no teste que entrou na tag `cap08-antes`:

CAP08-ANTES

```
× sem login, avisa em vez de quebrar
TypeError: achados.map is not
  a function
```

CAP08

```
✓ sem login, avisa em vez de quebrar
✓ painel quebrado não derruba vizinho
```

REGRAS · UI-08 A UI-10

UI-08: feature não importa feature; base não importa tela; sem ciclos (`npm run fronteiras`). **UI-09:** só `src/api/` chama `fetch` (`no-restricted-globals`). **UI-10:** erro esperado vira aviso; o inesperado sobe ao limite do painel (os dois testes acima).

8.7

Com agentes de IA

ARMADILHA DE IA

Diante de um erro na busca, o agente costuma escrever `catch { return [] }`: a tela não cai e passa a mentir "0 encontrado(s)", como em 8.2. Diante de `feature-isolada`, copia o componente ou abre exceção em `pathNot`. Peça que o erro inesperado apareça, proteja a configuração do portão e peça a refatoração em passos pequenos, com o portão verde entre eles.

PROMPT · AGENTE · CAP 8

```
Goal
Admissão mostra "Sessão expirada" no 401;
com a rede caída, só o painel avisa.

Context
@docs/UI.md @INVARIANTES.md @src/api/

Constraints
- só src/api chama fetch (UI-09)
- erro inesperado não vira lista vazia
  nem mensagem genérica: sobe ao limite
- não edite .dependency-cruiser.cjs
  nem .oxlintrc.json

Done when
- npm run gate verde
- teste novo: 401 mostra o aviso
- teste novo: fetch que rejeita mostra
```

```
"algo falhou" em Admissão, e o painel
Pacientes continua
```

> **PONTE · DEV DE IA, CAPS. 5 E 7**
>
> UI-08 a UI-10 entraram no `INVARIANTES.md` com a forma de verificar (cap. 5), e a verificação virou comando do portão (cap. 7). Regra sem verificação é sugestão.

Exercícios

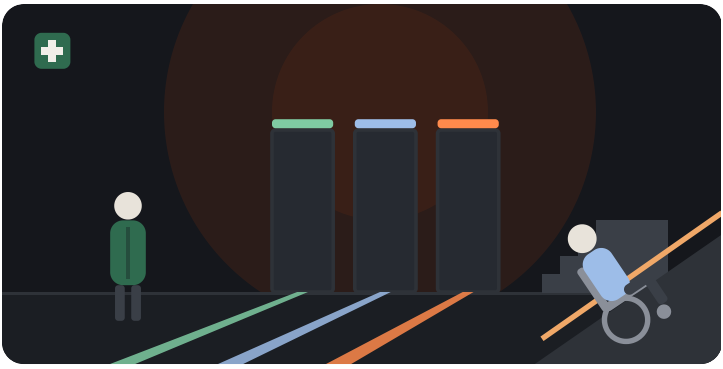
ESSENCIAL Mova `SeloRisco` para `compartilhado/` sem mudar nada e rode `npm run fronteiras`. Qual regra falha, e por quê?

INTERMEDIÁRIO Faça `StatusApi` distinguir rede caída de resposta 500. A diferença mora no cliente ou no componente?

DESAFIO Dê a cada feature um `index.ts` público e uma regra que proíba importá-la por outro caminho. Prove que `app/` não alcança mais `useBuscaPacientes`.

Régua: 14, 35. **Invariantes:** UI-08, UI-09, UI-10. **Tags:** `cap08-antes`, `cap08` (tela) e `cap07` (API). **Prompts:** `prompts/cap08.md`.

Fontes. react.dev: *Component* (*Catching rendering errors with an Error Boundary*), *useTransition*. dependency-cruiser: *rules reference*. Oxc: *no-restricted-globals*. *Arquitetura em Tempos de IA*, cap. 4. Saídas em `saidas/c8-*.txt`.



PARTE III · ARQUITETURA

09

Estilo, tokens e acessibilidade

Cor com significado, foco que se vê, contraste que se calcula. Bonito é o que todo mundo consegue usar, nos dois temas.

Sexta, 22h. O enfermeiro do plantão usa o celular no modo escuro. Na lista da Ala B, o risco dos pacientes virou uma pílula clara com letras brancas, quase ilegível. De dia, no computador da recepção, tudo parecia ótimo.

Estilo parece gosto até alguém não conseguir ler a tela. O capítulo troca o gosto por três coisas verificáveis: **tokens** com nome de função, **escopo** por componente e auditoria de acessibilidade com teclado de verdade.

9.1

Quatro jeitos de estilizar

JEITO	ESCOPO	CUSTO
CSS global	a página	toda regra pode vazar
CSS Modules	o componente	um arquivo por componente
Tailwind	o elemento	marcação longa; mais build
Kit de componentes	o kit	as decisões são do kit

O projeto usa CSS Modules com tokens (decisão D4) e, em 9.5, compara com Tailwind. Nenhum dos quatro, sozinho, resolve contraste ou foco.

H+ **O piso sinalizado**

No hospital, a faixa verde no chão leva à emergência e a azul ao ambulatório. Toda placa repete o destino por escrito, e ao lado de cada escada há uma rampa. A cor ajuda quem enxerga cor; o texto e a rampa garantem que todos cheguem.

ONDE A ANALOGIA PARA

O piso é pintado uma vez. A tela troca de tema com um clique do sistema operacional, e cada cor precisa funcionar nas duas versões. Por isso o token guarda um par de valores, não uma tinta.

9.2

"Deixe com cara de produto"

Na tag `cap09-antes`, um agente recebeu o pedido "deixe com cara de sistema moderno, com tema escuro" e entregou um `index.css` global de 48 linhas. O trecho decisivo:

index.css — cap09-antes

```
…
button { background: #2563eb; color: #fff;
  border: 0; border-radius: 6px;
  padding: 4px 10px; cursor: pointer; }
button:focus { outline: none; }
button[aria-pressed="true"] {
  background: #93c5fd; }

.leito { color: #9ca3af; font-size: 13px; }
.selo { font-size: 12px; padding: 1px 6px;
  border-radius: 999px; background: #e5e7eb;
  color: #fff; }
…
```

Numa olhada, está bonito. O instrumento `vigia-a11y` roda o axe-core com as regras WCAG 2.2 AA nos dois

temas e percorre a página com Tab, conferindo se cada parada tem contorno ou sombra visível:

```
medicoes/vigia_a11y.py · Medido

tela cap09-antes
  tema light: 1 regra(s), 19 elemento(s)
    color-contrast (serious): 19
      #9ca3af sobre #ffffff: 2.53:1 (mínimo 4.5:1)
…
  tema dark: 1 regra(s), 27 elemento(s)
    color-contrast (serious): 27
      #9ca3af sobre #ffffff: 2.53:1 (mínimo 4.5:1)
      #e5e7eb sobre #ffffff: 1.23:1 (mínimo 4.5:1)
…
  teclado: 37 paradas, 22 sem foco visível
    button “Verificar API”
    button “Entrar”
…
```

Três defeitos clássicos. O cinza `#9ca3af` sobre branco dá 2,53:1, e a WCAG pede 4,5:1 para texto. O `outline: none` sem substituto apagou o foco de 22 das 37 paradas: quem usa teclado não sabe onde está. E o tema escuro trocou só o `body`: texto claro caiu sobre painéis brancos, 1,23:1. Na tag `cap08`, sem visual nenhum, o tema escuro já tinha 6 selos ilegíveis: cor fixa e `color-scheme: light dark` não combinam.

9.3

Tokens: o nome diz a função

Na tag `cap09`, toda cor nasce num arquivo só, com nome de função e os dois temas lado a lado:

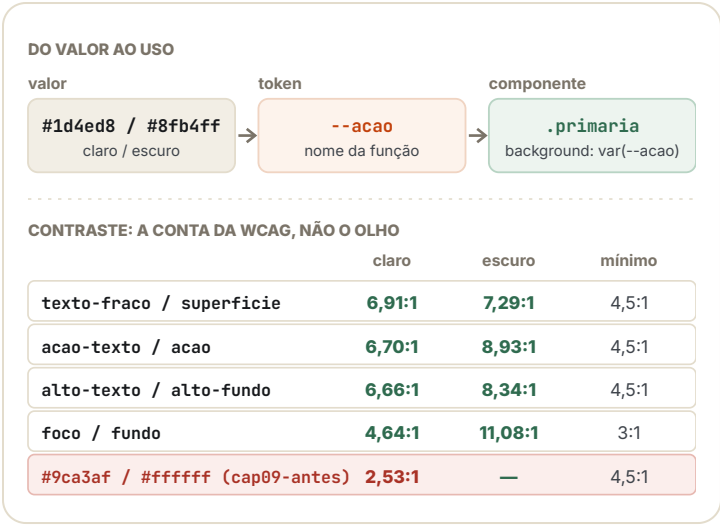


Fig. 9.1 — Em cima, as três camadas de uma cor: trocar o tema muda só a primeira. Embaixo, contrastes calculados por `tokens.test.ts`, com o cinza do `cap09-antes` para comparar.

estilo/tokens.css cap09

```
:root {
  color-scheme: light dark;

  --fundo: light-dark(#f5f6f8, #0f141a);
  --superficie: light-dark(#ffffff, #1a2129);
  --texto: light-dark(#1b2430, #e6eaef);
  --texto-fraco: light-dark(#4f5b6a, #a3afbd);
  --borda: light-dark(#d5dbe2, #33404d);
  --acao: light-dark(#1d4ed8, #8fb4ff);
  --acao-texto: light-dark(#ffffff, #0f141a);
  --foco: light-dark(#b45309, #fbbf24);
```

`light-dark(claro, escuro)` escolhe o valor pelo `color-scheme` em vigor, e é suportado pelos navegadores atuais desde 2024. Ninguém usa `#1d4ed8`; usa `var(--acao)`. Quando o tema muda, muda o valor e nenhum componente percebe.

Contraste é conta, não opinião. `tokens.test.ts` lê o arquivo de tokens, aplica a fórmula de luminância relativa da WCAG e, para cada par da lista `PARES` (texto sobre fundo, 4,5:1; foco, 3:1), exige o mínimo nos dois temas:

estilo/tokens.test.ts cap09

```
…
test.each(PARES)("%s sobre %s ≥ %s:1",
  (frente, fundo, minimo) => {
    for (const tema of [0, 1]) {
      const r = contraste(
        cor.get(frente)![tema],
        cor.get(fundo)![tema]);
      expect(r, `${frente} tema ${tema}`)
        .toBeGreaterThanOrEqual(minimo);
    }
  });
…
```

O teste também proíbe hexadecimal em qualquer `.css` fora de `tokens.css`. A prova de revert: volte o cinza antigo num módulo e rode o teste.

terminal · Medido

```
 .leito {
   font-size: var(--texto-pequeno);
-  color: var(--texto-fraco);
+  color: #9ca3af;
 }
…
 ❯ src/estilo/tokens.test.ts (12 tests | 1 failed) 8ms
   × cor só nasce em tokens.css 4ms
```

9.4

Base nativa e escopo por componente

Elementos nativos ganham estilo uma vez, em `estilo/base.css`, só com tokens. É ali que mora a regra de foco:

estilo/base.css — cap09

```
/* Nunca "outline: none" sem substituto. */
:focus-visible {
  outline: 3px solid var(--foco);
  outline-offset: 2px;
}
```

`:focus-visible` mostra o contorno no teclado e não no clique de mouse em botão: é a resposta a quem tirou o `outline` "porque o clique deixava uma borda feia". O laranja do foco tem pelo menos 3:1 contra os fundos.

O resto é CSS Module. O selo virou `SeloRisco.module.css`, e o componente usa

`className={css.selo}`. No DOM, a classe chega como `_cartao_1q05y_1`: o Vite acrescenta um sufixo por arquivo, e uma regra de um componente não alcança outro.

pacientes/SeloRisco.module.css cap09

```
…
.selo[data-risco="alto"] {
  background: var(--alto-fundo);
  color: var(--alto-texto);
}
…
```

O selo escreve "alto", "médio" ou "baixo". A cor reforça, mas não é a única pista: a WCAG pede que informação não dependa só de cor (critério 1.4.1), e cerca de um em cada doze homens tem alguma deficiência de visão de cores (Colour Blind Awareness). A nova medição:

medicoes/vigia_a11y.py · Medido

```
tela cap09
  tema light: 0 regra(s), 0 elemento(s)
  tema dark: 0 regra(s), 0 elemento(s)
  teclado: 37 paradas, 1 sem foco visível
    input “nascimento”
```

Zero violações nos dois temas. A parada que sobra é ponto cego do vigia: no botão do calendário do campo de data, o Chromium desenha o anel dentro do shadow DOM, onde o `getComputedStyle` não chega. Uma

captura de tela confirmou o anel. Instrumento também precisa de alguém olhando.

9.5

A mesma tela em Tailwind

A tag lateral `cap09-tailwind` refaz o cartão e o selo com Tailwind 4.3, sem o *preflight* e com um `@theme inline` que transforma os tokens em utilitários (`bg-alto-fundo`, `text-texto-fraco`):

pacientes/CartaoPaciente.tsx `cap09-tailwind`

```
…
    <li aria-current={atual} className="flex
      flex-wrap items-center gap-2 border-b
      border-borda py-2
      aria-[current=true]:border-l-4
      aria-[current=true]:border-l-acao
      aria-[current=true]:pl-3">
      <button className="bg-transparent p-0
        font-semibold text-texto"
        onClick={() => onSelecionar(id)}>
…
```

medicoes/cap09.py · Medido

```
tela cap09
  CSS do build: 3129 bytes, 990 com gzip
  cartão + selo: 101 linhas, 4 arquivos
  classe no DOM: _cartao_1q05y_1
  tema light: 0 regra(s), 0 elemento(s)
  tema dark: 0 regra(s), 0 elemento(s)

tela cap09-tailwind
```

```
  CSS do build: 6026 bytes, 1718 com gzip
  cartão + selo: 74 linhas, 2 arquivos
  classe no DOM: flex flex-wrap items-center gap-2
border-b b
  tema light: 0 regra(s), 0 elemento(s)
  tema dark: 0 regra(s), 0 elemento(s)
```

Os dois passam no axe: usam os mesmos tokens. Tailwind escreve menos (74 linhas em dois arquivos contra 101 em quatro, mais 26 de configuração, uma vez). O CSS final ficou maior (1,7 KB contra 1 KB comprimido). Aberto o arquivo, o excesso são 21 declarações `@property` com as variáveis internas dos utilitários e um bloco de valores iniciais para navegadores sem suporte a elas. Um detalhe pegou na montagem: `base.css` fora de camada vence qualquer utilitário em `@layer`, então foi preciso importá-lo em `layer(base)`.

O fio continua em CSS Modules porque, para quem aprende, é CSS de verdade com escopo e não esconde a cascata. Tailwind é escolha legítima numa equipe que já o usa; os tokens servem aos dois.

REGRAS · UI-11 E UI-12

UI-11: cor, espaço e raio vêm de token; hexadecimal só em `tokens.css`; todo par de texto passa 4,5:1 nos dois temas. Verificação: `tokens.test.ts`.

UI-12: toda parada de teclado tem foco visível; informação nunca só por cor. Verificação: `medicoes/vigia_a11y.py`.

9.6

Com agentes de IA

ARMADILHA DE IA

"Deixe bonito" volta com os três vícios de 9.2 (cinza-claro, `outline: none`, tema escuro só no `body`) e, quando há ação, um quarto: `<div onClick>`, que o mouse aciona e o teclado nem alcança. Nada disso quebra o portão comum: peça contraste e teclado como critério de pronto.

PROMPT · AGENTE · CAP 9

```
Goal
Destacar o paciente selecionado na lista
e o campo com erro no formulário.

Context
@docs/UI.md @src/estilo/tokens.css
@INVARIANTES.md (UI-11, UI-12)

Constraints
- só var(--token); cor nova entra em
  tokens.css com os dois temas e o par
  em tokens.test.ts
- nunca outline: none sem substituto
- erro com texto, não só borda vermelha

Done when
- npm run gate verde
- vigia_a11y.py: 0 violações nos dois
  temas; foco visível em toda parada
```

PONTE · DEV DE IA, CAP. 6

`docs/UI.md` ganhou a seção Estilo: onde ficam os tokens, como criar um, o que nunca fazer, no formato do capítulo 6 do *Dev de IA*.

Exercícios

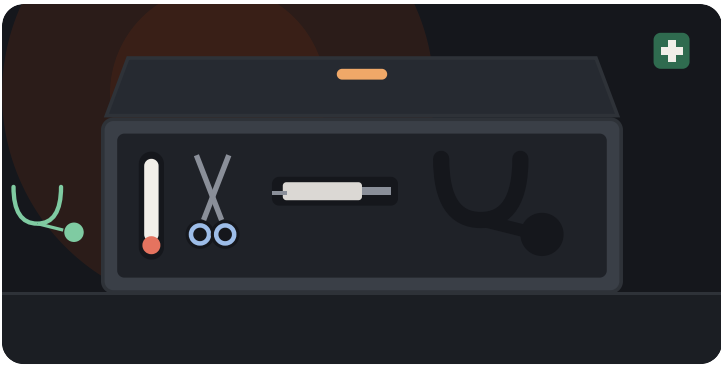
ESSENCIAL Mude o `--texto-fraco` escuro para `#6b7280`. Quem acusa primeiro, `tokens.test.ts` ou o vigia? Por quê?

INTERMEDIÁRIO Deixe a lista legível em 320 px de largura, sem rolagem horizontal, só com tokens. Prove com uma captura do Playwright.

DESAFIO Acrescente ao vigia o alvo de toque mínimo de 24 × 24 px (WCAG 2.2, 2.5.8). Quantos falham hoje?

Régua: 15, 16. **Invariantes:** UI-11, UI-12. **Tags:** `cap09-antes`, `cap09`, `cap09-tailwind`. **Prompts:** `prompts/cap09.md`.

Fontes. W3C: WCAG 2.2 (1.4.1, 1.4.3, 1.4.11, 2.4.7, 2.5.8) e a definição de luminância relativa. MDN: *light-dark()*, *:focus-visible*, *color-scheme*. Vite: *CSS Modules*. Tailwind CSS 4: *Preflight*, *Theme variables*. Deque: axe-core 4.14. Colour Blind Awareness: *About colour blindness*. Saídas em `saidas/c9-*.txt`.



PARTE III · ARQUITETURA

10

Componentes reutilizáveis e design system

Um componente de catálogo é uma decisão tomada uma vez: visual, semântica, foco e teclado. Quem usa só escolhe a variante.

Segunda, 10h. Um clique errado no botão "Alta" tirou da lista um paciente que ia ficar. A equipe pede uma confirmação. Em meia hora o agente entrega a caixa "Dar alta a Maria Souza?". Funciona com o mouse. Com o teclado, não.

A caixa de confirmação parece fácil e esconde meia dúzia de decisões: foco, Esc, fundo clicável, o que o leitor de tela anuncia, para onde o foco volta.

Resolvidas uma vez no catálogo, nenhuma tela as resolve de novo.

10.1

Genérico e de domínio

O projeto já tem dois tipos de componente. `CartaoPaciente` sabe o que é um paciente; `Painel` não sabe o que mostra. O primeiro é **de domínio** e mora na feature; o segundo é **genérico** e mora em `compartilhado/`. O catálogo do projeto, o seu pequeno *design system*, é o segundo grupo com regras escritas.

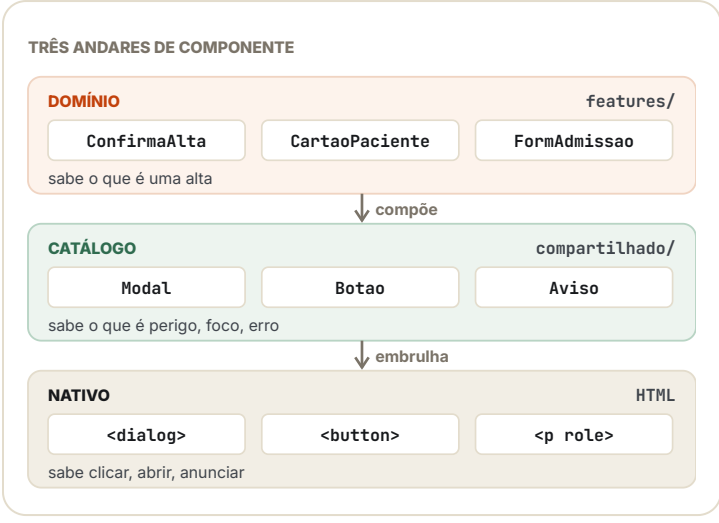


Fig. 10.1 — Três andares, e cada um só conhece o de baixo. O catálogo embrulha o HTML nativo e toma as decisões de acessibilidade uma vez.

H+ **A maleta padronizada**

Na maleta de instrumentos, cada peça tem um encaixe com a forma dela. Quem abre sabe na hora o que falta, e ninguém improvisa uma tesoura. A equipe não discute qual termômetro usar: discute o paciente.

ONDE A ANALOGIA PARA

A maleta vem pronta da fábrica. O catálogo cresce com o projeto, e cresce errado se alguém criar o encaixe antes de existir a peça. A regra do `docs/UI.md` vale aqui: peça nova entra no catálogo no terceiro uso real.

10.2

A caixa que só o mouse vê

Na tag `cap10-antes`, a confirmação é uma `div` fixa por cima da tela, com os tokens do capítulo 9:

pacientes/ConfirmaAlta.tsx cap10-antes

```
“”
    <div classNameName={css.fundo}>
      <div classNameName={css.caixa}>
        <p>Dar alta a {nome}?</p>
        <button onClick={onConfirmar}>
          Confirmar alta
        </button>{" "}
        <button onClick={onCancelar}>
          Cancelar
        </button>
      </div>
“”
```

O véu cobre a tela e bloqueia o clique no fundo, então quem testa com o mouse aprova. Este roteiro abre a confirmação com Enter no botão Alta de Maria e segue só pelo teclado. A árvore de acessibilidade vem do próprio Chromium, pelo protocolo de depuração (CDP):

medicoes/cap10.py · Medido

```
tela cap10-antes
  ao abrir, foco em: button “Alta” (fora da caixa)
  árvore de acessibilidade: 0 diálogo(s), 24
botões expostos
  8 Tabs: 8 no fundo da página, 0 na barra do
navegador
  Esc: caixa continua aberta
  (fechada pelo botão Cancelar)
  ao fechar, foco em: body
  depois da alta, foco em: body
```

O foco ficou no botão Alta, atrás do véu. Oito Tabs andaram pelos cartões de baixo, invisíveis para quem enxerga e acessíveis para quem não enxerga: o leitor de tela recebe 24 botões e nenhum diálogo. O Esc não faz nada. Ao cancelar, o botão clicado sai da árvore e o foco cai no `body`, no topo da página.

10.3

Modal com `<dialog>`

O HTML tem o elemento certo desde 2022 em todos os navegadores. Aberto com `showModal()`, o `<dialog>` vai para a camada superior da página, torna o resto inerte (nem clique, nem foco, nem leitor de tela), fecha no Esc e, ao fechar, devolve o foco a quem o abriu. O componente do catálogo só liga o estado do React a essa API:

compartilhado/Modal.tsx cap10

```
…
  const ref = useRef<HTMLDialogElement>(null);
  const idTitulo = useId();

  // Sincroniza o estado do React com a API
  // imperativa do <dialog>: efeito legítimo.
  useEffect(() => {
    const d = ref.current;
    if (!d) return;
    if (aberto && !d.open) d.showModal();
    if (!aberto && d.open) d.close();
  }, [aberto]);

  return (
```

```
    <dialog ref={ref} className={css.modal}
      aria-labelledby={idTitulo}
      onClose={onFechar}>
      <h2 id={idTitulo}>{titulo}</h2>
      {children}
    </dialog>
  );
}
```

É um efeito legítimo, no sentido do capítulo 7: sincroniza o estado `aberto` com um sistema que o React não controla, a API imperativa do elemento. O `onClose` dispara também no Esc, e por isso o pai fica sabendo. `useId` gera o id que liga o título ao diálogo, e o leitor de tela anuncia o título junto com o papel de diálogo.

pacientes/ConfirmaAlta.tsx cap10

```
  return (
    <Modal aberto={nome !== undefined}
      titulo={`Dar alta a ${nome}?`}
      onFechar={onCancelar}>
      <p>O paciente sai da lista da Ala B.</p>
      <Botao variante="secundaria"
        onClick={onCancelar}>
        Cancelar
      </Botao>{" "}
      <Botao variante="perigo"
        onClick={onConfirmar}>
        Confirmar alta
      </Botao>
  );
```

O `showModal()` põe o foco no primeiro elemento focável. Numa ação sem volta, esse elemento deve ser

o Cancelar: um Enter distraído não dá alta a ninguém. A mesma medição, na tag `cap10`:

```
medicoes/cap10.py · Medido

tela cap10
  ao abrir, foco em: button “Cancelar” (dentro da caixa)
  árvore de acessibilidade: 1 diálogo(s), 2 botões expostos
  8 Tabs: 0 no fundo da página, 3 na barra do navegador
  Esc: caixa fechou
  ao fechar, foco em: button “Alta”
  depois da alta, foco em: body
```

As paradas "na barra do navegador" são de propósito: o diálogo nativo deixa o Tab sair para a interface do navegador e voltar, e ninguém fica preso. Uma linha não mudou: depois da alta, o foco cai no `body`, porque o botão que abriu o diálogo saiu com o paciente. Esse é o desafio dos exercícios.

ATENÇÃO · O JSDOM NÃO TEM `SHOWMODAL`

O jsdom 29 não implementa `showModal()` nem `close()`; `src/teste/dialogo.ts` simula só abrir e fechar. O teste de unidade prova papel e nome do diálogo; foco, Esc e fundo inerte, só o roteiro no Chromium.

CAP10-ANTES

```
× alta pede confirmação num diálogo
→ Unable to find an accessible
  element with the role "dialog"
```

CAP10

```
✓ alta pede confirmação num diálogo
```

10.4

A API de um componente

Todo componente do catálogo responde a quatro perguntas: que **variantes** existem, o que ele **repassa** ao elemento nativo, que **padrão** ele muda e onde entra o **conteúdo**. O `Botao`:

compartilhado/Botao.tsx cap10

```tsx
// O <button> nativo com duas decisões já
// tomadas: o visual vem da variante, e o
// type é "button" (o nativo, num form,
// envia). No React 19, ref chega como prop.
export function Botao({
  variante = "primaria",
  type = "button",
  className,
  ...resto
}: Props) {
  const classes = [css.botao, css[variante],
    className].filter(Boolean).join(" ");
  return (
    <button type={type} className={classes}
      {...resto} />
```

```
  );
}
```

As variantes são poucas e com significado: `primaria` (uma por área), `secundaria`, `perigo` (ação sem volta) e `texto` (o nome clicável do paciente). `ComponentProps<"button">` e o `...resto` repassam tudo o que o nativo aceita: `onClick`, `disabled`, `aria-pressed`, o texto como `children`. O padrão que muda é o `type`: o `<button>` nativo dentro de um formulário é `submit` e envia, uma surpresa conhecida; no catálogo, envio é sempre explícito (`type="submit"`).

No React 19, `ref` chega como prop comum; o `forwardRef` de tanto código antigo deixou de ser necessário e será descontinuado. O conteúdo entra por `children`. Moldura com dois buracos, corpo e ações, recebe um segundo `ReactNode` por prop (um *slot*), nunca dez props de configuração.

O `Aviso` junta uma decisão espalhada pelas telas: `tom="erro"` vira `role="alert"`, lido na hora; `tom="info"` vira `role="status"`, lido quando o leitor fica livre.

10.5

O catálogo vigiado

Catálogo que ninguém é obrigado a usar vira sugestão. O oxlint tem a regra `react/forbid-elements`: `<button>` e `<dialog>` crus passam a ser erro fora de `Botao.tsx`

e `Modal.tsx`. Ligada antes da migração, a regra virou a lista de tarefas:

```
terminal · Medido

$ npx oxlint
src/compartilhado/LimiteDeErro.tsx:37:10: error
react(forbid-elements): <button> is forbidden.
help: Use Botao de compartilhado/ (UI-13).
src/features/acesso/FormLogin.tsx:58:8: error
react(forbid-elements): <button> is forbidden.
help: Use Botao de compartilhado/ (UI-13).
…
```

Sete botões em seis arquivos, migrados um a um com o portão verde. O `docs/UI.md` ganhou a tabela do catálogo com a regra de uso de cada peça.

Catálogo próprio faz sentido enquanto o nativo resolve o comportamento, como aqui. Para combobox, menu ou calendário, cujo teclado é difícil de acertar, uma biblioteca *headless* entrega comportamento e acessibilidade sem visual. Um kit pronto serve a painel interno com prazo curto, desde que o visual do kit seja aceitável.

REGRAS · UI-13 E UI-14

UI-13: botão é `Botao`; aviso é `Aviso`. Verificação: `react/forbid-elements`.

UI-14: caixa sobre a tela é `Modal`; ação sem volta pede confirmação, Cancelar primeiro. Verificação: `Ala.test.tsx` e `cap10.py`.

10.6

Com agentes de IA

ARMADILHA DE IA

A caixa de `div` de 10.2; o "genérico" com dez props booleanas criado antes do segundo uso; `forwardRef` e `defaultProps` de React antigo em componente novo. Com o catálogo no `docs/UI.md` e o lint, o agente acha a peça pronta e o portão recusa a improvisada.

PROMPT · AGENTE · CAP 10

```
Goal
Botão Limpar na admissão, que pede
confirmação se houver algo digitado.

Context
@docs/UI.md @INVARIANTES.md

Constraints
- use Modal e Botao; nada de dialog,
  button ou div clicável crus
- não generalize ConfirmaAlta: é o
  segundo uso, copie e espere o terceiro
- Cancelar primeiro (UI-14)

Done when
- npm run gate verde
- teste: cancelar mantém o digitado
- cap10.py adaptado: foco entra, Esc
  fecha e o foco volta ao botão
```

PONTE · DEV DE IA, CAP. 6

O `docs/UI.md` tem agora os três blocos do capítulo 6 do *Dev de IA*: onde cada coisa mora, como estilizar e o catálogo com regras de uso.

Exercícios

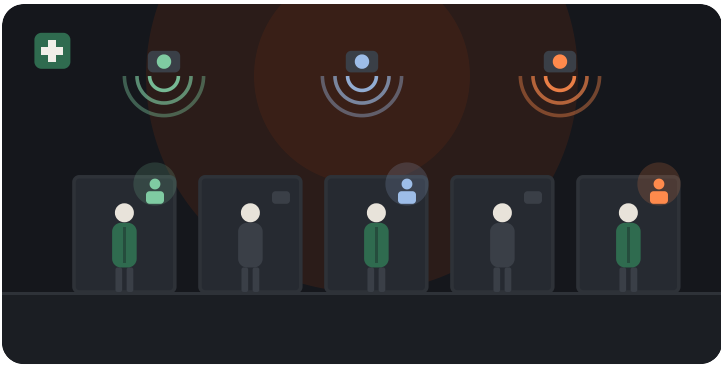
ESSENCIAL Troque a ordem dos botões em `ConfirmaAlta` e rode `cap10.py`. Que linha muda, e por que importa?

INTERMEDIÁRIO Dê ao `Campo` os estados de erro e desabilitado, só com tokens. Que pares novos entram em `tokens.test.ts`?

DESAFIO Depois da alta, leve o foco ao cartão seguinte, ou à busca se a lista esvaziar, com `ref` como prop. Prove com `cap10.py` que a última linha deixa de ser `body`.

Régua: 17, 35. **Invariantes:** UI-13, UI-14. **Tags:** `cap10-antes`, `cap10`. **Prompts:** `prompts/cap10.md`.

Fontes. HTML Living Standard: *the dialog element*. MDN: *<dialog>*, *HTMLDialogElement.showModal()*, *ARIA: alert role*, *ARIA: status role*. react.dev: *React 19* (*ref as a prop*), *useId*. Oxc: *react/forbid-elements*. WAI-ARIA APG: *Dialog (Modal) Pattern*. Saídas em `saidas/c10-*.txt`.



PARTE III · ARQUITETURA

11

Prop drilling e Context

Passar de mão em mão é explícito e bom. Até deixar de ser. Então abra um canal, um por assunto.

A tela da Ala B vai para a TV do corredor, à vista de visitantes. A coordenação pede um modo discreto: iniciais no lugar do nome, em todo lugar. "M. S." em vez de "Maria Souza".

O modo discreto é uma preferência que quase toda a tela lê: os cartões, o detalhe, a confirmação de alta e a busca, em duas features diferentes. Junto vem o tema (sistema, claro ou escuro), que só o seletor lê. Onde guardar esse estado, e como ele chega a quem precisa?

11.1

Passar de mão em mão

A primeira resposta é a do capítulo 5: o estado sobe ao ancestral comum, o `App`, e desce por props. Na tag `cap11-antes`, `discreto` vai do `App` à `Ala`, à `ListaPacientes` e ao `CartaoPaciente`; a `Ala` e a lista não usam o valor, só o repassam. Isso é o **prop drilling**.

pacientes/Ala.tsx — cap11-antes

```
…
          <ListaPacientes
            pacientes={visiveis}
            discreto={discreto}
            selecionadoId={selecionadoId}
            onSelecionar={setSelecionadoId}
            onPrioridade={alternarPrioridade}
            onAlta={setAConfirmar}
          />
…
```

Props explícitas são boas: quem lê o código vê de onde vem cada valor. O custo aparece quando o caminho fica longo. Nove testes da Ala e da busca passaram a escrever `discreto={false}` só para compilar. E um caminho foi esquecido, como o teste mostra:

```
terminal · Medido

 ❯ src/app/App.test.tsx (1 test | 1 failed) 336ms
   × modo discreto não mostra nome inteiro 335ms

⎯⎯⎯⎯⎯⎯ Failed Tests 1 ⎯⎯⎯⎯⎯⎯

 FAIL  src/app/App.test.tsx > modo discreto não
mostra nome inteiro
TestingLibraryElementError: Unable to find an
accessible element with the role "dialog" and name
"Dar alta a M. S.?"
```

A confirmação de alta, irmã da lista, não recebeu a prop e anuncia "Dar alta a Maria Souza?" na TV do corredor. É o mesmo erro do capítulo 1: um lugar a mais para lembrar.

PRIMEIRA SAÍDA: COMPOSIÇÃO

Quando o intermediário só repassa, às vezes basta passar o elemento pronto como `children`, como o `Painel` do capítulo 4: quem monta a peça já tem o valor. Aqui não basta, porque quatro consumidores em duas features precisam do mesmo valor.

H+ **Os canais do intercomunicador**

No hospital, um aviso não passa de mão em mão: sai pelo alto-falante. Cada equipe escuta o seu canal. A limpeza não reage ao "código azul", e a UTI não para pelo aviso da cozinha.

ONDE A ANALOGIA PARA

O rádio deixa cada um ignorar a parte do aviso que não lhe diz respeito. O Context, não: todo componente que lê um contexto renderiza quando o valor dele muda, por qualquer motivo. Por isso cada assunto ganha o seu próprio canal.

11.2

Context: criar, prover, ler

Um contexto tem três partes. `createContext` cria o canal com um valor padrão. Um provedor, acima na árvore, diz o valor atual; no React 19, o próprio contexto é o provedor (`<TemaContext value={tema}>`), sem `.Provider`. E quem precisa lê com `use(Contexto)`, que, ao contrário de `useContext`, pode ser chamado dentro de `if` e de laços.

exibicao/contextos.ts cap11

```
…
// Um contexto por assunto: quem lê o modo
// discreto não renderiza quando o tema muda;
// quem só age não renderiza nunca.
export const DiscretoContext =
  createContext(false);
export const TemaContext =
  createContext<Tema>("sistema");
export const AcoesContext =
  createContext<Acoes | null>(null);

export function useExibirNome() {
  const discreto = use(DiscretoContext);
```

```
  return (nome: string) =>
    discreto ? iniciais(nome) : nome;
}
```

O padrão `false` do canal discreto tem uma consequência boa: um componente renderizado sem provedor, como nos testes de unidade, recebe "nome completo" e funciona. Os nove testes voltaram a `<Ala />`. Já as ações não têm padrão útil: `useAcoesExibicao()` lança erro fora do provedor, para o engano aparecer cedo.

exibicao/Provedor.tsx cap11

```
  // Os setters do useState são estáveis; o
  // objeto que os junta precisa ser também.
  const acoes = useMemo(
    () => ({ setDiscreto, setTema }), []);

  return (
    <AcoesContext value={acoes}>
      <TemaContext value={tema}>
        <DiscretoContext value={discreto}>
          {children}
        </DiscretoContext>
      </TemaContext>
    </AcoesContext>
  );
```

O provedor guarda os dois `useState` e recebe a tela inteira como `children`. O `App` perdeu o estado: virou só composição. Todo nome de paciente passa por

`useExibirNome()`, inclusive o da confirmação de alta, e o teste volta a passar.

11.3

O custo de cada leitura

A documentação é direta: quando o `value` de um provedor muda, segundo `Object.is`, o React renderiza de novo todos os componentes abaixo que leem aquele contexto, e `memo` não impede. O Renderômetro do capítulo 3 envolve cada cartão da Ala. Trocamos o tema e depois ligamos o modo discreto em três versões:

```
medicoes/cap11.py · Medido

== props a partir do App
  tema → Escuro          6 de 6 cartões renderizaram
  modo discreto ligado   6 de 6 cartões renderizaram
  primeiro cartão: M. S.

== contexto único
  tema → Escuro          6 de 6 cartões renderizaram
  modo discreto ligado   6 de 6 cartões renderizaram
  primeiro cartão: M. S.

== três contextos
  tema → Escuro          0 de 6 cartões renderizaram
  modo discreto ligado   6 de 6 cartões renderizaram
  primeiro cartão: M. S.
```

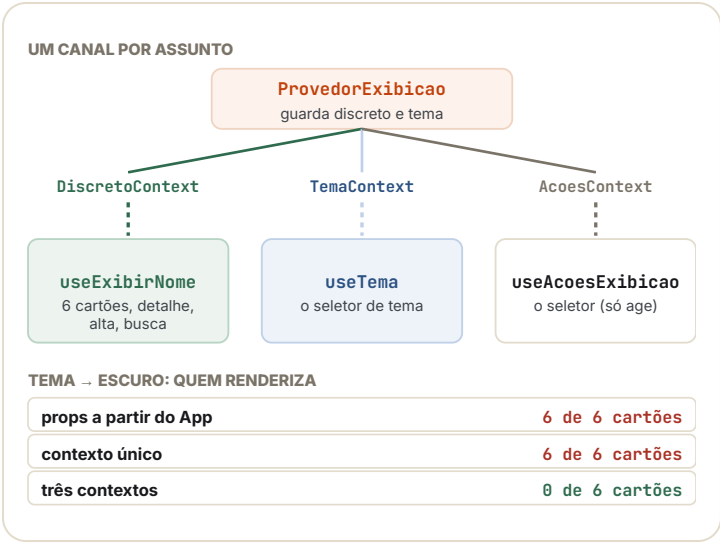


Fig. 11.1 — Três canais saindo do mesmo provedor. Os cartões escutam só o discreto; trocar o tema não os alcança.

Com props, o estado está no `App`: trocar o tema renderiza o `App` e tudo abaixo, seis cartões que nem olham o tema. Com um contexto só, cujo valor é um objeto `{ discreto, tema, setDiscreto, setTema }` recriado a cada render do provedor, trocar o tema muda a identidade do objeto e todos os leitores renderizam de novo. Com três contextos, a troca de tema só muda o `TemaContext`, que nenhum cartão lê: zero. Ligar o modo discreto renderiza os seis nas três versões, como deve, porque o nome mudou.

Dois detalhes fazem o zero. O provedor recebe `children`: os elementos da tela foram criados pelo `App`, que não renderizou de novo, e o React pula quem não lê o contexto que mudou. E o objeto de ações é

estável (`useMemo` sobre setters que já são estáveis), então quem só age nunca renderiza por causa dele.

11.4

O que vai para contexto, e o que não vai

ESTADO	ONDE MORA
Campo digitado, modal aberto	No componente (capítulo 6)
Preferência de exibição, usuário logado	Context, um por assunto
Pacientes, atendimentos	Cache do servidor (capítulo 14), nunca em contexto
Paciente aberto, busca, página	URL (capítulo 13)

Context não é "estado global" para tudo: dado do servidor num contexto envelhece sem aviso, e valor que muda a cada tecla renderiza todos os leitores a cada tecla. Estado que vive fora do React (store em módulo, `localStorage` entre abas) se assina com `useSyncExternalStore`. Zustand, Redux Toolkit e Jotai são stores externas prontas, com assinatura seletiva; valem a dependência quando muito estado compartilhado muda com frequência. Aqui, ainda não. O teste do modo discreto, que falhava em 11.1, passa no `cap11`.

REGRA · UI-15

Estado que muitas features leem mora num provedor em `compartilhado/` que recebe a tela como `children`: um contexto por assunto, ações num contexto estável. Nome de paciente passa por `useExibirNome()`. Dado do servidor nunca vai para contexto. Verificação: `App.test.tsx` e `medicoes/cap11.py`.

11.5

Com agentes de IA

ARMADILHA DE IA

Pedido "evite prop drilling", o agente cria um `AppContext` com tudo dentro, valor como objeto literal, e às vezes põe a lista de pacientes do servidor junto. Funciona, e cada tecla em qualquer campo renderiza a tela inteira. Quando alguém reclama da lentidão, ele espalha `memo`, que não resolve: leitor de contexto renderiza mesmo assim. Peça um contexto por assunto e um número medido.

PROMPT · AGENTE · CAP 11

```
Goal
Preferência "densidade compacta" para a
lista da Ala.

Context
@docs/UI.md (Estado compartilhado)
```

```
@src/compartilhado/exibicao/

Constraints
- contexto próprio; não mexa nos outros
- ações no AcoesContext, valor estável
- nada de memo como remendo

Done when
- npm run gate verde
- cap11.py: trocar o tema continua
  com 0 de 6 cartões renderizados
```

Exercícios

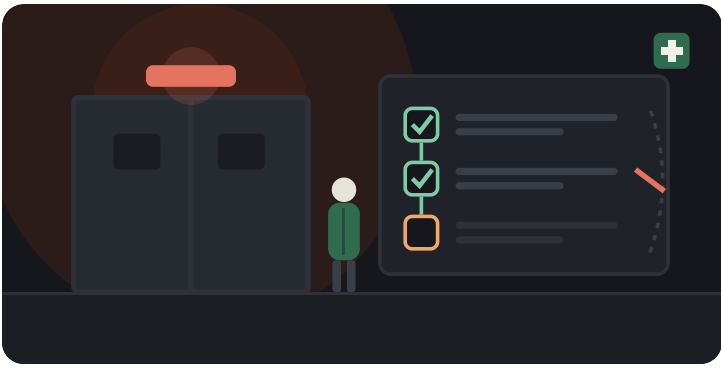
ESSENCIAL Tire o `useMemo` do objeto de ações e rode `cap11.py`. O número muda? Explique pelo que os cartões leem.

INTERMEDIÁRIO Guarde as preferências no `localStorage` e leia na primeira renderização, sem efeito que copie estado (UI-06).

DESAFIO Sincronize o modo discreto entre duas abas com `useSyncExternalStore` e o evento `storage`. Prove com duas páginas no Playwright.

Régua: 18. **Invariante:** UI-15. **Tags:** `cap11-antes`, `cap11`; variante `medicoes/cap11-contexto-unico.patch`. **Prompts:** `prompts/cap11.md`.

Fontes. react.dev: *createContext*, *useContext* (*Caveats*), *use*, *useSyncExternalStore*, *Passing Data Deeply with Context*, *React 19* (*<Context> as a provider*). Saídas em `saidas/c11-*.txt`.



PARTE III · ARQUITETURA

12

Reducer e transições de estado

Quando o próximo estado depende de regras, escreva as regras num lugar só, e torne impossível o estado que não pode existir.

A admissão ganhou três passos: identificação, contato, confirmação. Na recepção, com a rede lenta, a atendente clica Admitir e, enquanto espera, clica Voltar para conferir o telefone. A tela diz "Paciente admitido" e, ao mesmo tempo, pede para corrigir o CNS.

Nenhuma linha está errada sozinha: o defeito está entre elas, em estados que deveriam mudar juntos e mudam em lugares diferentes.

12.1

Seis estados que mudam juntos

Na tag `cap12-antes`, o assistente guarda o passo, os dados, os erros, se está enviando, o aviso e o id admitido, cada um no seu `useState`:

admissao/Assistente.tsx `cap12-antes`

```
  const [passo, setPasso] =
    useState<Passo>("identificacao");
  const [dados, setDados] =
    useState<Dados>(lerRascunho);
  const [erros, setErros] =
    useState<Erros>({});
  const [enviando, setEnviando] =
    useState(false);
  const [aviso, setAviso] =
    useState<string>();
  const [admitidoId, setAdmitidoId] =
    useState<string>();
```

Seis variáveis permitem dezenas de combinações, e só algumas fazem sentido. "Enviando no passo 2" não faz. "Admitido e editando" também não. O código não impede nenhuma delas. O agente que escreveu desabilitou o botão Admitir durante o envio e esqueceu o Voltar; "Nova admissão" zera passo e dados e esquece o aviso de admitido. Com a API real e o POST retido até o roteiro mandar:

```
medicoes/cap12.py · Medido

tela cap12-antes
  Voltar durante o envio: clicado
  a tela mostra: Passo 2 de 3 · Contato
  aviso: Paciente p101 admitido.
  prontuários de Rosa na API: 1
  segue o fluxo e admite de novo: Passo 1 de 3 ·
Identificação
  aviso: Paciente p101 admitido.
  aviso: Já existe paciente com este CNS.
```

Um prontuário na API e a tela no passo 2 com "admitido" à vista. Quem segue o fluxo recebe o 409 do capítulo 6 ao lado do aviso de sucesso: o servidor segurou o dado; a tela perdeu o fio.

H+ **A lista de verificação de cirurgia segura**

A lista da OMS divide a cirurgia em três momentos: antes da anestesia, antes da incisão e antes de o paciente sair da sala. Só se passa ao seguinte com o anterior conferido. Com a luz de "em cirurgia" acesa, ninguém volta à conferência da anestesia. Quem conduz a lista não decide caso a caso: segue a regra escrita para cada fase.

ONDE A ANALOGIA PARA

Na sala, a equipe pode julgar uma exceção. O reducer, não: é função pura, sem rede, sem relógio. Por isso é previsível e testável sem tela, e tudo o que acontece no mundo chega a ele de fora, como ação.

12.2

Estado por fase, regras num lugar só

A primeira mudança não é de API, é de forma. Em vez de seis variáveis soltas, o estado é uma **união discriminada**: cada fase guarda só o que faz sentido nela.

admissao/assistente.ts cap12

```ts
// Cada fase guarda só o que faz sentido
// nela: enviando não tem passo, admitido não
// tem dados. Estado impossível não compila.
export type Estado =
  | { fase: "editando"; passo: Passo;
      dados: Dados; erros: Erros;
      aviso?: string }
  | { fase: "enviando"; dados: Dados }
  | { fase: "admitido"; id: string };

export type Acao =
  | { tipo: "campo"; campo: keyof Dados;
      valor: string }
  | { tipo: "avancar" }
  | { tipo: "voltar" }
  | { tipo: "enviar" }
```

```
  | { tipo: "recomecar" }
  | Resposta;
```

"Enviando no passo 2" deixou de ser um bug possível: a fase `enviando` nem tem passo. As ações também são uma união, pelo campo `tipo`, e a `Resposta` do servidor entra nela sem conversão. O **reducer** recebe o estado atual e uma ação e devolve o próximo estado:

admissao/assistente.ts — cap12

```
export function assistente(
  e: Estado, a: Acao,
): Estado {
  switch (e.fase) {
    case "editando":
      return editando(e, a);
    case "enviando":
      if (a.tipo === "admitido") {
        return { fase: "admitido", id: a.id };
      }
      if (a.tipo === "recusado") {
        return { ...inicial(e.dados),
          erros: a.erros };
      }
      if (a.tipo === "falhou") {
        return { ...inicial(e.dados),
          passo: a.passo, aviso: a.msg };
      }
      return e;
    case "admitido":
      return a.tipo === "recomecar"
        ? inicial() : e;
  }
}
```

Aqui mora a diferença entre **evento** e **transição**. A tela despacha o evento "voltar" sempre que o botão é clicado; quem decide se isso muda alguma coisa é o reducer. Na fase `enviando`, voltar devolve o mesmo estado. O TypeScript estreita `e` em cada `case`: dentro de `"enviando"`, `e.passo` nem compila.

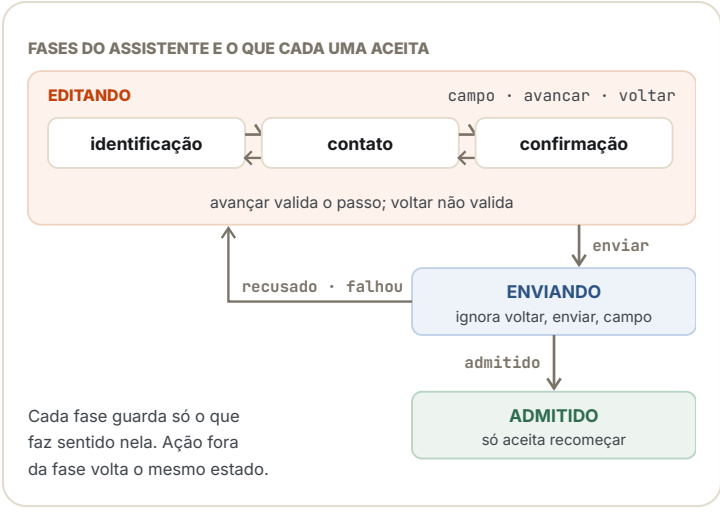


Fig. 12.1 — As três fases e o que cada uma aceita. A resposta do servidor só tem efeito na fase `enviando`.

12.3

A tela despacha e desenha

admissao/Assistente.tsx — cap12

```
export function AssistenteAdmissao() {
  const [estado, despachar] = useReducer(
    assistente, undefined,
    () => inicial(lerRascunho()));
  const emCurso = useRef(false);
```

```
  // Rascunho: sincroniza com sessionStorage.
  useEffect(() => {
    if (estado.fase === "editando") {
      salvarRascunho(estado.dados);
    } else if (estado.fase === "admitido") {
      apagarRascunho();
    }
  }, [estado]);
```

`useReducer(reducer, arg, init)` chama `init` só na montagem: o rascunho é lido uma vez, sem efeito que copie estado. O efeito que sobra sincroniza com um sistema externo, o `sessionStorage` (e não o `localStorage`: na recepção o computador é compartilhado, e o rascunho com CNS some quando a aba fecha).

admissao/Assistente.tsx cap12

```
  function enviar() {
    if (emCurso.current) return;
    emCurso.current = true;
    despachar({ tipo: "enviar" });
    startTransition(async () => {
      try {
        // a resposta já é uma ação
        despachar(await admitir(dados));
      } finally {
        emCurso.current = false;
      }
    });
  }
```

A trava do `useRef` (UI-04) continua, porque dois cliques no mesmo instante leem o mesmo estado, e a `startTransition` leva erro inesperado ao limite do painel (UI-10). `despachar(await admitir(dados))` é toda a ponte entre rede e tela. O Voltar desabilitado é derivado da fase, não um sétimo estado.

12.4

Testar sem tela

Um reducer é uma função. Seu teste não monta componente, não clica, não espera: aplica ações e compara.

admissao/assistente.test.ts cap12

```
// O reducer é uma função: testa-se sem tela.
const rosa = { ...VAZIO, nome: "Rosa Amaral",
  cns: "898001112223334",
  nascimento: "1971-05-20" };
const aplicar = (e: Estado, ...as: Acao[]) =>
  as.reduce(assistente, e);
const confirmando = aplicar(inicial(rosa),
  { tipo: "avancar" }, { tipo: "avancar" });
```

Os quatro testes do reducer rodam em milissegundos e cobrem as regras; os cinco testes da admissão do capítulo 6 continuam passando no assistente. A mesma medição, na tag `cap12`:

```
medicoes/cap12.py · Medido

tela cap12
  Voltar durante o envio: desabilitado
  a tela mostra: (fim do assistente)
  aviso: Paciente p101 admitido.
  prontuários de Rosa na API: 1
```

CAP12-ANTES

× voltar durante o envio não sai da confirmação
× nova admissão começa limpa

CAP12

✓ voltar durante o envio não sai da confirmação
✓ nova admissão começa limpa

12.5

Quando usar, e até onde vai

`useState` basta para um valor isolado: o termo da busca, o modal aberto. Reducer começa a valer quando vários valores mudam juntos ou quando a pergunta é "nesta fase, isto vale?". Combinado com o capítulo 11, o `despachar` devolvido pelo `useReducer` é estável e cabe num contexto de ações: componentes distantes despacham eventos sem renderizar quando o estado muda.

O nativo tem limites: dado do servidor é outro problema (capítulo 14); muito estado compartilhado mudando com frequência pede assinatura seletiva, que Context não tem; e uma linha do tempo de ações pede ferramentas como as do Redux Toolkit, que usa a mesma ideia deste capítulo: estado, ação, função pura.

REGRA · UI-16

Fluxo com fases e regras de transição mora num reducer puro, com o estado em união discriminada por fase. Ação fora da fase é ignorada pelo reducer, não só pelo botão. Verificação: `assistente.test.ts` (sem tela) e `Assistente.test.tsx`.

12.6

Com agentes de IA

ARMADILHA DE IA

Diante do Voltar no envio, o agente acrescenta um booleano e um `useEffect` que "corrige" o passo: estado sincronizado à mão, contra a UI-06. No reducer, às vezes chama `fetch` ou muta o estado recebido; o teste sem tela denuncia as duas coisas.

```
PROMPT · AGENTE · CAP 12

Goal
Na confirmação, "Editar contato" leva
direto ao passo 2.
```

```
Context
@src/features/admissao/assistente.ts
@INVARIANTES.md (UI-06, UI-16)

Constraints
- nova ação na união Acao; o reducer
  decide; nada de setState solto
- fora de "editando", a ação é ignorada
- reducer puro: sem fetch, sem mutação

Done when
- npm run gate verde
- teste do reducer: editar contato
  durante o envio devolve o mesmo estado
```

Exercícios

ESSENCIAL	Teste "campo no envio é ignorado". Por que já passa?
INTERMEDIÁRIO	Ponha o `despachar` num contexto de ações e faça o `Resumo` despachar "editar".
DESAFIO	Reescreva o reducer como tabela `fase → tipo → função`, com `satisfies` exigindo cada par.

Régua: 19. **Invariante:** UI-16. **Tags:** `cap12-antes`, `cap12` (tela) e `cap12` (API). **Prompts:** `prompts/cap12.md`.

Fontes. react.dev: *useReducer*, *Extracting State Logic into a Reducer*, *Scaling Up with Reducer and Context*. TypeScript: *Narrowing*. OMS: *Surgical Safety Checklist*. Saídas em `saidas/c12-*.txt`.



PARTE IV · A APLICAÇÃO COMPLETA

13

Rotas e navegação

A URL é estado. O que cabe num link deve morar nela, e o que não está nela some no primeiro recarregar.

O Mini-Prontuário ganhou menu: Ala B, Prontuário, Admissão, Entrar. A recepcionista liga o modo discreto, clica em Prontuário e o modo discreto desliga sozinho. Ela busca "ma", abre Maria, clica Voltar, e a busca sumiu.

Até aqui a aplicação era uma página só. Com várias telas, surgem perguntas novas: como trocar de tela sem recarregar, como guardar onde o usuário está, o que fazer com um endereço que não existe. As respostas passam todas pela URL.

13.1

Navegar sem recarregar

Num site comum, cada clique em `<a href>` baixa um documento novo: o JavaScript recomeça do zero, e todo estado do React morre. Numa SPA, o roteador intercepta o clique, troca a URL com a History API (`history.pushState`), escuta o Voltar (`popstate`) e desenha a tela da rota nova sem sair da página. Na tag `cap13-antes`, o agente montou as rotas, mas fez o menu com `<a href>` e guardou o termo da busca num `useState`:

```
medicoes/cap13.py · Medido

tela cap13-antes
  menu → Prontuário: 1 documento(s) baixado(s);
modo discreto desligado
  abre o 1º resultado: /pacientes/p1
  Voltar: /pacientes · campo '' · 0 resultado(s)
  /pacientes/p999: (sem layout) Unexpected
Application Error! HTTP 404 ErroHttp: HTTP 404 at
  /nada-aqui: (sem layout) Unexpected Application
Error! 404 Not Found 💿 Hey developer
```

Um documento novo a cada clique no menu, e com ele o provedor de exibição do capítulo 11 nasceu de novo. O termo da busca morava num componente que desmontou ao abrir o paciente. E endereço desconhecido mostra a tela de erro padrão do roteador, sem menu.

H+ O endereço do leito

"Ala B, leito 7" leva qualquer colega ao paciente, hoje ou amanhã, a pé ou por telefone. "O paciente que eu estava vendo" não leva ninguém. Uma URL boa é um endereço: diz onde e o quê, e funciona fora da sua cabeça.

ONDE A ANALOGIA PARA

O endereço do hospital não fica registrado em lugar nenhum. A URL fica: no histórico do navegador, nos logs do servidor e no cabeçalho `Referer`. Por isso nela vai o termo da busca e o id do paciente, nunca o CNS nem dado clínico.

13.2

React Router 8, modo de dados

O React Router tem três modos. No **declarativo**, as rotas são componentes `<Route>`. No de **dados**, são objetos passados a `createBrowserRouter`, e cada rota pode ter `loader`, `ErrorBoundary` e `lazy`. O **framework** acrescenta convenção de arquivos e renderização no servidor. Este livro usa o modo de dados. Na versão 8 o pacote é só `react-router`, e o `RouterProvider` vem de `react-router/dom`. O roteador é criado uma vez, fora da árvore do React, no `main.tsx`.



Fig. 13.1 — Cada parte da URL tem um leitor. Embaixo, a árvore de rotas da tag `cap13`: tudo dentro do `Layout` e de uma rota sem caminho que captura erros.

app/rotas.tsx cap13

```
export const rotas: RouteObject[] = [{
  path: "/",
  Component: Layout,
  children: [{
    // rota sem caminho: o erro de qualquer
    // página aparece dentro do Layout
    ErrorBoundary: ErroDeRota,
    children: [
      { index: true, Component: Ala },
      { path: "pacientes",
        handle: { titulo: "Prontuário" },
        Component: BuscaProntuario },
      { path: "pacientes/:id",
```

```
        handle: { titulo: "Paciente" },
        loader: carregarPaciente,
        Component: DetalhePaciente },
      { path: "admissao",
        handle: { titulo: "Admissão" },
        lazy: admissao },
      { path: "entrar",
        handle: { titulo: "Entrar" },
        Component: FormLogin },
      { path: "*",
        handle: { titulo: "Não encontrada" },
        Component: NaoEncontrada },
    ],
  }],
}];
```

O `Layout` desenha cabeçalho e menu e põe a rota filha no `<Outlet/>`, dentro de um `Painel` com o `handle.titulo` da rota. O menu agora é `NavLink`, que navega sem recarregar e marca a página atual com `aria-current="page"`.

13.3

A busca mora na URL

prontuario/Busca.tsx cap13

```
// O termo mora na URL: sobrevive ao Voltar,
// ao recarregar e a um link enviado.
const [params, setParams] =
  useSearchParams();
const termo = params.get("q") ?? "";
// replace: uma tecla não vira um passo no
// histórico. flushSync: o roteador atualiza
// dentro de uma transição, e o campo, que
// lê a URL, perderia teclas.
const setTermo = (t: string) =>
  setParams(t ? { q: t } : {},
    { replace: true, flushSync: true });
```

Três decisões em doze linhas. O termo é lido da URL, então sobrevive ao Voltar, ao recarregar e a um link enviado. O `replace` evita que cada tecla vire um passo no histórico. E o `flushSync` entrou depois de uma medição. No modo de dados, o roteador atualiza o estado dentro de uma transição do React, e um campo controlado pela URL pode ser redesenhado com o valor antigo entre duas teclas:

medicoes/cap13.py · Medido

```
digitar 'maria souza' na busca 20 vezes; quantas
perdem tecla
  sem flushSync,  0 ms entre teclas: 20 de 20
perderam tecla
  sem flushSync, 50 ms entre teclas: 0 de 20
perderam tecla
  com flushSync,  0 ms entre teclas: 0 de 20
perderam tecla
  com flushSync, 50 ms entre teclas: 0 de 20
perderam tecla
```

Sem o `flushSync`, digitação sem pausa entre teclas, como a de um leitor de código de barras ou de um preenchimento automático, perdeu tecla em todas as tentativas. No ritmo humano, não. O teste em jsdom passa nas duas versões, porque o `act` do Testing Library esvazia as transições antes da próxima tecla. Esse defeito só o navegador mostra.

13.4

Loader, erro e redirecionamento

O detalhe do paciente precisa do paciente para existir. Em vez de um efeito que busca depois de desenhar (capítulo 7), a rota tem um `loader`, que roda antes de a tela aparecer e recebe o `signal` da navegação:

prontuario/detalhe.ts cap13

```
export async function carregarPaciente(
  { params, request }: LoaderFunctionArgs,
) {
```

```
  try {
    return await pedir<PacienteApi>(
      `/api/patients/${params.id}`,
      { sinal: request.signal });
  } catch (e) {
    if (!(e instanceof ErroHttp)) throw e;
    if (e.status === 401) {
      // entra e volta para cá
      const aqui = new URL(request.url);
      throw redirect("/entrar?volta="
        + encodeURIComponent(aqui.pathname));
    }
    if (e.status === 404) {
      throw data("Paciente não encontrado.",
        { status: 404 });
    }
    throw e;
  }
}
```

O 401 vira `redirect` para o login, levando o caminho atual em `?volta=`. O 404 vira uma resposta que o `ErroDeRota` mostra dentro do layout. Depois de entrar, o login navega de volta com `useNavigate`, mas só para caminho interno. Sem esse filtro, `?volta=//site-falso.com` faria do login um trampolim para fora, um **redirecionamento aberto**:

acesso/FormLogin.tsx cap13

```
// Só caminhos internos: "?volta=//site.com"
// não pode levar ninguém para fora.
function destino(volta: string | null) {
  return volta?.startsWith("/")
    && !volta.startsWith("//") ? volta : "/";
}
```

```
medicoes/cap13.py · Medido

tela cap13
  menu → Prontuário: 0 documento(s) baixado(s);
modo discreto ligado
  abre o 1º resultado: /pacientes/p1
  Voltar: /pacientes?q=ma · campo 'ma' · 2
resultado(s)
  /pacientes/p999: PACIENTE Paciente não
encontrado. Voltar à busca
  /nada-aqui: NÃO ENCONTRADA Página não
encontrada. Ir para a Ala B
```

13.5

Sob demanda e em produção

A admissão traz o Zod, e só a recepção a abre. Com `lazy` e um `import()` de caminho literal, o Vite separa esse código num arquivo próprio, baixado na primeira visita à rota. Um detalhe pegou na montagem: com o caminho guardado numa variável, o Vite não divide nada.

```
medicoes/cap13.py · Medido

build de produção (JS, com gzip)
  cap13-antes: index 122.3 kB
  cap13: index 100.3 kB + Assistente 24.0 kB

quem responde GET /pacientes/p1 com o build
  python -m http.server: 404
  vite preview: 200
```

A última linha é a armadilha do primeiro deploy. Em produção, o build é um `index.html` e uns arquivos estáticos. Quem abre `/pacientes/p1` direto pede ao servidor um arquivo que não existe. O `vite preview` devolve o `index.html` para qualquer caminho desconhecido e o roteador resolve no navegador; um servidor estático simples devolve 404. No servidor de produção, isso se configura uma vez: no nginx, `try_files $uri /index.html;`.

CAP13-ANTES

- `× a busca mora na URL`
- `× paciente que não existe`
- `× rota desconhecida fica no layout`
- `× sem login, o detalhe leva ao login`

CAP13

`✓` os quatro, e o modo discreto do capítulo 11, agora aberto pela rota `/`

REGRAS · UI-17 E UI-18

UI-17: o que cabe num link mora na URL; campo ligado à URL usa `replace` e `flushSync`; navegação interna é `Link` / `NavLink`. **UI-18:** erro de rota aparece dentro do layout; caminho desconhecido cai no `*`; o loader traduz 401 em login com `?volta=` só interno e 404 em mensagem. Verificação: `rotas.test.tsx` e `medicoes/cap13.py`.

13.6

Com agentes de IA

ARMADILHA DE IA

Três padrões: `<a href>` no menu (funciona, recarrega tudo); filtro e página em `useState` "porque é mais simples"; e `window.location.href = "/entrar"` dentro de um efeito para redirecionar. Os três passam no portão comum. Peça explicitamente que o estado navegável viva na URL e que o redirecionamento saia do loader.

PROMPT · AGENTE · CAP 13

```
Goal
Filtro "só prioridade" na Ala B, que
sobrevive ao recarregar e ao Voltar.

Context
@docs/UI.md (Rotas) @src/app/rotas.tsx
@INVARIANTES.md (UI-17)
```

```
Constraints
- o filtro mora em ?prioridade=1
- Link/NavLink; nada de <a href> interno
- nada de useEffect lendo window.location

Done when
- npm run gate verde
- teste: abrir /?prioridade=1 já filtra
- Voltar desfaz o filtro
```

Exercícios

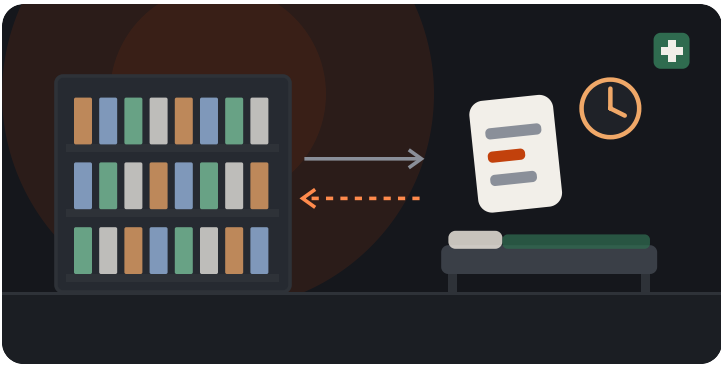
ESSENCIAL Troque o `replace: true` da busca por `false`, digite "maria" e conte quantos Voltar levam de volta à Ala B.

INTERMEDIÁRIO Mostre "Carregando..." durante o loader do detalhe com `useNavigation()`, só quando a espera passar de 300 ms.

DESAFIO Escreva um teste para `destino()` com cinco valores maliciosos de `?volta=`, incluindo `/\site.com`. Algum passa?

Régua: 20, 21. **Invariantes:** UI-17, UI-18. **Tags:** `cap13-antes`, `cap13`; variante `medicoes/cap13-sem-flushsync.patch`; API `cap12`. **Prompts:** `prompts/cap13.md`.

Fontes. React Router 8.5 (documentação no pacote): *Data Mode: Installation, Routing, Route Object*; *React Transitions*; CHANGELOG v8.0.0. MDN: *History API*, *Referer*. OWASP: *Unvalidated Redirects and Forwards Cheat Sheet*. nginx: *try_files*. Vite: *Static Deploy*. Saídas em `saidas/c13-*.txt`.



PARTE IV · A APLICAÇÃO COMPLETA

14

Backend, HTTP e estado remoto

Os dados do servidor não são seus. Você guarda uma cópia com prazo de validade, e precisa saber quando jogá-la fora.

Com João Lima aberto na tela, a recepcionista corrige o nome para "João Lima Neto" e salva. A API registra a mudança. Em Maria Souza.

Nenhum erro apareceu, nenhum teste antigo falhou. O defeito nasceu do jeito mais comum de lidar com dado do servidor: copiá-lo para um `useState` e esquecer que a cópia envelhece. Este capítulo estende a API, escreve o contrato e troca a cópia por um cache com regras.

14.1

A API cresce, o contrato muda

A busca precisava de páginas e a tela precisava editar e inativar. A tag `cap14` da API, feita com um agente a partir do prompt da seção 14.6, trouxe:

ROTA	O QUE FAZ
`GET /api/patients?q=&pagina=&porPagina=`	Responde `{ itens, total, pagina, porPagina }`, sem inativos
`PATCH /api/patients/:id`	Muda nome, telefone e responsável; CNS e nascimento, não (400)
`POST /api/patients/:id/inativar`	Tira da busca sem apagar o prontuário

A busca deixou de devolver um array e passou a devolver um envelope com o total. Isso é mudança de contrato, e a suíte herdada do *Crachá & Porta* provou: um agente de laboratório que lia a lista como array quebrou, e o teste dele falhou na hora. Do lado da tela, o contrato agora mora em dois lugares que mudam juntos: `docs/API.md`, para gente e agente lerem, e `src/api/pacientes.ts`, com os tipos e as funções que chamam `pedir()`. Nenhum componente monta URL de API.

H+ **A prancheta na beira do leito**

O prontuário oficial fica no arquivo central. Na beira do leito vai uma prancheta com a cópia do que importa agora, e a hora em que foi tirada. Quando alguém altera o oficial, a cópia da prancheta precisa ser trocada, senão o plantão seguinte age sobre informação velha.

ONDE A ANALOGIA PARA

A enfermagem sabe que a prancheta é cópia. O `useState` não sabe: guarda o valor como se fosse dele, para sempre. A biblioteca de estado remoto é o que dá à cópia um dono, uma etiqueta (a chave) e um prazo.

14.2

O erro clássico: cópia em `useState`

Na tag `cap14-antes`, busca e detalhe viraram mestre-detalhe: `/pacientes/:id` é rota filha de `/pacientes`, e a lista continua na tela com o paciente aberto embaixo. O detalhe copia o que o loader trouxe:

prontuario/Detalhe.tsx cap14-antes

```
…
export function DetalhePaciente() {
  const doServidor = useLoaderData<
    typeof carregarPaciente>();
  // cópia local, para mostrar o que foi salvo
  const [p, setP] = useState(doServidor);
  const [aviso, setAviso] =
    useState<string>();
  const [confirmar, setConfirmar] =
    useState(false);
  const exibir = useExibirNome();
…
```

`useState(inicial)` usa o valor só na montagem. Ao trocar de Maria para João, o roteador reaproveita o mesmo componente e o loader traz João, mas o estado continua sendo Maria. O formulário mostra Maria, e o `salvar` usa `p.id`:

medicoes/cap14.py · Medido

```
tela cap14-antes
  URL /pacientes/p2?q=a · CNS no detalhe: Maria
  salvou: p1 na API = 'João Lima Neto' · p2 na API
= 'João Lima'
  lista: Maria Souza, João Lima, Ana Clara Reis,
Pedro Alves, Luzia Freitas
  inativou o aberto · lista: Maria Souza, João
Lima, Ana Clara Reis, Pedro Alves, Luzia Freitas
  Ala B e Voltar: 2 busca(s) à rede
```

Três sintomas da mesma causa. A URL diz João, e a edição e a inativação caem em Maria. A lista, que tem a

sua própria cópia, não vê nada do que mudou. E cada volta à busca refaz o pedido, porque ninguém sabe que o dado ainda está bom. Dois testes novos descrevem o comportamento certo e falham:

```
terminal · Medido

 ❯ src/app/rotas.test.tsx (7 tests | 2 failed)
2419ms
   × a lista mostra o nome editado 1100ms
   × trocar de paciente troca o detalhe 1049ms
```

14.3

Um cache com dono: TanStack Query

Estado remoto tem problemas que estado local não tem: tem outro dono, envelhece, é pedido por vários componentes ao mesmo tempo, precisa ser buscado de novo depois de uma mudança. O TanStack Query (5.106) resolve isso com um cache único, o `QueryClient`, onde cada dado tem uma **chave**.

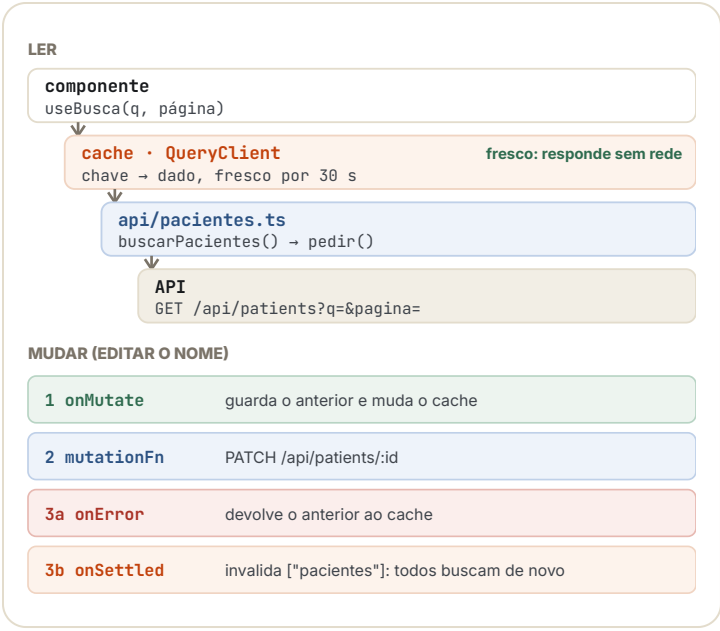


Fig. 14.1 — Em cima, a leitura: o componente pede pela chave, e o cache decide se vai à rede. Embaixo, a edição, em três momentos.

api/consultas.ts cap14

```
export const consultas = new QueryClient({
  defaultOptions: { queries: {
    // por 30 s o dado é fresco: quem pedir
    // a mesma chave usa o cache sem rede
    staleTime: 30_000,
    // repetir 401, 403, 404 não muda nada
    retry: (tentativas, e) => tentativas < 2
      && !(e instanceof ErroHttp
        && e.status < 500),
  } },
});
```

Por 30 segundos o dado é **fresco**: quem pedir a mesma chave recebe do cache, sem rede. Depois fica **velho**: ainda aparece, mas a próxima leitura busca de novo por baixo. Repetir pedido só vale para erro do servidor; um 404 repetido continua 404.

prontuario/consultas.ts cap14

```ts
// Chaves em árvore: invalidar ["pacientes"]
// alcança todas as buscas e os detalhes.
export const chaves = {
  todos: ["pacientes"] as const,
  busca: (q: string, p: number) =>
    ["pacientes", "busca", q, p] as const,
  um: (id: string) =>
    ["pacientes", "um", id] as const,
};

// A mesma receita serve ao loader da rota e
// ao componente: um pedido, um lugar no
// cache.
export const consultaPaciente =
  (id: string) => queryOptions({
    queryKey: chaves.um(id),
    queryFn: ({ signal }) =>
      obterPaciente(id, signal),
  });
```

As chaves formam uma árvore. `["pacientes", "busca", "jo", 1]` e `["pacientes", "um", "p2"]` têm a mesma raiz, e invalidar `["pacientes"]` alcança as duas. A `consultaPaciente` é uma receita usada em dois lugares: o loader da rota chama `consultas.ensureQueryData(consultaPaciente(id))`, que busca só se o cache não tiver, e o componente lê a

mesma chave com `useSuspenseQuery`. O loader continua traduzindo 401 e 404, como no capítulo 13.

A busca usa `useQuery` com `placeholderData: keepPreviousData`: ao trocar de página, a lista velha fica na tela até a nova chegar, sem piscar vazio. E `throwOnError` separa o erro esperado (401, 403: aviso) do inesperado, que sobe ao limite do painel (UI-10). O `useBuscaPacientes` do capítulo 7, com seu `AbortController`, saiu: o Query entrega o `signal` e cancela o pedido de uma chave que saiu de uso. O teste da corrida daquele capítulo continua passando.

14.4

Mudar: otimista, com volta

prontuario/consultas.ts cap14

```
return useMutation({
  mutationFn: (dados: Edicao) =>
    editarPaciente(id, dados),
  // Otimista: a tela muda antes da
  // resposta.
  onMutate: async (dados) => {
    await cliente.cancelQueries(
      { queryKey: chave });
    const antes =
      cliente.getQueryData(chave);
    cliente.setQueryData(chave,
      (p) => p && { ...p, ...dados });
    return { antes };
  },
  // Deu errado: volta ao que era.
  onError: (_e, _dados, ctx) =>
    cliente.setQueryData(chave, ctx?.antes),
```

```
    // Certo ou errado: o servidor tem a
    // palavra final, em todas as buscas.
    onSettled: () =>
      cliente.invalidateQueries(
        { queryKey: chaves.todos }),
  });
```

Antes de a requisição sair, `onMutate` cancela leituras em andamento daquela chave, guarda o valor anterior e escreve o novo no cache: a tela muda na hora. Se o servidor recusar, `onError` devolve o anterior. Dando certo ou errado, `onSettled` invalida `["pacientes"]`, e toda busca aberta pede de novo o que o servidor tem. No detalhe não há mais cópia: o componente lê do cache, e o formulário tem `key={p.id}`, para outro paciente ganhar outro formulário.

medicoes/cap14.py · Medido

```
tela cap14
  URL /pacientes/p2?q=a · CNS no detalhe: João
  salvou: p1 na API = 'Maria Souza' · p2 na API =
'João Lima Neto'
  lista: Maria Souza, João Lima Neto, Ana Clara
Reis, Pedro Alves, Luzia Freitas
  inativou o aberto · lista: Maria Souza, Ana
Clara Reis, Pedro Alves, Luzia Freitas, Raimundo
Nonato
  Ala B e Voltar: 0 busca(s) à rede

  edição otimista, servidor lento que responde 500
    título antes: 'Ana Clara Reis' · logo após
Salvar: 'Ana C. Reis'
    depois do 500: 'Ana Clara Reis' · aviso: 'Não
foi salvo; o nome voltou ao anterior.'
```

João é editado em João. A lista mostra o nome novo e perde o inativado. Voltar à busca não vai à rede, porque o dado ainda é fresco. E quando o servidor demora e responde 500, o nome muda na hora e volta sozinho, com o aviso.

ATENÇÃO · QUANDO NÃO SER OTIMISTA

Otimista serve para mudança que quase sempre dá certo e é fácil de desfazer: um nome, uma prioridade. Prescrição, alta e pagamento esperam a confirmação do servidor antes de aparecer como feitos. Ninguém deve agir sobre uma tela que mostra o que ainda não aconteceu.

CAP14-ANTES

`× a lista mostra o nome editado`
`× trocar de paciente troca o detalhe`

CAP14

`✓` os dois, e os 41 testes anteriores

REGRAS · UI-19 E UI-20

UI-19: dado do servidor mora no cache do Query, com chave em árvore; nunca em `useState` nem em contexto; mutação invalida `["pacientes"]`; otimista guarda o anterior e o devolve no erro. **UI-20:** o contrato mora em `docs/API.md` e `src/api/pacientes.ts`, que mudam juntos com os testes.

14.5

Loader, Query ou os dois?

O loader responde "o que esta rota precisa para existir?" e decide redirecionamento e 404 antes da tela. O Query responde "quanto tempo esta cópia vale, e quem mais a usa?". Juntos, o loader enche o cache e a tela lê dele. Sem o Query, as `action`s do roteador revalidam os loaders, o que basta a apps com pouco dado compartilhado. O que não se faz é a terceira via: `useEffect`, `fetch` e `useState` em cada tela.

14.6

Com agentes de IA

ARMADILHA DE IA

Mesmo com o Query instalado, o agente costuma copiar `data` para um `useState` "para editar" e sincronizar com um efeito (UI-06). Invalida a chave errada, e a lista não muda. E escreve `onMutate` otimista sem `onError`: no primeiro 500, a tela mente para sempre. O teste "a lista mostra o nome editado" pega os dois primeiros; o roteiro com 500 pega o terceiro.

PROMPT · AGENTE · CAP 14 (API)

```
Goal
Busca paginada com total, edição e
inativação de paciente na API.

Context
@src/pacientes/rotas.ts
@src/acesso/permissoes.ts @tests/

Constraints
- GET responde { itens, total, pagina,
  porPagina }; porPagina até 50
- PATCH só nome, telefone, responsável
- inativar não apaga; sai da busca
- permissão nova patient:update
- mudança de contrato: atualize todo
  consumidor da lista no repositório

Done when
```

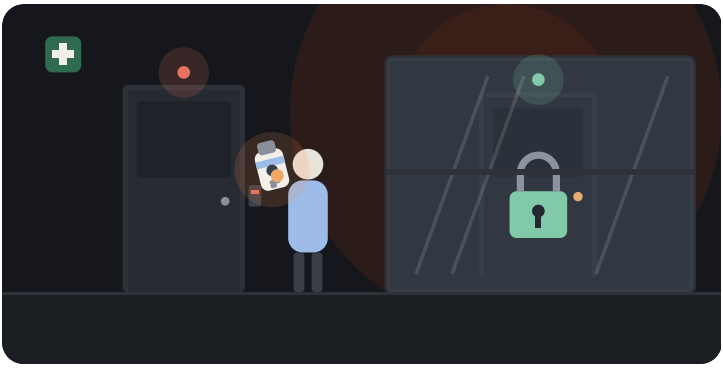
```
- npm test verde, com testes novos
  de página, edição e inativação
```

Exercícios

ESSENCIAL	Mude `staleTime` para 0 e rode `cap14.py`. Quantas buscas o Voltar faz agora?
INTERMEDIÁRIO	Torne a inativação otimista na lista, com volta no erro. Prove com um 500.
DESAFIO	Antecipe a próxima página com `prefetchQuery` e meça o tempo até ela aparecer.

Régua: 21, 22, 23. **Invariantes:** UI-19, UI-20. **Tags:** `cap14-antes`, `cap14` (tela e API). **Prompts:** `prompts/cap14.md`.

Fontes. TanStack Query v5: *Query Keys*, *Important Defaults*, *Optimistic Updates*, *Query Invalidation*, *Paginated Queries*, *queryOptions*, *ensureQueryData*. react.dev: *You Might Not Need an Effect*. React Router 8: *Data Loading*. Saídas em `saidas/c14-*.txt`.



PARTE IV · A APLICAÇÃO COMPLETA

15

Autenticação, sessão e autorização na interface

Quem entrou, quem diz é o servidor. A interface esconde o que você não pode fazer; só o servidor proíbe.

A recepção pediu login na tela. O agente entregou numa tarde: entrar, sair, rotas protegidas e o botão Inativar escondido de quem é da recepção. Passou no portão. Na manhã seguinte, Carla abriu as ferramentas do navegador, trocou uma palavra e inativou um paciente.

15.1

Autenticar e autorizar, vistos da tela

Autenticar é saber quem é; **autorizar** é decidir o que essa pessoa pode fazer. No *Crachá & Porta*, as duas coisas foram construídas no servidor: senha com argon2, sessão no banco, cookie `__Host-sessao` com `HttpOnly` e `SameSite=Lax`, checagem de `Origin` contra CSRF e uma permissão exigida em cada rota. A tela não reimplementa nada disso: o navegador manda o cookie sozinho a cada `fetch` para a mesma origem, e ela pergunta e obedece. A primeira decisão é onde a credencial fica:

ONDE	UM SCRIPT DA PÁGINA LÊ?
Cookie HttpOnly (este livro)	Não: o JavaScript nem enxerga o cookie. CSRF fica com `SameSite` e `Origin`, já na API
Token em memória	Enquanto a página está aberta; some ao recarregar. Com JWT, um backend da tela (BFF) guarda o token e entrega um cookie
Token no `localStorage`	Sim, qualquer script, até alguém apagar

15.2

O erro clássico: a tela que acredita no navegador

Na tag `cap15-antes`, o agente guardou o usuário do login no `localStorage`. Uma guarda em componente, `Protegida`, redireciona quem não tem nada guardado, e o detalhe esconde Inativar quando o papel guardado é `"recepcao"`:

compartilhado/sessao.ts — cap15-antes

```
const CHAVE = "usuario";

export function usuarioAtual():
  Usuario | null {
  const s = localStorage.getItem(CHAVE);
  return s ? JSON.parse(s) : null;
}
```

A tela passa a ter uma opinião sobre quem entrou, e o servidor, outra. Elas divergem quando a sessão acaba no servidor: expirou, o papel mudou, ou a pessoa clicou "sair de todos os dispositivos" em outro computador. O roteiro faz Ana entrar e, de outro navegador, encerra todas as sessões dela:

medicoes/cap15.py · Medido

```
tela cap15-antes · API cap14
  1. sessão encerrada no servidor, recarregar
     endereço: /pacientes?q=ma
     cabeçalho: Dra. Ana · botão Sair
     avisos: Entre para buscar no prontuário.
```

```
  2. sessão cai, Salvar
     endereço: /pacientes/p2?q=jo
     cabeçalho: Dra. Ana · botão Sair
     avisos: Não foi salvo; o nome voltou ao
anterior.; Entre para buscar no prontuário.
  4. Ana logada; um script da página lê
     document.cookie: ''
     localStorage['usuario']:
       {"id":"u2","nome":"Dra.
Ana","email":"ana@hospital.dev","papel":"profissio
nal"}
     Carla troca o papel no localStorage e
recarrega:
     botão Inativar: 0 → 1
     na API: Pedro Alves · ativo: False
```

O cabeçalho diz que Ana está logada; a busca pede que ela entre; o Salvar desfaz a edição sem dizer por quê. E o `localStorage` é de qualquer script, inclusive do console: o cookie não aparece em `document.cookie`, o usuário guardado aparece inteiro. Carla mudou o próprio papel ali, o botão apareceu e a API `cap14` aceitou. São dois defeitos: a tela acreditou num dado que o usuário controla, e a API não proibia o que a tela escondia.

H+ **O adesivo e o crachá**

Na portaria, o visitante recebe um adesivo onde ele mesmo escreve o nome. Ajuda a enfermagem a chamá-lo, e só. A porta da UTI lê o crachá que a recepção emitiu. Quem escreve "médico" no adesivo continua do lado de fora.

ONDE A ANALOGIA PARA

O segurança pode olhar o crachá. A tela não: o cookie é `HttpOnly`. Ela não confere nada; pergunta ao servidor quem entrou e acredita só na resposta.

15.3

Quem entrou, segundo o servidor

Na tag `cap15` da API, `/api/auth/me` passou a devolver também as permissões do papel. A tela pergunta ali, e a resposta é dado do servidor como qualquer outro: mora no cache do capítulo 14, numa chave própria.

api/sessao.ts cap15

```
// A sessão é dado do servidor: mora no
// cache, como os pacientes. Muda no login,
// no sair e no primeiro 401.
export const consultaSessao = queryOptions({
  queryKey: CHAVE_SESSAO,
  queryFn: ({ signal }) =>
    obterSessao(signal),
  staleTime: Infinity,
});
```

`obterSessao` transforma o 401 de `/api/auth/me` em `null`: ali, 401 é "ninguém entrou". Com `staleTime: Infinity`, o cache não pergunta de novo sozinho; login, sair e 401 mexem nele de propósito. Os componentes leem com `useSessao()` e `usePode("patient:deactivate")`, dois hooks de cinco linhas sobre essa consulta. A tabela do capítulo 11 pôs

"usuário logado" em Context; agora dá para ser preciso: é dado do servidor, e o cache já é o provedor. O `localStorage` sumiu do projeto.

15.4

Middleware: decidir antes do loader

A guarda em componente tem outro defeito. No modo de dados, o loader roda antes de a tela desenhar, e quem abre `/pacientes/p1` sem sessão dispara o pedido do paciente antes de ser redirecionado:

```
medicoes/cap15.py · Medido

  3. sem sessão, abrir /pacientes/p1
     endereço: /entrar?volta=%2Fpacientes%2Fp1
     pedidos à API: GET /api/patients/p1
…
  3. sem sessão, abrir /pacientes/p1
     endereço: /entrar?volta=%2Fpacientes%2Fp1
     pedidos à API: GET /api/auth/me
```

O React Router 8 tem **middleware** no modo de dados: uma função da rota que roda antes dos loaders das filhas, a cada navegação. As rotas protegidas ficam sob uma rota sem caminho com `middleware: [exigirSessao]` e `handle: { protegida: true }`:

```
app/protecao.ts                                  cap15

// Roda antes dos loaders das rotas filhas:
// sem sessão, ninguém pede o paciente.
export const exigirSessao:
  MiddlewareFunction = async (
```

```
    { request },
  ) => {
    const u = await consultas
      .ensureQueryData(consultaSessao);
    if (!u) {
      throw redirect(
        paraLogin(new URL(request.url)));
    }
  };
```

Na primeira navegação, `ensureQueryData` pergunta ao servidor; nas seguintes, responde do cache. Recarregar zera o cache e a pergunta volta ao servidor: a sessão sobrevive ao F5 sem nada guardado no navegador.



Fig. 15.1 — O middleware decide antes do loader. Com a tela aberta, 401 e 403 têm destinos diferentes.

15.5

401 e 403 não são o mesmo erro

O middleware só roda na navegação. Se a sessão acaba com a tela parada, a próxima consulta ou mutação recebe 401. O cache tem um ponto só para isso:

api/consultas.ts cap15

```ts
// Ninguém entrou: quem lê a sessão (o
// Layout, o middleware) fica sabendo.
const semSessao = () =>
  consultas.setQueryData(CHAVE_SESSAO, null);

// Sair: a tela esquece tudo o que viu. Só
// depois de deixar as rotas protegidas, ou
// as telas abertas pediriam tudo de novo.
export function esquecerSessao() {
  consultas.removeQueries({ predicate: (q) =>
    q.queryKey[0] !== CHAVE_SESSAO[0] });
  semSessao();
}

// 401 numa consulta ou mutação: o servidor
// encerrou a sessão com a tela aberta.
function se401(e: Error | null) {
  if (e instanceof ErroHttp
    && e.status === 401) semSessao();
}
```

`se401` fica no `onError` do `QueryCache` e no `onSettled` do `MutationCache`, que roda depois de a edição otimista devolver o nome anterior. Gravar `null` avisa o `Layout`, que conhece a rota atual: se ela tem `handle.protegida`, ele devolve `<Navigate to={paraLogin(local)} />`.

```
medicoes/cap15.py · Medido

tela cap15 · API cap15
  1. sessão encerrada no servidor, recarregar
     endereço: /entrar?volta=%2Fpacientes%3Fq%3Dma
     cabeçalho: ninguém · link Entrar
     avisos: —
  2. sessão cai, Salvar
     endereço: /entrar?
volta=%2Fpacientes%2Fp2%3Fq%3Djo
     cabeçalho: ninguém · link Entrar
     avisos: —
     entra de novo: /pacientes/p2?q=jo · CNS de
João na tela: sim
```

Ana perde a edição não salva, mas não o lugar: entra de novo e volta para João, com a busca na URL. O 403 é outra história. A pessoa está logada e não tem permissão; no login, entraria de novo para receber o mesmo 403. Por isso 403 vira aviso no lugar, como já acontecia na busca (UI-10), e um teste prova que a URL não muda.

ATENÇÃO · O LAÇO QUE O TESTE PEGOU

Na primeira versão, o 401 também apagava os pacientes do cache. O teste travou: o detalhe, ainda montado, pedia o paciente de novo, recebia 401, o cache apagava, e assim até o processo cair. Só se limpa o cache depois de sair das rotas que o leem. Por isso `esquecerSessao()` roda no Sair depois de navegar para `/entrar`, e o login começa com `clear()`.

15.6

Esconder não é proibir

Inativar agora some por permissão, não por papel: `usePode("patient:deactivate")`. Mudou a matriz no servidor, a tela acompanha sem deploy. Mas quem proíbe é a rota. O instrumento `porta-dos-fundos` testa a tela por fora: entra como profissional e como recepção, anota os botões que só o profissional vê, intercepta o pedido de cada um e o repete com a sessão da recepção:

```
medicoes/porta_dos_fundos.py · Medido

porta-dos-fundos · tela cap15-antes · API cap14
/pacientes/p3: profissional vê ['Salvar',
'Inativar']; recepcao vê ['Salvar']
  escondidos de recepcao: ['Inativar']
  Inativar → POST /api/patients/p3/inativar
    recepcao, chamando direto: 200  ✗ esperado 403
1 porta(s) dos fundos
…
  Inativar → POST /api/patients/p3/inativar
    recepcao, chamando direto: 403  ✓
nenhuma porta dos fundos
```

A correção não estava na tela. A rota de inativar exigia `patient:update`, que a recepção tem para editar telefone. A tag `cap15` da API criou `patient:deactivate` para admin e profissional, atualizou `docs/PERMISSOES.md` e a suíte papel × rota, e ganhou o teste "recepção chamando direto recebe 403". Quem escondeu é quem confere.

CAP15-ANTES

× sem sessão, não pede o paciente
× recepção não vê Inativar
× sessão que cai: login e volta
× 403 avisa e não leva ao login
× sair esquece o que a tela viu

CAP15

✓ os cinco, e os seis anteriores

15.7

Segredos e HTML no navegador

Tudo o que o Vite entrega é público. Variável com prefixo `VITE_` é escrita no bundle no build, e a documentação do Vite avisa que ela não deve conter informação sensível. Chave de serviço fica no servidor, que faz o pedido pela tela. E o React escapa texto: um paciente chamado `<img onerror=…>` aparece como texto. A brecha é `dangerouslySetInnerHTML`, que o lint agora recusa (`react/no-danger`); uma política de conteúdo (CSP) no servidor é a segunda camada.

REGRAS · UI-21 A UI-23

UI-21: quem entrou mora no cache, lido de `/api/auth/me`; nada no `localStorage`; rota protegida sob `exigirSessao`; 401 leva ao login com volta; entrar e sair limpam o cache. **UI-22:** botão escondido por permissão, nunca por papel; toda ação escondida tem 403 na API; 403 avisa no lugar. **UI-23:** nada de segredo em `VITE_` nem de `dangerouslySetInnerHTML`.

15.8

Com agentes de IA

ARMADILHA DE IA

Pedido de login, o agente costuma entregar usuário ou token no `localStorage`, permissão conferida pelo nome do papel e um `VITE_API_KEY` "para desenvolvimento". Tudo passa no portão. O teste da sessão que cai pega o primeiro; o `porta-dos-fundos`, o segundo; o `grep` da UI-23, o terceiro.

PROMPT · AGENTE · CAP 15

```
Goal
Sessão vinda do servidor; 401 leva ao
login e volta; Inativar só para quem
pode.

Context
@docs/API.md @docs/UI.md
```

```
@INVARIANTES.md (UI-19, UI-21, UI-22)

Constraints
- quem entrou mora no cache ["sessao"]
- nada em localStorage
- middleware nas rotas protegidas
- botão por permissão, nunca por papel
- só 401 leva ao login; 403 avisa

Done when
- npm run gate verde
- rotas.test.tsx: os cinco de sessão
- porta_dos_fundos.py: tudo 403
```

Exercícios

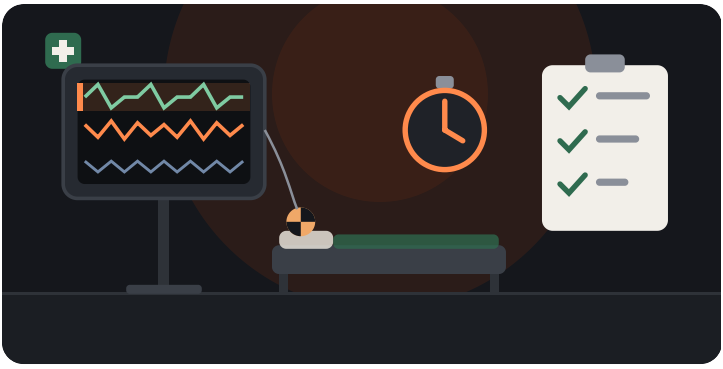
ESSENCIAL Logado, rode `document.cookie` no console. Explique o resultado com um atributo do `Set-Cookie`.

INTERMEDIÁRIO Esconda "Admissão" de quem não tem `patient:create` e proteja a rota com um segundo middleware.

DESAFIO Faça o `porta-dos-fundos` varrer também `/admissao`, com um papel novo na API.

Régua: 24 a 27. **Invariantes:** UI-21 a UI-23. **Tags:** `cap15-antes`, `cap15` (tela e API). **Prompts:** `prompts/cap15.md`.

Fontes. React Router 8.5: *Middleware*. TanStack Query v5: *QueryCache*, *MutationCache*. OWASP: *Session Management*, *HTML5 Security*. MDN: *Set-Cookie*. IETF: *OAuth 2.0 for Browser-Based Apps*. Vite: *Env Variables*. Saídas em `saidas/c15-*.txt`.



PARTE IV · A APLICAÇÃO COMPLETA

16

Testes, depuração e desempenho

Teste o que o usuário vê, simulando só o que está fora da aplicação. E meça antes de otimizar.

A recepção avisa: a página 2 da busca mostra os mesmos pacientes da página 1. A suíte tem 48 testes verdes, e um deles se chama "Próxima pede a página 2".

16.1

O teste que mente

Na tag `cap16-antes`, um agente recebeu dois pedidos: testar a paginação "rápido e isolado" e "padronizar os parâmetros em inglês, como as rotas". Ele trocou

`pagina` e `porPagina` por `page` e `perPage` em `api/pacientes.ts` e escreveu o teste isolando o componente:

prontuario/Paginacao.test.tsx cap16-antes

```
// Isola o componente: sem rede, sem cache,
// sem atraso. O hook devolve o que o teste
// mandar.
vi.mock("./consultas.ts", () => ({
  AVISO: {},
  useBusca: vi.fn(),
}));
```

O hook falso devolve cinco pacientes e um total de sete. O teste clica em Próxima e confere `expect(useBusca).toHaveBeenLastCalledWith("a", 2)`:

terminal · Medido

```
 ✓ src/features/prontuario/Paginacao.test.tsx >
Próxima pede a página 2 122ms
```

Verde, e verdadeiro: o componente pede mesmo a página 2. O defeito está um andar abaixo. A API lê `pagina` e ignora o que não conhece; com `page=2`, devolve a primeira página, com dez itens. O `vi.mock` trocou justamente o pedaço que quebrou. O teste prova que o componente chama o hook como o teste espera, não que a pessoa vê a página 2.

16.2

Onde cada teste corta

Todo teste troca alguma parte do sistema por uma versão controlada. Se o corte fica dentro da aplicação, a parte trocada deixa de ser testada. Se fica fora, na rede, todo o código da tela roda como em produção.

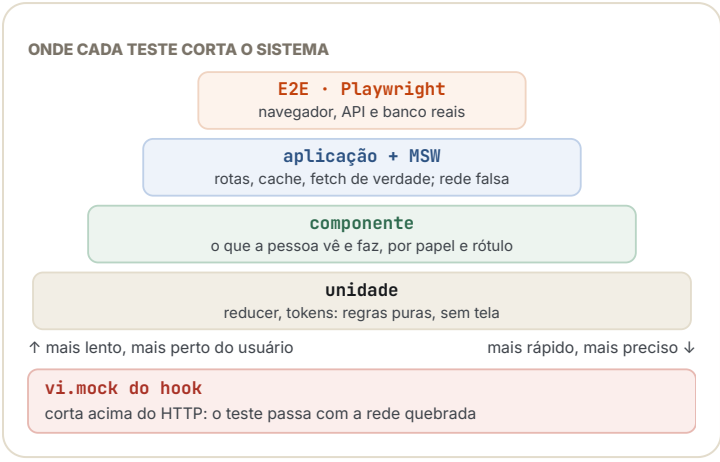


Fig. 16.1 — Quatro cortes. Quanto mais alto, mais do sistema real entra, e mais lento fica. O mock do hook corta dentro da aplicação.

```
medicoes/cap16.py · Medido

tag cap16 · Vitest 5 · jsdom
  unidade, sem tela        18 testes   0.01 s
  componente, rede à mão   18 testes   1.87 s
  aplicação + MSW          12 testes   0.92 s
  1 passed (2.9s)  ← E2E, 1 jornada
```

Os testes de unidade (reducer, tokens) cobrem regra pura em centésimos de segundo. Os de componente

leem a tela como a pessoa: `getByRole`, `getByLabelText`, `userEvent`; nunca classe CSS ou estado interno. Os de aplicação montam as rotas de verdade com a rede simulada. A jornada E2E é uma só, a mais lenta, e a única que passa pelo navegador, pela sessão e pelo banco. O Vitest também tem um modo navegador, mais fiel e mais lento que o jsdom.

H+ **O manequim de simulação**

No laboratório de simulação, a equipe treina parada cardíaca num manequim que responde: a pressão cai se a compressão é fraca. Ninguém treinaria num boneco que diz "ótimo" a tudo. O mock do hook é esse boneco: responde o que o teste mandou.

ONDE A ANALOGIA PARA

O manequim é fabricado a partir de fisiologia conhecida. A rede falsa é escrita por você, e pode divergir da API real sem aviso. Por isso ela imita o contrato de `docs/API.md` e existe ao menos uma jornada contra a API de verdade.

16.3

A rede simulada na fronteira: MSW

A `apiFalsa` dos capítulos 14 e 15 já simulava na fronteira, trocando o `fetch` global. Na tag `cap16`, ela passa a ser escrita com o MSW 3, que intercepta o

pedido depois que ele sai da aplicação e descreve cada rota como a API a define. A interface não mudou: nenhum teste antigo foi tocado.

teste/api.ts cap16

```
“
    // como a API: lê pagina e porPagina,
    // ignora o que não conhece
    http.get("/api/patients",
      ({ request }) => {
        const p = new URL(request.url)
          .searchParams;
        const q = p.get("q") ?? "";
        const pag =
          Number(p.get("pagina") ?? 1);
        const por =
          Number(p.get("porPagina") ?? 10);
”
```

O handler lê `pagina` e `porPagina` e ignora o resto, como a API. O servidor do MSW sobe com `onUnhandledFrame: "error"`: pedido que nenhum teste descreveu falha, em vez de passar em silêncio. O teste novo não conhece hook nenhum:

app/rotas.test.tsx cap16

```
test("Próxima mostra a página 2",
async () => {
  const sete = Array.from({ length: 7 },
    (_, i) => ({ id: `x${i + 1}`,
      nome: `Paciente ${i + 1}`,
      cns: String(i), nascimento: "" }));
  apiFalsa(sete);
  abrir("/pacientes?q=paciente");
```

```
  const links = async () => within(
    await lista()).findAllByRole("link");
  expect(await links()).toHaveLength(5);
  await userEvent.click(await screen
    .findByRole("button",
      { name: "Próxima" }));
  expect(await screen.findByRole("link",
    { name: "Paciente 6" })).toBeTruthy();
  expect(await links()).toHaveLength(2);
});
```

```
terminal · cap16-antes com o teste novo · Medido

 ❯ src/app/rotas.test.tsx (12 tests | 1 failed)
827ms
   × Próxima mostra a página 2 30ms
AssertionError: expected [ <a …(2)></a>, <a …(2)>
</a>, …(5) ] to have a length of 5 but got 7
```

Sete links na primeira página: a API falsa, como a real, não entendeu `perPage`. Com os nomes de volta, passa. O `Paginacao.test.tsx` saiu do projeto.

16.4

A jornada E2E, com a API real

Simular a rede deixa de fora o servidor, a sessão em cookie e o navegador. Uma jornada no Playwright 1.64, contra a API `cap15` em `:3000`, cobre isso: entrar, buscar, ir à página 2, editar o telefone, recarregar para provar que o valor veio do servidor, e sair.

e2e/jornada.spec.ts cap16

```
…
    await page.goto("/pacientes?q=a");
    await page.getByLabel("E-mail")
      .fill("carla@hospital.dev");
    await page.getByLabel("Senha")
      .fill("rede-na-varanda-de-tarde");
    await page.getByRole("button",
      { name: "Entrar" }).click();

    const achados = page.getByRole("list",
      { name: "Resultados" })
      .getByRole("link");
    await expect(achados).toHaveCount(5);
    await page.getByRole("button",
      { name: "Próxima" }).click();
    await expect(achados).toHaveCount(1);
…
```

medicoes/cap16.py e2e · Medido

```
== tela cap16-antes · API cap15 ·
e2e/jornada.spec.ts
    Error: expect(locator).toHaveCount(expected)
failed
    Locator:  getByRole('list', { name:
'Resultados' }).getByRole('link')
    Expected: 5
    Received: 6
…
== tela cap16 · API cap15 · e2e/jornada.spec.ts
Running 1 test using 1 worker
[1/1] e2e/jornada.spec.ts:7:1 › recepção busca,
pagina, edita e sai
  1 passed (2.9s)
```

CAP16-ANTES

`✓ Próxima pede a página 2` (mock)
`× Próxima mostra a página 2` (MSW)
`× jornada E2E` (API real)

CAP16

`✓` os 48 do Vitest e a jornada

16.5

Depurar sem adivinhar

O React DevTools mostra árvore, props e estado, e seu Profiler grava quem renderizou e por quê. No painel Performance do Chrome, as *React Performance Tracks* (React 19.2) põem o trabalho do React na linha do tempo da rede e da pintura. O Renderômetro do capítulo 3 mede em roteiro. A ordem não muda: reproduzir, medir, mudar uma coisa, medir de novo.

16.6

Desempenho: o compilador, medido

O React Compiler 1.0 memoiza componentes e hooks no build: em vez de escrever `useMemo` e `useCallback` à mão, você deixa o compilador decidir. No Vite 8 ele entra pelo Babel:

vite.config.ts cap16

```
  plugins: [
    react(),
    // React Compiler 1.0: memoiza componentes
    // e hooks no build (capítulo 16)
    babel(
      { presets: [reactCompilerPreset()] }),
  ],
```

A documentação do React recomenda o compilador em código novo. Ainda assim, ligar sem medir é otimizar por fé. O roteiro põe o Renderômetro também no Resumo e no Detalhe da Ala B e repete três interações:

medicoes/cap16.py compilador · Medido

```
== cap16-antes
  ação                 cartões · painéis
  digitar "a"          6/6 · Resumo, Detalhe
  prioridade de Maria  6/6 · Resumo, Detalhe
  selecionar João      6/6 · Resumo, Detalhe
  renders no total: 24
== cap16 · React Compiler
  ação                 cartões · painéis
  digitar "a"          6/6 · nenhum
  prioridade de Maria  6/6 · Resumo
  selecionar João      6/6 · Detalhe
  renders no total: 20
```

Com o compilador, cada painel só renderiza quando muda o que ele lê. Os seis cartões continuam renderizando: nascem de um `map` sobre a lista filtrada, e cada render da lista cria elementos novos. O compilador memoiza valores e JSX dentro de um

componente; não pula um componente que recebeu elemento novo. O custo:

```
medicoes/raio_x.py · Medido

== cap16-antes · build em 0.4 s (mediana de 3)
  Assistente.js       83.6 kB    24.4 kB gzip
  index.js           368.0 kB   114.3 kB gzip
  total              451.6 kB   138.7 kB gzip
== cap16 · build em 1.2 s (mediana de 3)
  Assistente.js       84.4 kB    24.7 kB gzip
  index.js           376.8 kB   117.9 kB gzip
  total              461.2 kB   142.6 kB gzip
```

São 3,9 kB a mais compactados (cerca de 3%) e o build três vezes mais lento, por causa do Babel; o plugin do Vite tem um compilador nativo em Rust, ainda experimental. O compilador ficou: tira `useMemo` da cabeça de quem escreve, inclusive do agente. Mas a decisão saiu de três números, não de um anúncio.

REGRAS · UI-24 E UI-25

UI-24: teste simula a rede no MSW e lê a tela por papel e rótulo; nunca `vi.mock` de módulo da aplicação; pedido sem handler é erro; a jornada E2E roda contra a API real antes de toda entrega. **UI-25:** memoização é do compilador; `useMemo`, `useCallback` e `memo` só com número do Renderômetro antes e depois.

PONTE · DEV DE IA

`npm run gate` roda a cada mudança; `npm run gate:completo` acrescenta a jornada E2E, antes do merge. Níveis de portão e checklist de merge: *Dev de IA*, capítulos 22 e 23.

16.7

Com agentes de IA

ARMADILHA DE IA

Diante de um teste vermelho, o agente tende a consertar o teste: `vi.mock`, `expect` mais frouxo, `setTimeout`. E espalha `useMemo` "por precaução". O `grep` da UI-24 pega o primeiro; a revisão pede o número do segundo.

PROMPT · AGENTE · CAP 16

```
Goal
A suíte pega HTTP quebrado: rede no MSW,
paginação testada pela tela, jornada
E2E contra a API real.

Context
@src/teste/ @src/app/rotas.test.tsx
@docs/API.md @INVARIANTES.md (UI-24)

Constraints
- nada de vi.mock de módulo do app
- apiFalsa com a mesma interface
- pedido sem handler é erro
- consultas por papel e rótulo
```

```
Done when
- npm run gate:completo verde
- o teste novo FALHA no cap16-antes
```

Exercícios

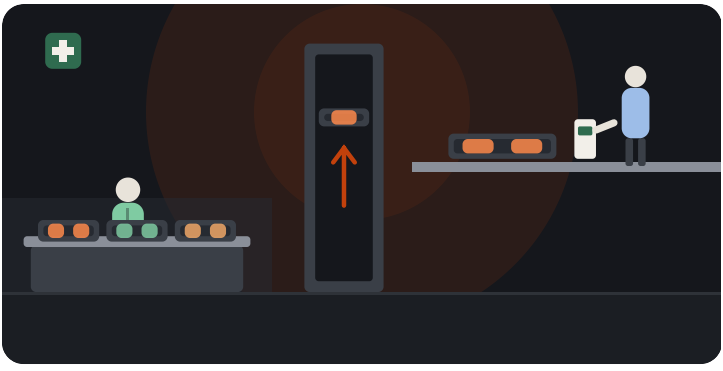
ESSENCIAL No `cap16-antes`, troque o `vi.mock` pela `apiFalsa` no `Paginacao.test.tsx`. O que precisou mudar no `expect`?

INTERMEDIÁRIO Escreva uma jornada E2E para a sessão que cai: encerre a sessão por `/api/auth/logout-all` e confira a volta ao paciente.

DESAFIO Ponha `memo` no `CartaoPaciente` e rode `cap16.py compilador`, com e sem o compilador. Explique os quatro números.

Régua: 16, 28 a 31. **Invariantes:** UI-24, UI-25. **Tags:** `cap16-antes`, `cap16`; API `cap15`. **Prompts:** `prompts/cap16.md`. **Instrumentos:** `cap16.py`, `raio_x.py`.

Fontes. MSW 3: *setupServer*. Testing Library: *Guiding Principles*. Vitest 5: *Browser Mode*. Playwright 1.64: *webServer*. react.dev: *React Compiler*, *Performance Tracks*. @vitejs/plugin-react 6 (README). Saídas em `saidas/c16-*.txt`.



PARTE V · RENDERIZAÇÃO MODERNA

17

Servidor, cliente e a fronteira entre eles

Onde cada pedaço da tela é montado, e o que atravessa a fronteira. Nada atravessa de graça.

Até aqui, o servidor mandava um HTML quase vazio, e a tela nascia no navegador. Numa 4G lenta, a recepcionista espera um segundo e meio pelo CNS de Maria. Montada no servidor, a mesma ficha aparece em menos de meio segundo, e cobra o próprio preço.

17.1

Quatro maneiras de montar a mesma tela

CSR (no cliente) é o que este livro fez: o navegador baixa o JavaScript, pede os dados e desenha. **SSR** (no servidor) monta o HTML com os dados a cada pedido; o navegador mostra na hora e depois **hidrata**: o React percorre o HTML pronto, liga os eventos e assume a tela. **SSG** monta o HTML no build, uma vez para todos; a geração incremental o refaz de tempos em tempos. O **streaming** manda a casca primeiro e, no mesmo HTML, as partes que dependem de dados, quando ficam prontas.

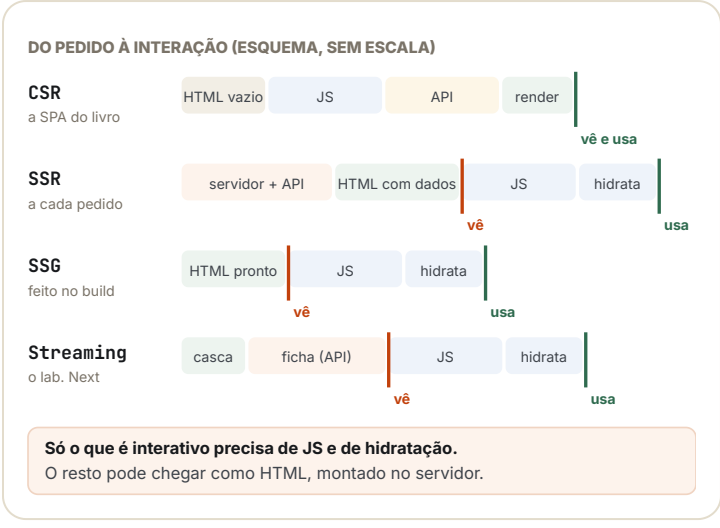


Fig. 17.1 — Quando a pessoa vê e quando pode usar. No CSR, as duas coisas chegam juntas e tarde.

17.2

O que chega ao navegador

O laboratório é um repositório novo, `estado-e-tela-next`: a ficha do paciente em Next.js 16.4, com a mesma API. O roteiro abre `/pacientes/p1`, com Ana logada, na SPA do capítulo 16 (build de produção) e no Next, numa 4G lenta emulada:

```
medicoes/cap17.py chegada · Medido

Chromium 141 · 4G lenta emulada: 150 ms, 1,6 Mbps
SPA · web cap16 · vite preview
  CNS no HTML recebido: não
  JavaScript: 1 arquivo(s), 377 kB; 119 kB pela
rede
  pedidos do navegador à API: auth/me, patients/p1
  CNS na tela em 1486 ms (mediana de 3)
Next · next cap17 · next start
  CNS no HTML recebido: sim
  JavaScript: 7 arquivo(s), 467 kB; 141 kB pela
rede
  pedidos do navegador à API: nenhum
  CNS na tela em 375 ms (mediana de 3)
```

No Next, o CNS vem no HTML e o navegador não fala com a API; quem fala é o servidor do Next, perto dela. Na SPA são três esperas em série: o JavaScript, "quem entrou?", o paciente. Mas o Next mandou *mais* JavaScript, 141 kB contra 119 kB: Server Components tiram do navegador o código dos *seus* componentes, não o do framework. E a página da SPA tem a busca ao lado da ficha; o que a medição mostra é a ordem das esperas.

H+ **A cozinha central e a copa do andar**

A cozinha central monta a bandeja e a sobe pelo monta-carga. Na copa do andar, a copeira confere a etiqueta com a prescrição e completa o que se faz ali. Se a etiqueta não bate, ela refaz a bandeja. A cozinha é o servidor; a copa, a hidratação.

ONDE A ANALOGIA PARA

A copeira tolera um item fora de ordem. O React compara texto e atributos exatos: qualquer diferença entre o HTML do servidor e o primeiro render no navegador é erro.

17.3

Server Components

No Next, páginas e componentes são **Server Components** por padrão. Rodam só no servidor, podem ser `async`, leem cookie e banco, e mandam ao navegador só o resultado: HTML na primeira carga e um formato serializado (o *RSC payload*) nas navegações seguintes. A página entrega a casca e põe a ficha num `<Suspense>`:

app/pacientes/[id]/Ficha.tsx cap17

```
…
export async function Ficha({ params }:
  { params: Promise<{ id: string }> }) {
```

```
  const { id } = await params;
  const sessao = (await cookies())
    .get("__Host-sessao")?.value;
  const volta =
    `/entrar?volta=/pacientes/${id}`;
  if (!sessao) redirect(volta);

  const r = await obterPaciente(id, sessao);
  if (r.tipo === "sem-sessao")
    redirect(volta);
  if (r.tipo === "nao-existe") notFound();
  const p = r.paciente;
"""
```

O servidor do Next lê o cookie e o repassa à API: é o BFF do capítulo 15, sem configuração a mais. A API continua decidindo. Server Components são do React, mas exigem um framework ou bundler que os suporte (Next, React Router, Waku, Parcel, o plugin de RSC do Vite). Numa SPA, não existem. E componente cliente não pode ser `async`. `<Suspense>`, as diretivas e a hidratação também são do React; casca com buracos em streaming, `loading.tsx`, `"use cache"` e rotas por pastas são convenções do Next.

17.4

'use client' marca uma fronteira

O botão Copiar CNS precisa de estado e de clique, então mora num arquivo que começa com `"use client"`. A diretiva não marca um componente, marca uma **fronteira** no grafo de módulos: tudo o que esse arquivo importa vai junto para o navegador. Por isso ela

fica no ponto mais baixo da árvore. O que atravessa a fronteira são props serializáveis: texto, número, objeto simples, data, JSX já renderizado. Função, não, a não ser uma Server Function (`"use server"`). Um Server Component pode entrar num componente cliente como `children` , já renderizado.

17.5

O erro clássico: hidratação que diverge

Na tag `cap17-antes` , o agente recebeu "mostre a data de nascimento no formato de quem está vendo" e fez um componente cliente:

app/pacientes/[id]/Nascimento.tsx cap17-antes

```
"use client";

// Data no formato de quem está vendo.
export function Nascimento({ iso }:
  { iso: string }) {
  return (
    <>{new Date(iso).toLocaleDateString()}</>
  );
}
```

Componente cliente também roda no servidor, para gerar o HTML da primeira carga. Lá, idioma e fuso são os da máquina; no navegador, os da pessoa. O roteiro põe o servidor em UTC e inglês, como na nuvem, e o navegador em Fortaleza:

medicoes/cap17.py hidratacao · Medido

```
servidor: TZ=UTC, LANG=en_US.UTF-8 · navegador:
pt-BR, America/Fortaleza
cadastro de Maria: nascimento 1958-03-14

next cap17-antes
  no HTML do servidor: 3/14/1958
  na tela, depois de hidratar: 13/03/1958
  erro: Minified React error #418

next cap17
  no HTML do servidor: 14/03/1958
  na tela, depois de hidratar: 14/03/1958
  erros: nenhum
```

O erro 418 do React é "o texto do servidor não bate com o do cliente": o React refaz aquele trecho no navegador, que mostra 13 de março. Dois defeitos: a hidratação divergiu, e o dia está errado, porque `new Date("1958-03-14")` é meia-noite em UTC, ainda dia 13 em Fortaleza. A correção tira o `Date` do caminho e devolve a data ao servidor:

lib/datas.ts — cap17

```ts
// "1958-03-14" → "14/03/1958". Sem Date: não
// depende de fuso nem de idioma, então o
// servidor e o navegador escrevem o mesmo.
export function dataBr(iso: string) {
  const [ano, mes, dia] = iso.split("-");
  return `${dia}/${mes}/${ano}`;
}
```

CAP17-ANTES

`× HTML e tela com o mesmo texto`
`× o dia do cadastro`
`× console sem erro`

CAP17

`✓` os três

Vale para tudo o que depende de onde o código roda: `Date.now()`, `Math.random()`, idioma, fuso, `window`. Ou o valor é calculado num lugar só e passado como prop, ou aparece depois de hidratar, num efeito. `suppressHydrationWarning` silencia; não corrige.

17.6

Segredos ficam do lado de lá

`lib/api.ts` sabe onde a API mora e como repassar o cookie; não tem o que fazer no navegador. A primeira linha dele é `import "server-only"`. Um agente "reaproveitou" `obterPaciente` no botão:

```
medicoes/cap17.py fronteira · Medido

$ next build   (código de saída 1)
Error: 'server-only' cannot be imported from a
Client Component module
  Client Component Browser:
    ./lib/api.ts [Client Component Browser]
    ./app/pacientes/[id]/CopiarCns.tsx [Client
Component Browser]
```

O build parou, com a trilha de quem importou quem. Sem aquela linha, passaria. Segundo a documentação do Next, variável sem o prefixo `NEXT_PUBLIC_` vira texto vazio no cliente, mas o código vai inteiro, e com ele qualquer segredo escrito no código.

17.7

A fronteira é uma porta

Server Functions transformam um pedido HTTP numa chamada de função no servidor. A fronteira é uma porta HTTP, e foi por ela que veio a falha mais grave do ecossistema em 2025.

DADO · AS FALHAS DE DEZEMBRO DE 2025

Em 3 de dezembro, o React publicou a CVE-2025-55182 (CVSS 10,0): um pedido forjado, sem autenticação, executava código no servidor por uma falha na decodificação dos dados enviados a Server Functions. Afetava `react-server-dom-*` 19.0 a 19.2.0, mesmo em apps sem Server Functions próprias; corrigida em 19.0.1, 19.1.2 e 19.2.1. Em 11 de dezembro vieram negação de serviço (CVSS 7,5) e exposição do código-fonte de Server Functions (CVSS 5,3), com correção definitiva em 19.0.4, 19.1.5 e 19.2.4. Apps React sem servidor não foram afetados.

Três lições. Server Function valida entrada e permissão como qualquer rota, porque é uma. Segredo não fica escrito no código, nem no servidor. E o framework entra na rotina de atualização com a urgência da API.

REGRAS DO LABORATÓRIO NEXT

Componente é do servidor até precisar de estado ou evento; `"use client"` só na ilha. Render igual nos dois lados. Módulo com dados ou configuração do servidor começa com `import "server-only"`. Server Function valida como rota.

17.8

Com agentes de IA

ARMADILHA DE IA

Diante de "useState só funciona em componente cliente", o agente põe `"use client"` no topo da página, e ela vai inteira para o navegador. Diante do erro de hidratação, `suppressHydrationWarning`. E busca dados com `useEffect`, como numa SPA. O build não pega nenhum dos três; o console e o JavaScript da rota pegam.

PROMPT · AGENTE · CAP 17

```
Goal
Ficha de João com a idade, no laboratório
Next.

Context
@app/pacientes/[id]/ @lib/

Constraints
- a idade é calculada no servidor
- "use client" só no que tem evento
- nada de suppressHydrationWarning
- lib/ com dados: import "server-only"

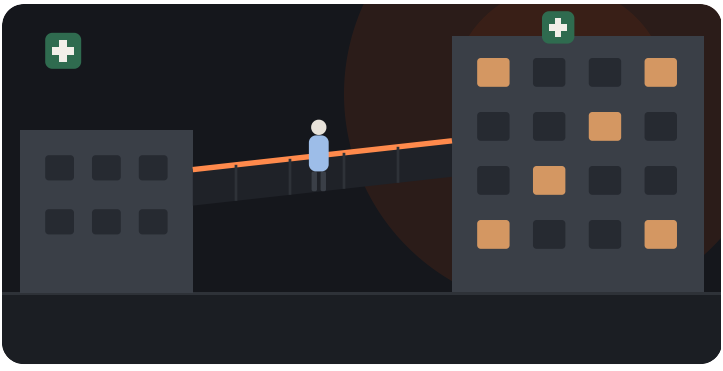
Done when
- next build sem erro
- cap17.py hidratacao: console sem erro
- mesma quantidade de JS na rota
```

Exercícios

ESSENCIAL	Abra o código-fonte da página (Ctrl+U) da ficha na SPA e no Next. Onde está o CNS em cada um?
INTERMEDIÁRIO	Mostre "Última consulta há N dias" na ficha sem erro de hidratação. Onde o "hoje" é calculado?
DESAFIO	Ponha `"use client"` no topo da `Ficha` e explique cada erro do build.

Régua: 32. **Tags:** `cap17-antes`, `cap17` (repositório `estado-e-tela-next`); tela `cap16`; API `cap15`. **Instrumentos:** `cap17.py`.

Fontes. react.dev: *Server Components*, *'use client'*, erro 418; blog, 3 e 11 dez. 2025. Next.js 16.4: *Server and Client Components*, *Environment Variables*. Saídas em `saidas/c17-*.txt`.



PARTE V · RENDERIZAÇÃO MODERNA

18

Next.js: visão geral e próximo livro

Um mapa, não um curso. O suficiente para decidir quando usar, e para saber o que vem depois.

O React desenha a tela. O Next.js decide onde e quando ela é desenhada, cuida de rotas, dados e cache, e põe um servidor ao lado do seu código. Aqui vai o mapa, no laboratório do capítulo 17, e uma tabela de decisão.

18.1

O que o Next acrescenta ao React

PEÇA	O QUE É NO NEXT 16
Rotas	Pastas em `app/` : `page` , `layout` , `loading` , `error` , `not-found` . Layout aninhado não renderiza de novo ao trocar de página
Renderização	Server Components por padrão, streaming com `<Suspense>` , casca estática com partes dinâmicas
Dados e cache	`fetch` e banco no servidor; `"use cache"` com prazo (`cacheLife`) e invalidação por etiqueta
Servidor	Route Handlers (endpoints HTTP), Server Functions (`"use server"`), `proxy.ts` antes de cada pedido

18.2

O mapa no laboratório

A tag `cap18` do laboratório acrescenta quatro peças: um `proxy.ts` , um layout em `/pacientes` , um Route Handler em `/saude` e uma página `/sobre` em cache. O próprio build diz como cada rota será servida:

```
medicoes/cap18.py · Medido

$ next build   (next cap18)
Route (app)              Revalidate  Expire
┌ ○ /
```

```
├ ○ /_not-found
├ ○ /entrar
├   /pacientes/[id]
│ └ ◐ /pacientes/[id]
├ ƒ /saude
└ ○ /sobre                      1d      1w
ƒ Proxy (Middleware)

○  (Static)             prerendered as static
content
◐  (Partial Prerender)  prerendered as static HTML
with dynamic server-streamed content
ƒ  (Dynamic)            server-rendered on demand
```

`○` é HTML pronto no build; `/sobre` também, com prazo de um dia, porque usa `"use cache"`. `◐` é a ficha: casca pronta, e a parte que lê o cookie chega em streaming. `ƒ` roda a cada pedido. É a figura 17.1, impressa pelo framework para cada rota.

proxy.ts cap18

```ts
// Roda antes de renderizar, em toda rota do
// matcher. É uma checagem otimista: só vê
// se o cookie existe. Quem valida a sessão
// continua sendo a API.
export function proxy(req: NextRequest) {
  if (req.cookies.has("__Host-sessao"))
    return;
  const volta = req.nextUrl.pathname;
  const login = new URL("/entrar", req.url);
  login.searchParams.set("volta", volta);
  return NextResponse.redirect(login);
}
```

```
export const config =
  { matcher: "/pacientes/:caminho*" };
```

```
medicoes/cap18.py · Medido

GET /pacientes/p1, sem cookie: 307 → /entrar?
volta=%2Fpacientes%2Fp1
GET /pacientes/p1, cookie inventado: 200; o HTML
traz o redirecionamento para /entrar?
volta=/pacientes/p1
GET /saude: 200 {"next":"ok","api":"ok"}
```

Sem cookie, o proxy responde 307 antes de renderizar, como o middleware do capítulo 15. Com um cookie inventado, deixa passar: só confere se o cookie existe. Quem recusa é a API, dentro da ficha, e como a resposta já tinha começado com 200 e a casca, o redirecionamento chega dentro do HTML. A documentação do Next chama o proxy de checagem *otimista*, que não deve ser a autorização. Quem decide é quem guarda os dados.

18.3

Dados, cache, autenticação e implantação

Dados e cache. Com Cache Components, ligado nos projetos novos do Next 16, nada vai para o cache sem pedido. Dado que depende do pedido (cookie, parâmetro) fica num `<Suspense>` e chega em streaming; o que pode ser reaproveitado leva `"use cache"` e um prazo (em `/sobre`: `cacheLife("days")`),

e uma mutação invalida a etiqueta. É o raciocínio do capítulo 14, no servidor.

Autenticação. As regras do capítulo 15 continuam: cookie HttpOnly, checagem no servidor, perto dos dados, numa camada de acesso a dados que toda leitura atravessa; o proxy é atalho. **Implantação.** Servidor Node (`next start`) ou contêiner suportam tudo; a exportação estática perde streaming, Server Functions e proxy. A SPA do Vite é só arquivos; um Next completo precisa de um processo rodando.

18.4

Vite SPA ou Next.js

A escolha não é de moda. Três perguntas resolvem quase todos os casos. A primeira visita precisa mostrar dados rápido, ou ser indexada? Já existe uma API separada, com dono? Quem opera um servidor Node em produção? Com "não", "sim" e "ninguém", a SPA basta. O resto do livro vale nos dois. Cinco cenários:

CENÁRIO	ESCOLHA E MOTIVO
Sistema interno com login (o Mini-Prontuário)	**SPA.** Ninguém indexa; a API já existe; menos peças para operar
Site público do hospital, com notícias e serviços	**Next.** HTML pronto para buscadores e para a primeira visita; SSG com prazo
Portal do paciente no celular, com rede ruim	**Next.** O dado vem no HTML; medido no capítulo 17: CNS em 375 ms contra 1486 ms
Painel de leitos em tempo real	**SPA.** Quase tudo é interação e atualização no cliente
Produto novo, time pequeno, sem backend separado	**Next.** Tela e servidor num projeto. Cada Server Function é uma rota a validar (17.8)

18.5

O próximo livro

O Next.js ganha um livro próprio na série, no mesmo formato: um projeto que cresce, medições e agentes sob portões. Ele começa onde este termina e pressupõe:

- ✔ onde cada estado mora; efeito só para sincronizar (caps. 6 a 14)
- ✔ rotas aninhadas, loader e redirecionamento interno (cap. 13)

- [x] sessão em cookie, 401 diferente de 403, esconder não é proibir (cap. 15)
- [x] teste pela tela, rede simulada na fronteira, uma jornada E2E (cap. 16)
- [x] fronteira servidor/cliente, hidratação e `server-only` (cap. 17)

18.6

Com agentes de IA

ARMADILHA DE IA

O agente ainda escreve `middleware.ts` (virou `proxy.ts` no Next 16), `getServerSideProps` (do Pages Router) e conta com cache implícito em `fetch`. O Next 16 traz a documentação no pacote, em `node_modules/next/dist/docs`: mande o agente ler ali antes.

```
PROMPT · AGENTE · CAP 18

Goal
Página /pacientes com a lista, no
laboratório Next.

Context
@node_modules/next/dist/docs/01-app
@app/pacientes/ @lib/api.ts

Constraints
- Server Component; busca no servidor
```

```
- a busca na URL (?q=), sem useState
- proxy.ts não autoriza; a API sim

Done when
- next build: /pacientes como ◐ ou ƒ
- sem cookie: 307 para /entrar
```

Exercícios

ESSENCIAL	Rode `next build` na tag `cap18` e explique o símbolo de cada rota.
INTERMEDIÁRIO	Faça `/sobre` mostrar o total de pacientes, em cache por uma hora. O que muda no build?
DESAFIO	Responda as três perguntas de 18.4 para um projeto seu.

Régua: 33. **Tags:** `cap18` (repositório `estado-e-tela-next`); API `cap15`. **Instrumentos:** `cap18.py`.

Fontes. Next.js 16.4 (docs no pacote): *Caching*, *Proxy*, *Route Handlers*, *Authentication*, *Deploying*. Saídas: `saidas/c18-*.txt`.